Small Language Models as the Substrate of Agentic AI: A Survey of Architectures, Compression, Deployment, and Evaluation Practice

Mohammed Gamal Ragab [1,2*], Hitham Alhussian [1,2*], Said Jadid Abdulkadir [1,2], and W. Eltayeb Ahmed [4]

[1]Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[2]Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[4]Department of Mathematics and Statistics, College of Science, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia
[*]Corresponding authors: mohammed.ragab@utp.edu.my (M.G.R.); seddig.alhussian@utp.edu.my (H.A.)

Abstract

Large Language Models (LLMs) with hundreds of billions of parameters have advanced natural language processing dramatically, but their cost, memory demand, and latency make them a poor fit for the fastest-growing consumer of language intelligence: the AI agent, which issues many narrow, repetitive, well-specified model calls rather than a few open-ended ones. Small Language Models (SLMs) have closed much of the capability gap through knowledge distillation, synthetic-data curation, and post-training quantization. This paper is a combined survey and position statement, and it fixes a definition the literature currently leaves loose: an SLM is a decoder-only or hybrid transformer bounded at roughly twenty billion parameters on *both* active parameters per token, which governs latency and energy, and total parameters, which governs storage and resident memory. Models meeting only the first bound are distinguished as small-active-parameter mixtures of experts and excluded from on-device claims. We catalog 215 publicly documented SLM variants (2022–2026), including the 2025–2026 wave of hybrid state-space, unified-reasoning, and mobile-first multimodal families, under a multi-label faceted taxonomy, and release the inventory in machine-readable form. We review compression for the sub-10B regime, reporting what each technique reduces along five separate axes rather than as one ratio, and finding that quantization outperforms pruning below 4B parameters. We then audit the public evaluation record for an 18-model cohort and report a negative result: a cross-model accuracy table cannot be assembled from current public reporting without misrepresenting its sources, because vendors report non-intersecting benchmark sets, shot counts that differ per benchmark and per vendor, base and instruction-tuned figures that are routinely conflated, and few matched baselines. Against that record we state the agentic case as a conditional hypothesis rather than a universal claim: for calls that are bounded, recurrent, short-context, and cheaply verifiable, a specialized SLM is the appropriate default execution substrate, with a frontier model on escalation. We grade the evidence for each premise using function-calling, tool-use, structured-output, and end-to-end agent benchmarks rather than general model scores, finding it strong at the level of the individual call and absent at the level of cost per completed task, and we state the workload classes for which the hypothesis fails. We survey heterogeneous SLM–LLM routing and the LLM-to-SLM conversion pipeline, and close with the open problems and a specification of the six experiments that would supply the controlled evidence this field does not yet have.

Highlights

- Fixes a two-condition definition of a small language model that separates per-token cost (active parameters) from residency cost (total parameters), and distinguishes small-active-parameter mixtures of experts from SLMs proper.
- Surveys 215 documented small language model variants (2022–2026), including the May–September 2026 release wave, under a multi-label faceted taxonomy, released in machine-readable form with per-record C1/C2 classification.
- Audits the public evaluation record for 18 SLMs and shows that it does not support the cross-model accuracy comparisons routinely published in surveys, leaderboards, and secondary sources.
- Reviews compression for the sub-10B regime, separating parameter, memory, training-compute, latency, and quality effects rather than collapsing them into one ratio, and finding that quantization outperforms pruning below 4B parameters.
- States the agentic case as a conditional hypothesis, grades the evidence for each premise against agent-specific benchmarks, and sets out the workload classes for which the hypothesis fails.
- Specifies six experiments that would supply the controlled evidence the field currently lacks, including the cost-per-completed-task comparison the economic argument turns on.

Keywords: Small language models • Agentic AI • Model compression • Edge AI • On-device inference • LLM–SLM collaboration

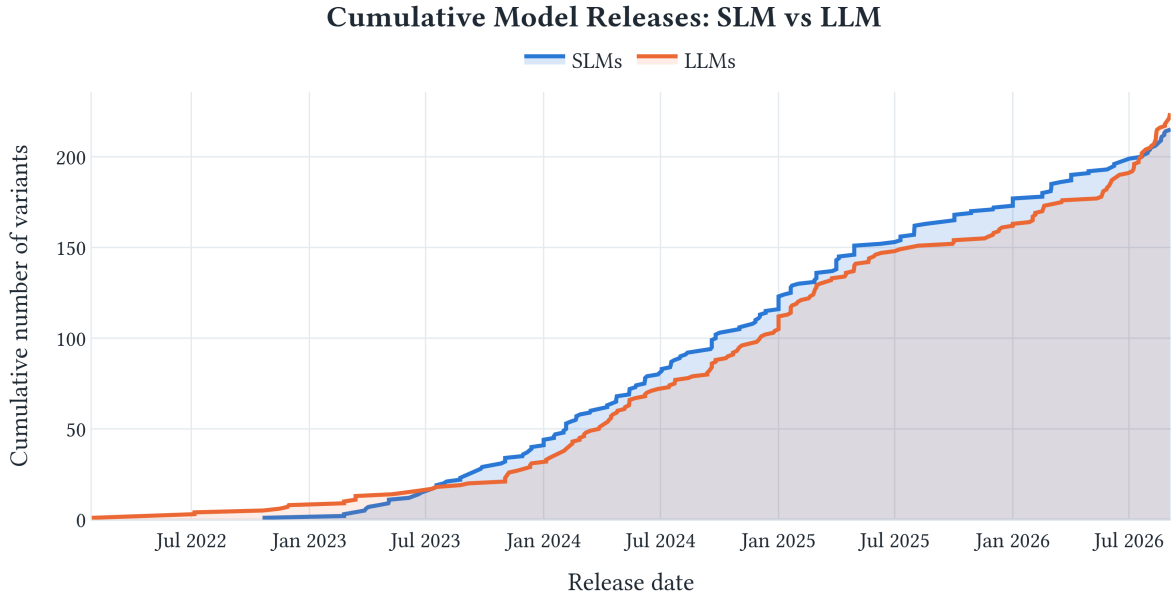


Figure 1. Cumulative count of documented small (SLM) and large (LLM) language-model variants by release date, from the model inventory of Appendix B (snapshot 5 September 2026). SLM releases accelerate sharply from mid-2024.

1 Introduction

The landscape of natural language processing (NLP) has been reshaped by large language models (LLMs). Systems such as GPT-4 [1], Claude [2], Gemini [3], and the Llama series [4] demonstrate broad competence across reasoning, code generation, and multilingual tasks. That competence, however, comes at extraordinary cost. GPT-3's 175 billion parameters [5] inaugurated an era in which scale was treated as synonymous with capability, and scaling laws [6, 7] appeared to promise predictable returns on ever-larger models. Frontier LLMs now require multi-thousand-GPU clusters to train, tens of gigabytes of accelerator memory to serve, and cloud infrastructure that introduces latency, recurring cost, and privacy exposure. Training GPT-3 is estimated to have consumed 1,287 MWh and emitted roughly 552 metric tons of CO_2 equivalent [8]; a single frontier-model query draws on the order of ten times the energy of a conventional web search [9].

1.1 The Rise of Small Language Models

Running against this trajectory is a counter-trend: small language models (SLMs) that achieve disproportionately strong performance for their size. Section 1.2 fixes the definition we use throughout; informally, an SLM is an autoregressive language model small enough to be served on a single consumer accelerator, laptop, or phone. Where earlier compact models such as DistilBERT [10] and TinyBERT [11] were task-specific encoders, the current generation are general-purpose, instruction-following systems. The Chinchilla scaling analysis [7] showed that most large models were badly undertrained and that a smaller model given proportionally more data could match them; the Phi family then demonstrated that "textbook-quality" synthetic data lets a 3.8B model rival models an order of magnitude larger [12], Gemma applied distillation from larger teachers [13], and Qwen2.5 pushed sub-7B training to 18 trillion tokens [14]. MIT Technology Review named SLMs one of its 10 Breakthrough Technologies of 2025 [15]. The 2025–2026 release wave adds hybrid state-space architectures [16], controllable-reasoning models [17], and mobile-first multimodal families [18, 19]. Figure 1 shows the resulting acceleration: the cumulative count of documented SLM variants has grown from one in 2022 to 215 by September 2026, with 43 of them released in 2026 alone.

1.2 Definitions and Terminology

Survey work on this class of models is complicated by the fact that "small" is used to mean at least four different things. We therefore fix the terminology before using it, and hold to it for the rest of the paper.

1.2.1 The Operative Definition

A model is a *small language model* for the purposes of this survey when it satisfies both of the following:

C1 (compute envelope)
It is an autoregressive decoder-only or hybrid attention/state-space model whose *active* parameter count per generated token is at most 2×10^{10} (roughly twenty billion).

C2 (residency envelope)
Its *total* parameter count is also at most roughly twenty billion, so that its weights can be resident in the memory of a single consumer accelerator, laptop, or high-end phone at 4-bit precision.

C1 bounds per-token latency, FLOPs, and energy. C2 bounds storage, download size, and resident memory. The two coincide for dense models, which is why a single threshold has been sufficient in most prior work, but they come apart for mixture of experts (MoE) models, and the distinction matters for deployment: a 119B-total model that activates 6.5B parameters per token has the inference economics of a 6.5B model and the storage and memory economics of a 119B one. Reporting only active parameters, as several vendor announcements and prior surveys do, systematically understates what it costs to put such a model on a device.

1.2.2 Four Distinguished Terms

SLM A model satisfying both C1 and C2. This is the class the survey is about, and the class to which every deployment, memory, and on-device claim in this paper applies.

Compact model
An SLM at or below roughly 3B total parameters. This is the band in which task-specific adaptation stops being optional (Section 11) and in which 4-bit weights fit in approximately 2 GB.

Edge model
A model, of any size, for which the developer documents a target of phone-, single-board-computer-, or neural processing unit (NPU)-class hardware, and ships an artifact for it. "Edge" is a claim about the intended deployment target, not about parameter count, and the two are correlated but not identical.

Small-active-parameter MoE
A model satisfying C1 but not C2: at most roughly twenty billion active parameters per token, but more in total. These have SLM-like per-token cost and LLM-like residency cost. They are retained in the inventory, because they are a live and growing part of the design space, but they are labelled separately and are excluded from every claim in this paper about storage footprint, download size, resident memory, or on-device feasibility.

Of the 215 variants in the inventory (Appendix B), 183 satisfy both C1 and C2 and are SLMs without qualification; 27 report an active-parameter count at or below the threshold against a total above it and are the small-active-parameter MoE group; and 5 are vendor-positioned small models whose parameter counts are not published at all, retained on the vendor's own positioning and marked as undisclosed rather than assumed. The 18-model cohort whose public evaluation record is examined in Section 5 is a strict subset of the 183, and is further restricted to below 10B total parameters with Phi-4 (14B) retained as a single upper-bound reference, because that is the band in which comparable public reporting actually exists.

One point about the count needs stating, because it changed during revision. Earlier drafts reported 126 variants. That figure reflected how records had been filed in the inventory rather than the inclusion rule as stated: an audit against C1 and C2 found 64 models that meet the rule, among them Llama-2-7B, Llama-3-8B, Qwen2.5-14B, OLMo 2-7B, Pixtral-12B, and Falcon 3, filed outside the SLM inventory, and two 24B dense models (Mistral Small 3.1 and 3.2) filed inside it. Placement is now derived from the rule rather than inherited, which is what takes the count to 215. The released inventory records the move per-record in its changelog, so the two counts can be reconciled exactly.

1.2.3 Reporting Dimensions

A single parameter count is not enough to determine whether a model will run on a given device. Table 1 lists the dimensions this survey reports where the primary source supplies them, and states which of them each of our claims depends on. Where a source omits a dimension, the entry is marked as not reported rather than estimated; Section 1.6 gives the coverage rates.

1.2.4 Model-Artifact Vocabulary

Because the inventory counts artifacts rather than papers, we also fix the following. A *family* is a named line of models sharing an architecture and a training recipe (Qwen2.5, Gemma 3). A *variant* is one released parameter size within a family together with one post-training status (Qwen2.5-3B base and Qwen2.5-3B-Instruct are two variants). A *checkpoint* is a specific set of weights published under a version identifier; a variant may be revised through several checkpoints, and the inventory records the variant, not each checkpoint. A *release* is a dated publication event covering one or more variants. A *pretrained* (or base) model is the artifact produced by pretraining

Table 1. Reporting dimensions for a small language model, the deployment question each answers, and the coverage achieved across the 215-variant inventory. "Derived" means the value follows arithmetically from others rather than being reported.

Dimension	Question it answers	Bounds	Inventory coverage
Total parameters	How large is the download?	C2	210/215
Active parameters per token	What does one token cost?	C1	35/215 stated; else = total
Weight memory at a precision	Will the weights fit?	C2	Derived
Peak runtime memory	Will it fit while running?	C2	Not reported by vendors
FLOPs per token	Compute cost per token	C1	Derived (approximate)
KV-cache size per token	Cost of long context	C1, C2	Derived from head config
Context length	Largest usable input	—	142/215
Target hardware	What did the vendor test on?	—	Vendor-stated only

alone; an *instruction-tuned* model has undergone supervised instruction fine-tuning and usually a preference-optimization stage; a *reasoning-tuned* model has additionally been trained, by distillation of long chain-of-thought traces, reinforcement learning, or both, to emit extended intermediate reasoning before its answer. These three post-training statuses are not interchangeable for evaluation purposes, and conflating them is one of the specific defects in the public record documented in Section 5.2.

Two conventions follow from this vocabulary and are used consistently throughout. Family names are written as their developers write them and with a space before the version, as in Gemma 3, Qwen2.5, Phi-4, and Llama 3.2; the original 2023 LLaMA release keeps its own capitalization, since Meta renamed the line to "Llama" from Llama 2 onward. Checkpoint identifiers are written in the hyphenated form used on the model hub, as in `Gemma-3-4B-it` or `Qwen2.5-7B-Instruct`, and are used wherever the specific artifact rather than the family is meant.

1.3 The Agentic Turn

At the same time, the way language models are consumed has shifted. The fastest-growing workload is not the chat session but the *AI agent*: a control loop in which a model is called repeatedly to choose a tool, fill its arguments, extract a field, classify an intent, or check a result [20, 21]. These calls are narrow, repetitive, and format-constrained, and they exercise only a thin slice of a frontier model's competence. Belcak, Heinrich, Diao, et al. [20] argue that for such workloads SLMs are not a compromise but the appropriate default: sufficiently capable for the delegated sub-task, cheaper to serve, cheaper to specialize, and easier to constrain to a code interface, with a strong generalist model reserved for the minority of calls that genuinely need planning or broad context. Surveys of LLM–SLM collaboration [22, 23] and production systems such as Apple's on-device/server split [19] indicate that this heterogeneous design is already the direction of travel.

1.4 The Thesis, Stated Conditionally

This paper is a survey with a position attached, and we state the position as a conditional design hypothesis rather than a universal claim, because that is what the available evidence supports.

> **H (substrate hypothesis).** For a language-model call inside an agent that is (i) *bounded*, in that the required output is a tool selection, a structured record, a label from a fixed set, or a short local edit; (ii) *recurrent*, in that calls of the same type arrive often enough to repay one-time specialization; (iii) *short-context*, in that the input needed to answer it fits well inside the model's window without retrieval over a large corpus; and (iv) *verifiable*, in that the surrounding scaffold can cheaply check whether the output is well-formed and, ideally, correct, a specialized SLM is the appropriate default execution substrate, and a frontier LLM should be invoked only on escalation.

The hypothesis is falsifiable and it is bounded. It does not claim that SLMs match frontier models in general, that scaling has stopped paying, or that agentic systems can dispense with large models. Section 7.4 sets out the workload classes for which the antecedent fails and a large model remains the right choice: open-ended planning over an unbounded action space, synthesis over long or heterogeneous context, tasks whose correctness cannot be checked without a stronger model, low-resource-language and safety-sensitive calls where small models have documented deficits, and any workload whose call distribution is too sparse or too unstable to specialize against. Section 7.3 grades the evidence for each premise of H, and is explicit that the evidence is strongest for premise (i) and weakest for the end-to-end claim.

1.5 Scope and Contributions

Existing SLM surveys [24–27] provide taxonomies and measurements but either predate the 2025–2026 model wave or treat agentic deployment only in passing; agentic position pieces [20] make the argument but do not ground it in a consolidated capability-and-deployment review. This paper joins the two. Its contributions are:

Landscape and taxonomy
A catalog of 215 documented SLM variants (2022–2026), including the recent hybrid, reasoning, and mobile-multimodal families, organized under a multi-label faceted taxonomy (Section 3 and Appendix B), with a machine-readable release (Section 1.6.4).

Compression review
A synthesis of distillation, quantization, pruning, and compound pipelines for the sub-10B regime, with the reduction achieved reported separately along parameter count, memory, training compute, and latency rather than collapsed into a single ratio, including the finding that quantization consistently outperforms pruning below 4B parameters [28] (Section 4).

An audit of the public evaluation record
An examination of what the public reporting record can actually support for an 18-model cohort, and the finding that a cross-model accuracy comparison cannot be assembled from it without misrepresenting the sources, because vendors report non-intersecting benchmark sets, differing shot counts per benchmark, inconsistently labelled base and instruction-tuned checkpoints, and few matched baselines (Section 5, Appendix A).

Deployment ecosystem
A survey of inference frameworks, quantization formats, and on-device platforms from Raspberry Pi to Apple silicon, with calculated and measured quantities kept apart (Section 6).

The agentic argument, with agent-specific evidence
A structured, evidence-graded case for hypothesis H, drawing on function-calling, tool-use, structured-output, and end-to-end agent benchmarks rather than general model scores, together with its stated counterexamples (Section 7).

Open problems and verification
An agenda of open research problems and targeted empirical-verification opportunities for the consolidated evidence base (Section 10).

1.6 Survey Method and Sources

This is a structured survey conducted under a documented protocol. It is not a PRISMA systematic review: the aim is representative coverage of the SLM design space and a complete audit of the public performance record for a defined cohort, not an exhaustive enumeration of every publication in the area. What follows is the protocol in full, so that the search can be repeated and its coverage assessed.

1.6.1 Research Questions

RQ1 What is the design space of small language models released between 2022 and 2026, and along which facets do the released artifacts actually differ?

RQ2 Which compression techniques are effective in the sub-10B regime, and along which axis (parameters, memory, training compute, latency, quality) does each one pay?

RQ3 What does the public evaluation record support about the relative capability of these models, and what does it not support?

RQ4 What does the evidence say about the suitability of small models for agentic workloads specifically, as opposed to general benchmarks?

RQ5 Under what conditions is a small model the appropriate default inside an agent, and under what conditions is it not?

1.6.2 Search Protocol

Three bodies of evidence were assembled, each under an explicit rule.

Model documentation (RQ1). Candidate variants were identified from four starting points: (a) the Hugging Face Hub model index, filtered to text-generation models with a published model card, sorted by downloads and by recency, and queried for the family names of every developer already known to ship in this band; (b) vendor release channels for Microsoft, Google DeepMind, Meta, Alibaba, Mistral AI, IBM, NVIDIA, TII, Apple, Hugging Face, AI2, Stability AI, DeepSeek, Liquid AI, and Cohere; (c) arXiv listings under cs.CL and cs.LG for technical reports announcing a model release; and (d) forward citation tracing from the four prior SLM surveys named below.

Table 2. Flow of records through retrieval and screening. Model-variant and technique-literature streams were screened separately against their respective criteria. Counts marked as pending are to be transcribed from the released screening ledger before submission.

Stage	Technique literature	Model variants
Records retrieved by search	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
After deduplication	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
Excluded at title/abstract screening	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
reason: out of parameter scope (T1 / M3)	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
reason: no stable identifier (T2 / M1)	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
reason: insufficient method detail (T3 / M2)	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
reason: derivative artifact (models only)	—	**[TO SUPPLY: count]**
reason: off-topic on screening	**[TO SUPPLY: count]**	—
Read in full	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
Excluded at full-text stage	**[TO SUPPLY: count]**	**[TO SUPPLY: count]**
Retained	**[TO SUPPLY: count]**	**215**

Searches were run over the collection window **[TO SUPPLY: search window: first and last search dates]** and cover releases dated up to **[TO SUPPLY: last release date included]**.

Technique literature (RQ2, RQ3). The ACL Anthology, arXiv (cs.CL, cs.LG), and the proceedings of NeurIPS, ICML, ICLR, EMNLP, ACL, and MLSys for 2019–2026 were searched with the Boolean query

```
("small language model" OR "compact language model" OR "efficient language model" OR "on-
device language model" OR "edge LLM") AND (survey OR quantization OR pruning OR distillation
OR "parameter-efficient" OR inference OR deployment OR agent OR "tool use" OR routing)
```

restricted to title, abstract, and keywords, seeded from Lu, Li, Cai, et al. [24], Nguyen, Shen, Aponte, et al. [25], Wang, Zhang, Zhang, et al. [26], and Belcak, Heinrich, Diao, et al. [20], and expanded by one round of forward and backward citation tracing from every retained work.

The evaluation record (RQ3). For each model in the 18-model cohort, the primary release document, the official model card, and any linked evaluation appendix were retrieved in full, together with community standardized evaluations and third-party reproductions where they existed. The collection and appraisal rules are set out in Section 5.1. No benchmark was executed for this survey.

1.6.3 Screening, Retrieval Counts, and Agreement

Deduplication was performed on normalized title plus first author, and then on arXiv identifier and DOI; the duplicate-identifier check is what surfaced the one bibliographic collision that survived into an earlier draft of this paper. Records were screened on title and abstract against the inclusion criteria, and retained records were read in full. Table 2 gives the flow of records through those stages; the per-record decisions are in the released screening ledger (Section 1.6.4).

Inclusion criteria for the technique literature were: (T1) the work concerns models at or below roughly twenty billion parameters, or a compression, adaptation, routing, or deployment technique whose reported evaluation includes that regime; (T2) it is available as a peer-reviewed paper, an archived preprint, or a vendor technical report with a stable identifier; and (T3) it reports enough methodological detail for a reader to determine what was measured. Exclusion reasons were recorded per record and are summarized in Table 2.

Inclusion criteria for model variants were: (M1) a citable primary source describing architecture and training; (M2) a public artifact or a documented evaluation card; (M3) satisfaction of C1, with C2 status recorded separately; and (M4) a dated release. Community fine-tunes, quantized re-uploads, and merges of other people's checkpoints were excluded, as were variants that differ from a listed variant only in serialization format.

Screening was carried out by **[TO SUPPLY: number]** of the authors working independently, with disagreements resolved by discussion; the agreement statistic at the title-and-abstract stage is **[TO SUPPLY: percent agreement and Cohen's κ]**. Quality appraisal of retained empirical works used four criteria, recorded per work in the released ledger: whether the evaluation harness is named, whether the evaluation conditions (shots, prompt, decoding) are stated, whether a matched baseline is reported, and whether the artifact evaluated is identified to the checkpoint. These four criteria are also what Section 5 audits the vendor record against, so the appraisal scheme and the audit instrument are deliberately the same.

1.6.4 Released Artifacts

To make the inventory checkable rather than merely stated, the following are released alongside the paper: the machine-readable variant inventory (`merged_llm_database.json`) with one record per variant carrying every field in Table 1 plus source URL, source retrieval date, license, and inclusion decision; the screening ledger recording every retrieved record and its inclusion or exclusion reason; the figure- and table-generation scripts, so that every generated figure in this paper can be reproduced from the inventory; and a changelog keyed to the snapshot dates. These are to be deposited in a versioned archival repository under a persistent identifier before submission, and that identifier is stated in the Data Availability declaration (Section 11). Because the repository is versioned, a reader can retrieve the exact snapshot underlying this paper as well as any later one, which is what makes the update policy of Section 1.6.5 checkable rather than merely promised.

1.6.5 Snapshot and Update Policy

This paper reports the state of the record as of 5 September 2026, with model releases through 4 September 2026. The field moves faster than a journal review cycle, so we state in advance how the snapshot is maintained. Releases appearing between submission and acceptance are added to the released inventory, with their addition recorded in the changelog, and are discussed in the text only where they change a conclusion rather than merely extending a list; the paper's tables and figures are regenerated from the inventory at the last revision before acceptance, and the snapshot date on each is updated accordingly. Works appearing during review that bear on RQ2–RQ5 are added to the related-work synthesis in Section 2. We do not claim the inventory is permanently current, and the released artifact is the authority on what it contained at any given date.

1.7 Paper Organization

Section 2 reviews prior work on the LLM-to-SLM transition, compression, edge inference, and LLM–SLM collaboration. Section 3 presents the faceted taxonomy. Section 4 reviews compression techniques. Section 5 documents the data-collection protocol and audits the public evaluation record. Section 6 covers resource-constrained deployment. Section 7 makes the agentic argument, grades its evidence, and states its counterexamples. Section 8 surveys domain-specific applications. Section 9 discusses implications, limitations, and ethics. Section 10 sets out open challenges and an empirical-verification agenda. Section 11 concludes.

Appendix B provides the full variant inventory, grouped chronologically by release year and family, together with the source-selection protocol; it reflects the public record as of September 2026.

2 Related Work and Research Gaps

2.1 Evolution from LLMs to SLMs

The modern era of language modeling began with the Transformer architecture [29], which replaced recurrent networks with self-attention mechanisms enabling efficient parallel training. BERT [30] demonstrated the power of pre-training on large corpora followed by task-specific fine-tuning, establishing a paradigm that dominated NLP from 2018 to 2020. The GPT series [1, 5, 31] progressively scaled this approach, with GPT-3 (175B parameters) revealing emergent capabilities, among them in-context learning and chain-of-thought reasoning, that appeared to arise only at sufficient scale. This observation motivated an aggressive scaling race: PaLM (540B) [32], Chinchilla (70B) [7], LLaMA (7B–65B) [33], Llama 2 (7B–70B) [34], Falcon (7B–180B) [35], and ultimately GPT-4 [1] and Gemini Ultra [3] pushed parameter counts into the hundreds of billions.

However, the scaling paradigm encountered critical challenges. Kaplan, McCandlish, Henighan, et al. [6] established power-law relationships between model performance and three factors: parameter count, training data size, and compute budget. Hoffmann, Borgeaud, Mensch, et al. [7] then revised these scaling laws with the Chinchilla study, demonstrating that for a fixed compute budget, models and training data should be scaled in equal proportion. Their 70B-parameter Chinchilla model, trained on 1.4 trillion tokens, outperformed the 280B-parameter Gopher model trained on only 300 billion tokens. This finding implied that most existing LLMs were substantially under-trained and that smaller, better-trained models could achieve equivalent performance at lower cost. Recent work has further validated and refined this principle: the Qwen2.5 series trains its 7B model on 18 trillion tokens [14], over 2,500× the parameter count in tokens, far exceeding the Chinchilla-optimal ratio and demonstrating that the benefits of extended training persist well beyond theoretical optima.

The counter-trend toward smaller models accelerated in 2023–2024 with the release of several SLM families that set the pattern for what followed. Microsoft's Phi-1 (1.3B) [36] demonstrated that a small model trained on carefully curated "textbook-quality" data could achieve surprisingly strong performance on code generation tasks. Phi-2 (2.7B) [37] extended this to general language understanding, matching or exceeding models with 25× more parameters on certain benchmarks. The Phi-3 family [12] formalized the synthetic-data-first training philosophy,

with Phi-3-mini (3.8B) achieving performance comparable to Mixtral 8 × 7B (46.7B total parameters) on Massive Multitask Language Understanding (MMLU). Google's Gemma [13] and Gemma 2 [38] used distillation from Gemini models, while Meta's Llama 3.2 [4] introduced 1B and 3B variants specifically designed for edge deployment. By early 2026, the Phi-4-mini (3.8B) achieves 88.6% on Grade School Math 8K (GSM8K) mathematical reasoning [39], a score that would have placed it among the top models at any parameter count in 2023.

2.2 Knowledge Distillation and Model Compression

Knowledge distillation, first formalized by Hinton, Vinyals, and Dean [40], transfers knowledge from a large "teacher" model to a smaller "student" by training the student to match the teacher's soft probability distributions (logits) rather than hard labels. This approach preserves the teacher's learned representations of inter-class similarities, information that is lost in one-hot labels, and has proven remarkably effective in the LLM era. Modern distillation strategies for language models encompass several complementary approaches.

Logit-based distillation trains the student to minimize the KL divergence between its output distribution and the teacher's softened distribution, parameterized by a temperature τ. Google's Gemma 2 [38] and Gemma 3 [41] employ this approach extensively: all Gemma 3 model sizes, including the 27B variant, are trained using knowledge distillation from a larger teacher. The distillation loss is combined with a standard next-token prediction loss, with the relative weighting serving as a key hyperparameter. Feature-based distillation [42] additionally aligns intermediate representations between teacher and student, capturing structural knowledge that may not be fully expressed in output logits. Microsoft's Orca 2 [43] demonstrated a variant where a smaller model is taught to use different reasoning strategies depending on task complexity, outperforming models with 5–10× more parameters on reasoning benchmarks.

Synthetic data generation represents an increasingly dominant form of implicit distillation. Rather than direct logit matching, a powerful LLM (e.g., GPT-4, Claude 3.5) generates high-quality training examples that are then used to train smaller models. The Phi series pioneered this approach with the "Textbooks Are All You Need" methodology [36], generating synthetic textbook-style content with carefully controlled complexity and diversity. Phi-4's training data comprises approximately 40% synthetic data generated across 50 distinct dataset types, including multi-agent conversations, structured reasoning chains, and domain-specific problem sets [44]. This synthetic-data-first approach has proven more effective than distillation from raw web corpora, particularly for STEM and reasoning tasks.

Post-training compression techniques enable further size reduction without retraining. Quantization reduces the numerical precision of model weights from 32-bit or 16-bit floating point to lower bit-widths. GPTQ [45] performs layer-wise quantization using approximate second-order (Hessian) information, achieving INT4 precision with minimal perplexity increase. activation-aware weight quantization (AWQ) [46] identifies and protects the small fraction (∼1%) of salient weights determined by activation magnitudes, achieving near-lossless INT4 quantization and winning the MLSys 2024 Best Paper award. GPT-generated unified format (GGUF), developed for the llama.cpp ecosystem [47], provides block-based K-quant schemes (Q4_K_M, Q5_K_M, Q8_0) optimized for CPU inference with fine-grained quality–size trade-offs. SqueezeLLM [48] introduced a sensitivity-based non-uniform quantization scheme with dense-and-sparse decomposition, demonstrating that sensitivity varies dramatically across weight dimensions.

Complementary compression techniques include structured and unstructured pruning, which remove redundant weights or entire attention heads [49]; low-rank adaptation (LoRA) [50] and its 4-bit variant quantized low-rank adaptation (QLoRA) [51], which enable parameter-efficient fine-tuning by learning low-rank update matrices while keeping the base model frozen; and vocabulary pruning, which reduces embedding table size for domain-specific deployments. The combination of distillation during training, quantization during deployment, and LoRA-based adaptation for task specialization forms the modern SLM optimization pipeline.

2.3 Efficient Fine-Tuning Methods

Full fine-tuning updates all model parameters using task-specific data, which for a 7B-parameter model can require tens of gigabytes of accelerator memory once gradients and optimizer states are included. This cost has made parameter-efficient fine-tuning (PEFT) central to the practical adaptation of SLMs, particularly when the target deployment environment is a single GPU or an edge-oriented workstation.

LoRA [50] addresses this constraint by freezing the pretrained backbone and learning low-rank update matrices for selected linear layers. For a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is parameterized as $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters while preserving the ability to specialize a model for downstream tasks. QLoRA [51] extends this idea by quantizing the frozen base model to 4-bit NormalFloat (NF4) precision and training the adapters in higher precision, allowing models that would ordinarily require multi-GPU setups to be fine-tuned on a single high-memory accelerator.

Beyond LoRA-style adapters, the PEFT landscape includes prompt-based and prefix-based adaptation strategies that shift task information into learned soft prompts or attention prefixes rather than updating the backbone itself. These approaches can be attractive when multiple lightweight task specializations must be maintained simultaneously. However, they tend to underperform LoRA-family methods on complex reasoning tasks, where direct adaptation of attention and feed-forward projections provides a stronger inductive bias for task transfer.

The choice between prompting and fine-tuning therefore depends on task specificity, data availability, and deployment constraints. In-context learning offers zero-training adaptation and is well suited to broad tasks when labeled data are scarce, but it consumes context length and raises inference latency. By contrast, PEFT methods incur an upfront training cost yet reduce per-query overhead and typically yield larger gains on narrow or reasoning-heavy tasks. This trade-off motivates the comparative synthesis in Section 5, where reported prompting and QLoRA results are placed side by side.

2.4 Overview of Key SLM Families

2.4.1 Microsoft Phi Series

The Phi series represents Microsoft's systematic exploration of data-quality-driven model development. Phi-1 (1.3B) [36] demonstrated strong code generation performance through training on synthetic "textbook-quality" data. Phi-2 (2.7B) [37] extended this to general language understanding. Phi-3-mini (3.8B) [12] formalized the approach with 3.3 trillion training tokens, achieving 68.8% on MMLU and 82.5% on GSM8K with instruction tuning. Phi-3.5-mini [52] added 128K context length support with improved multilingual capabilities. Phi-4 (14B) [44] introduced a fundamentally enhanced training pipeline incorporating ∼40% synthetic data, achieving 84.8% on MMLU and 80.4% on MATH, surpassing its teacher model GPT-4o on several STEM benchmarks. Phi-4-mini (3.8B) [39] updated the architecture with grouped-query attention (GQA) (24 query, 8 key–value (KV) heads), a 200K-token vocabulary, and training on 5 trillion tokens, achieving 88.6% on GSM8K. The Phi-4-reasoning variant [53] applied supervised fine-tuning on o3-mini reasoning demonstrations followed by Group Relative Policy Optimization (GRPO), reaching 82.5% on AIME 2025, competitive with DeepSeek-R1 at 671B parameters.

2.4.2 Google Gemma Series

Google's Gemma family [13] derives from the Gemini model architecture and training infrastructure. Gemma 1 offered 2B and 7B variants trained on 3T and 6T tokens respectively, using Multi-Query Attention (MQA) for the 2B model and standard Multi-Head Attention for the 7B. Gemma 2 [38] introduced significant architectural improvements: local-global interleaved attention layers, logit soft-capping, and knowledge distillation for the 2B and 9B variants (from a larger teacher model). Gemma 2-9B achieved 71.3% on MMLU (5-shot), establishing it as a strong 7B-class competitor. Gemma 3 [41] further advanced the architecture with a 5:1 local-to-global attention interleaving ratio (reducing the sliding window from 4096 to 1024 tokens), Rotary Position Embeddings, and logit soft-capping at 30.0 for attention and 50.0 for the final layer. Critically, Gemma 3 distills all model sizes from larger teachers and introduces a 1B variant that achieves 48.0% on MATH, close to Gemma 2-9B's 49.4% with 9× fewer parameters.

2.4.3 Alibaba Qwen Series

The Qwen2 [54] and Qwen2.5 [14] families from Alibaba represent the most extensively scaled SLM training effort, with models trained on 18 trillion tokens, substantially exceeding Chinchilla-optimal ratios. Qwen2.5 offers models at 0.5B, 1.5B, 3B, 7B, 14B, 32B, and 72B parameters, all sharing a consistent GQA-based architecture with rotary position embeddings (RoPE) and SwiGLU activations. The 3B variant achieves remarkable results: 65.6% on MMLU, 79.1% on GSM8K, and 42.6% on MATH, scores that establish it as arguably the strongest sub-4B base model available. Qwen2.5's training data includes substantial STEM and code content, explaining its particular strength on mathematical and programming benchmarks.

2.4.4 Other Significant SLM Families

TinyLlama (1.1B) [55] adopted the Llama 2 architecture and trained on 3 trillion tokens from SlimPajama and StarCoder data, demonstrating that extended training of very small models yields disproportionate gains. Mistral 7B [56] introduced Grouped-Query Attention and Sliding Window Attention in a 7.3B-parameter model, achieving performance competitive with Llama 2 13B while being 1.8× smaller. Meta's Llama 3.2 [4] released dedicated 1B and 3B models trained on 9 trillion tokens with 128K context length, specifically targeting edge deployment. SmolLM2 (135M, 360M, 1.7B) [57] from Hugging Face trained on 11 trillion tokens of carefully curated data. H2O-Danube (1.8B) [58] introduced sliding window attention in a compact model trained on 1 trillion tokens. OLMo 2 (1B, 7B, 13B) [59] from AI2 was released as a fully open model (data, code, weights, and training logs), achieving competitive performance through QK-norm and carefully staged training. StableLM-2 (1.6B) [60] was trained on 4 trillion tokens of multilingual data. The BLOOM family (560M, 1.1B, 1.7B, 3B, 7.1B) [61] remains notable as a multilingual

model trained on 46 languages using ALiBi positional encoding, though its 350B-token training budget is modest by current standards.

2.5 Edge AI and On-Device Inference

The edge AI inference stack has matured rapidly, transforming SLM deployment from a research exercise to a production-ready capability. The llama.cpp framework [47] is the most widely used runtime for CPU and mixed CPU-GPU inference, and the one against which the others in Table 13 position themselves, supporting the GGUF quantization format with dozens of precision variants (Q2_K through Q8_0). Its ecosystem includes llamafile (single-executable packaging), Ollama (user-friendly serving), and extensive platform support spanning x86, ARM, Apple Silicon, and NVIDIA GPUs. Benchmarking on single-board computers shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models [62].

Meta's ExecuTorch [63] reached version 1.0 GA in October 2025, providing a production-ready framework for on-device AI with a 50 KB base runtime and support for 12+ hardware backends including CoreML, Qualcomm QNN, ARM, MediaTek, and Vulkan. It is deployed in production across Instagram, WhatsApp, and Facebook, serving billions of users. MLC-LLM [64] provides universal deployment through ML compilation (TVM), supporting iOS, Android, web, and embedded platforms. ONNX Runtime [65] enables cross-platform enterprise deployment with optimization passes for diverse hardware targets. WebLLM [66] brings browser-based inference using WebGPU acceleration, retaining up to 80% of native performance with an OpenAI-compatible API. Google's LiteRT (formerly TensorFlow Lite) with QNN acceleration achieves 100× CPU speedup on Qualcomm NPUs [67], enabling sub-second time-to-first-token for sub-1B models on flagship smartphones.

The fundamental constraint in edge deployment is memory bandwidth rather than compute. Flagship mobile and laptop devices offer roughly 50–90 GB/s of memory bandwidth, and single-board computers well under 30 GB/s, versus 2–3 TB/s for datacenter GPUs [68]. Single-stream autoregressive decoding reads the full weight set once per generated token, so in that regime it is bandwidth-bound and quantization from 16 to 4 bits converts a storage saving into a decode-time saving roughly in proportion to the bandwidth freed. The proportionality is regime-dependent rather than general: it weakens under batching as arithmetic intensity rises, largely disappears during the compute-bound prefill phase, and is reduced further on hardware without native low-precision kernels. Section 6.4.1 sets out the conditions in full. Because single-stream interactive use is the dominant on-device regime, INT4 quantization is nevertheless a performance enabler there and not only a storage optimization.

2.6 LLM–SLM Collaboration and Agentic Systems

A distinct line of work studies how small and large models cooperate rather than compete. Chen and Varoquaux [22] frame the relationship along two axes, collaboration and competition, and argue that small models remain practically indispensable despite the scaling race. Chen, Zhao, and Han [23] organize the collaboration literature into five interaction patterns: pipeline, routing, auxiliary, distillation, and fusion. Routing is the most directly relevant to agents: RouteLLM [69] learns from preference data to send each query to a weak or strong model, reporting more than 2× cost reduction at matched quality, and Division-of-Thoughts [70] routes decomposed sub-questions between an on-device SLM and a cloud LLM. Surveys of edge–cloud collaborative inference [68] catalog the systems techniques (speculative decoding, cascades, KV-cache offload) that make the hybrid practical. On the agentic side specifically, Belcak, Heinrich, Diao, et al. [20] argue that SLMs should be the default model inside agents, and action-model families such as xLAM [71] show that sub-10B models can top function-calling leaderboards. This paper connects that argument to the evaluation and deployment review of Sections 5 to 7.

2.7 Gaps in Existing Literature

Several SLM surveys appeared in 2024–2025. Lu, Li, Cai, et al. [24] analyze 70 models on architecture, capability, and on-device runtime, but as a measurement study rather than a synthesis of the wider reported record. Nguyen, Shen, Aponte, et al. [25] and Wang, Zhang, Zhang, et al. [26] provide broad technique taxonomies and application coverage. All three predate the 2025–2026 wave of hybrid state-space, unified-reasoning, and mobile-multimodal models, and none foreground agentic deployment. Position pieces on SLMs in agents [20] make the forward-looking argument but do not ground it in a consolidated capability review.

Three more recent lines of survey work overlap this one and should be distinguished from it rather than merely listed. On *small reasoning models*, Srivastava, Cao, and Wang [27] benchmark 72 small models across 14 reasoning benchmarks under a common protocol and conclude that reasoning ability in this band tracks training method and data quality more than scale. That is a measurement study, and it is the closest thing in the literature to the controlled evaluation whose absence Section 5 documents; it is also narrower, covering reasoning only, and it does not treat compression, deployment, or agentic use. Feng, Fang, Ma, and Wang [72] survey efficient reasoning along three axes, shorter chains, smaller models, and faster decoding, which intersects our compression section at the

"smaller" axis and extends beyond it into decoding strategies we treat only in passing. On *agentic systems*, Chen, Liu, Han, Zhang, and Liu [21] survey LLM-based multi-agent systems by application, which is complementary: they ask what agent systems are built for, and we ask which model should execute the calls inside one. Our contribution is not to duplicate any of these but to connect the model landscape, the compression evidence, and the deployment ecosystem to a specific, conditional claim about where in an agent a small model belongs, and to audit whether the evidence base can carry it.

This paper therefore addresses five gaps: (1) a landscape and faceted taxonomy current to 2026, including hybrid and reasoning families, released in machine-readable form; (2) a compression synthesis that separates the axes along which each technique pays rather than collapsing them into one ratio; (3) an audit of whether the public evaluation record supports cross-model comparison at all, with documented provenance and reconciliation rules; (4) an evidence-graded bridge from that record to the agentic case, using agent-specific benchmarks rather than general model scores; and (5) a practitioner-facing decision and verification agenda. Table 3 positions this survey against its predecessors.

Table 3. Coverage of this survey relative to prior SLM surveys and position pieces (Y full, P partial, – absent). *Evaluation critique* denotes explicit analysis of whether the public reporting record supports cross-model comparison; *verification agenda* denotes a concrete specification of the experiments needed to close the resulting gaps.

Survey	Taxonomy	Compression review	Evaluation critique	Deployment / edge	Agentic focus	Verification agenda	Coverage cutoff
Lu, Li, Cai, et al. [24]	Y	P	P	Y	–	–	2024
Nguyen, Shen, Aponte, et al. [25]	Y	Y	–	P	–	–	2024
Wang, Zhang, Zhang, et al. [26]	Y	Y	–	P	–	–	2024
Belcak, Heinrich, Diao, et al. [20]	–	–	P	P	Y	–	2025
Srivastava, Cao, and Wang [27]	P	P	Y	–	–	P	2025
Feng, Fang, Ma, and Wang [72]	P	Y	–	P	–	–	2025
Chen, Liu, Han, Zhang, and Liu [21]	–	–	–	–	Y	–	2024
This survey	Y	Y	Y	Y	Y	Y	2026

3 A Faceted Taxonomy of Small Language Models

3.1 Why the Scheme Is Faceted and Multi-Label

We organize the SLM design space along four *primary* facets, *architecture*, *training strategy*, *modality*, and *application domain* (Figure 2), and five *secondary* facets that govern whether a model can actually be adopted (Table 4). Two properties of the scheme need stating up front, because earlier presentations of this taxonomy, including our own, got them wrong.

First, the facets are *not orthogonal*, and it would be misleading to present them as such. Training strategy and architecture are entangled: the pruning-plus-distillation route presupposes a dense teacher, and the mixture-of-experts route interacts with distillation through which experts are supervised. Modality and architecture are entangled through the encoder a vision-language model attaches. What the facets are is *separately observable*: each can be read off a model's primary source independently of the others, which is what makes them usable as an indexing scheme even though they covary.

Second, the scheme is *multi-label*, not a partition. A model may occupy several categories on one facet at once. Gemma 3 is distilled *and* trained on a curated corpus from scratch; Granite 4.0 is a hybrid state-space model *and* a mixture of experts; Qwen2.5-7B is used as a general-purpose model *and* as a multilingual and a code model. Forcing one label per facet, as a partition would, produces exactly the misclassifications a reader would then have to undo. In the inventory each variant therefore carries a set of labels per facet, and the counts in the figures that follow are counts of labels, not of models, except where stated.

The application-domain facet needs a further caveat. Unlike the other three, it is usually a statement about *how a model is used* rather than an intrinsic property of the artifact: a general-purpose 7B model fine-tuned by a hospital on clinical notes is a healthcare model in deployment and a general-purpose model on the Hub. We therefore assign a domain label only when the developer trained or tuned the released artifact for that domain and says so in the primary source, and we treat the facet as the weakest of the four.

Context length and active-parameter count, which would otherwise belong here, are reported per variant in Table 1 instead, because they are quantities rather than categories.

Table 4. Secondary facets. These do not describe what a model is but whether it can be adopted, and they are the facets practitioners report finding hardest to recover from model cards. Each is recorded per variant in the released inventory.

Facet	Values	Why it gates adoption
Licensing	Permissive (Apache-2.0, MIT); custom vendor licence with use restrictions; research-only; unspecified; proprietary/API-only	Determines commercial use, redistribution of fine-tunes, and whether an on-premises deployment is legal at all
Data transparency	Full (corpus released); partial (composition described, data not released); disclosed sources only; undisclosed	Determines whether contamination, licensing of training data, and domain fit can be assessed
Safety alignment	Preference-optimized with a published safety evaluation; instruction-tuned without one; base model only	A base checkpoint wired into an agent with tool access has no refusal behavior to rely on
Reasoning mode	None; always-on extended reasoning; controllable budget; separate reasoning checkpoint	Determines token cost per call and whether latency is predictable, which is decisive inside an agent loop
Deployment target	Documented inference engine(s) and hardware class the vendor ships an artifact for	An unquantized checkpoint with no GGUF, ONNX, or ExecuTorch export is not an edge model whatever its size



Figure 2. The four primary facets of the taxonomy. The facets are separately observable but not orthogonal, and assignment is multi-label: a model may carry more than one category on a facet. Representative models are italicised. Section 3 develops each facet, Table 4 gives the five secondary facets, and Table 5 gives the tabular form.

Figure 3 shows how the size distribution of new SLM releases has evolved: the sub-1B and 1–3B bands dominated early releases, but the 3–7B band has become the centre of gravity as capability-per-parameter has improved. Figure 4 shows the parameter span of the major families.

3.2 Taxonomy by Architecture

3.2.1 Dense Transformer Models

The majority of production SLMs use a dense transformer decoder: every token attends to every other token (within window) and every parameter is activated per forward pass. This category spans from sub-1B models (SmolLM2-135M [57]) to 7–8B flagships (Mistral 7B [56], Llama 3.1 8B [4]). Recent dense families are notable for pushing capability per parameter: SmolLM3-3B [73] uses grouped-query attention with NoPE and is trained on 11.2T tokens with a documented three-stage curriculum, matching or exceeding Llama-3.2-3B and Qwen2.5-3B; Qwen3's dense line [17] spans 0.6B–32B under a single architecture combining GQA, RoPE, and SwiGLU with a unified thinking / non-thinking switch. Key architectural improvements over the vanilla GPT-2 design include:

Grouped-query attention
: Introduced in Llama 2 [34] and adopted by Mistral 7B and most post-2023 SLMs, GQA reduces the KV cache memory by a factor of G (query heads divided by KV heads), enabling longer context windows without a proportional memory cost.

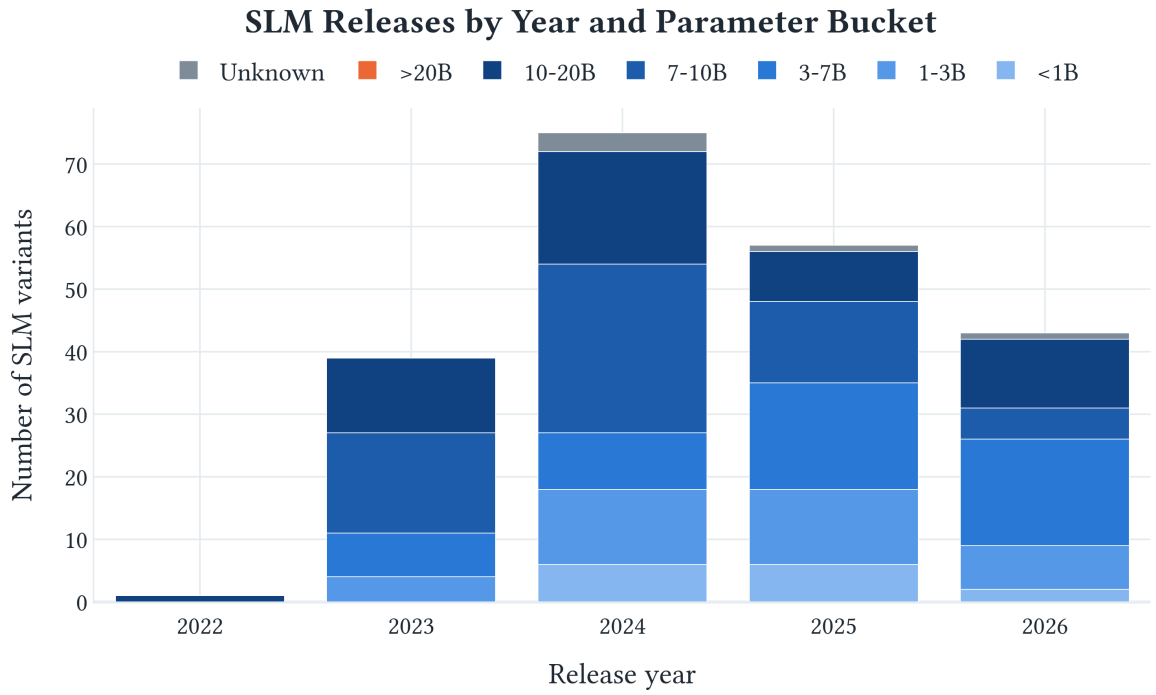


Figure 3. Documented SLM variants by release year and parameter band. The 3–7B band has grown from a minority of releases in 2023 to the largest single band by 2025.

Extended RoPE base
: Phi-3, Phi-4, Gemma 3, and Llama 3.2 all use RoPE with extended base frequencies (θ = 500,000–1,000,000) to support context windows up to 128K tokens without architectural modification.

SwiGLU feed-forward network
: Replacing the standard two-layer feed-forward network (FFN) with a Swish-gated linear unit (SwiGLU) variant, adopted across nearly all post-2022 SLMs, improves perplexity at equal parameter count and has become the default choice in every major family surveyed here.

These changes, together with RoPE base extension and positional interpolation, have driven a rapid expansion of context windows in the SLM inventory (Figure 5): median supported context grew from a few thousand tokens in 2023 to over 100K by 2025.

3.2.2 Mixture-of-Experts (MoE) SLMs

MoE architectures [74, 75] decouple *total* parameters from *active* parameters per token: only a subset of "expert" feed-forward network (FFN) blocks is activated by a learned router. This allows a model to have a large total parameter count while keeping per-token FLOPs comparable to a much smaller dense model.

Mixtral 8×7B [76] (47B total, 12.9B active per token) is the canonical example, competitive with 70B dense models while activating only about a quarter of its parameters per token. More recent MoE models in this band include DeepSeek-MoE [77] (finer-grained experts, 2B active from 16B total) and Qwen3-30B-A3B [17], which activates 3B of 30B parameters and reportedly matches Qwen2.5's dense base models at a fraction of the active parameter count. Meta's Llama 4 Scout [78] (109B total, 17B active, 16 experts) applies the same principle at a much larger total scale, with Behemoth serving as the distillation teacher.

These models sit precisely on the fault line that condition C2 of Section 1.2.1 is there to mark. Per-token latency, FLOPs, and energy track *active* parameters, so Qwen3-30B-A3B costs roughly what a 3B dense model costs to generate a token. Download size, disk footprint, and resident memory track *total* parameters, so the same model needs on the order of 17 GB of weight storage at 4 bits and cannot be resident on a phone. Both facts are true at once, and a survey that reports only the first has described a model that nobody can deploy on the hardware the first fact makes it sound suitable for. In this paper, therefore, Mixtral 8×7B, Qwen3-30B-A3B, and Llama 4 Scout are *small-active-parameter MoE* models, not SLMs: they appear in the inventory and in the architectural discussion, and they are excluded from every statement about on-device feasibility, storage, and resident memory. Models such as Granite 4.0 Tiny (~7B total, ~1B active) satisfy both conditions and are SLMs without qualification.

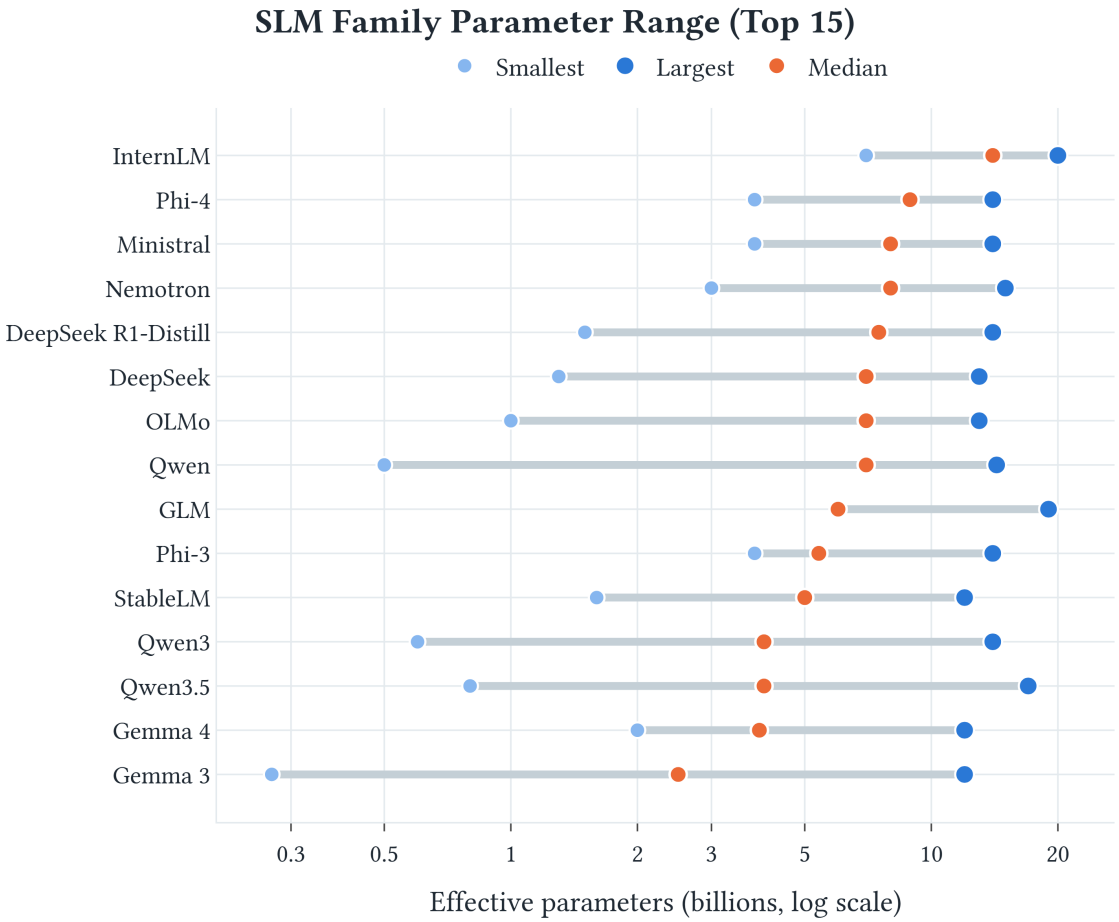


Figure 4. Parameter span (minimum, median, maximum, in billions, log scale) of the principal SLM families in the inventory.

Dense models still dominate the inventory numerically (Figure 6), but the small-active-parameter design is where the 2026 releases concentrated: 27 of the 215 inventory entries now activate at most 20B parameters against a larger total, up from a handful two years earlier. Nemotron 3.5 Lightning (30B total, 3B active), Cohere's North Mini Code (30B/3B), Poolside's Laguna XS 2.1 (33B/3B), inclusionAI's Ling 3.0 Flash (124B/5.1B), and DeepSeek V4 Flash (284B/13B) all appeared between June and August 2026. Each has the per-token economics of a 3–13B model and the storage and residency economics of something an order of magnitude larger, which is precisely the split C1 and C2 exist to keep visible.

3.2.3 Hybrid SSM-Transformer Models

A nascent but rapidly growing class of SLMs combines attention-based transformer layers with state-space models (SSMs) [79], which process sequences in linear rather than quadratic time. Mamba [79] introduced hardware-aware selective SSMs; Jamba [80] interleaves transformer and Mamba blocks in a 1:7 ratio, achieving competitive performance with 2.6× lower KV-cache memory than a pure transformer. In 2025–2026 this design moved from research to production at the small scale. Falcon-H1 [16] spans 0.5B–34B with a parallel Mamba-2/attention block. IBM's Granite 4.0 [81] interleaves self-attention and Mamba-2 layers at roughly a 1:9 ratio and pairs it with a fine-grained MoE, reporting more than 70% memory reduction for long-context and multi-session inference relative to transformer-only models, with "Tiny" (~7B total, ~1B active) and "Micro" variants aimed explicitly at edge and local use. NVIDIA's Nemotron Nano 2 [82] replaces most self-attention in a 9–12B model with Mamba-2 layers (6 attention, 28 Mamba-2, 28 FFN across 62 layers) and reports up to 6× higher throughput than Qwen3-8B when generating long reasoning traces. Alibaba's Qwen3.5 small series adopts a Gated DeltaNet hybrid at a 3:1 linear-to-full-attention ratio [83]. Linear-time inference is most valuable exactly where SLMs are deployed, in long-context, streaming, memory-bound edge settings, which is why the hybrid category is growing fastest.

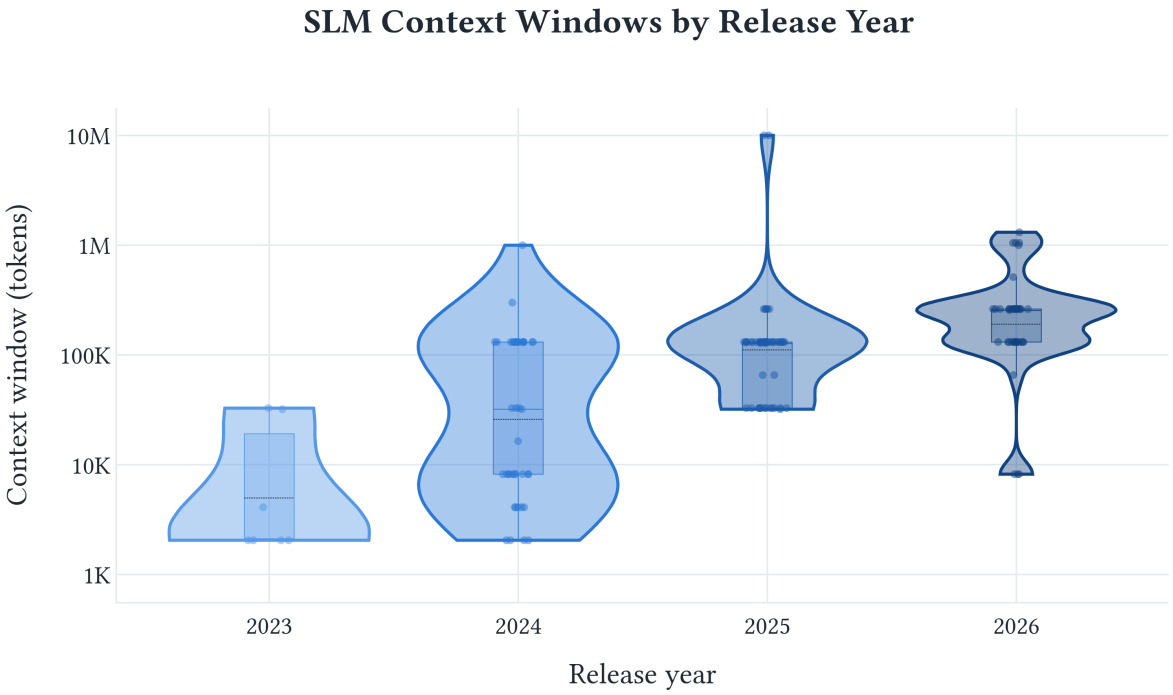


Figure 5. Supported context window of documented SLM variants by release year (log scale). Each violin is the kernel-density estimate of the distribution; the inner box marks the interquartile range, the dashed line the mean, and the dots individual models. Long context has become standard in the sub-10B regime.

3.3 Taxonomy by Training Strategy

3.3.1 Trained from Scratch

Most commercially significant SLMs are pre-trained from scratch on carefully curated datasets. The Phi family exemplifies a data-centric philosophy: rather than increasing parameters, Microsoft researchers found that training on "textbook-quality" synthetic and curated web data yields disproportionate gains [36, 84]. Phi-1 (1.3B) [36] trained on <8B tokens, two orders of magnitude less than contemporaries, yet achieved competitive Python coding benchmarks. Llama 3.2 3B [4] trained on 9T tokens from Meta's curated corpus. Gemma 3 [41] scales its token budget with model size rather than applying one figure across the family, from 2T tokens for the 1B variant to 14T for the 27B, under a curriculum that progressively increases the ratio of math, code, and multilingual data.

Key training innovations in this category include: (a) *data quality filtering* using heuristic and classifier-based pipelines; (b) *synthetic data generation* from larger teacher models [85, 86]; (c) *curriculum scheduling* where token mixture evolves over training to emphasize harder reasoning data in later stages; and (d) *long-context extension* via continued pre-training with positional interpolation or extended RoPE.

3.3.2 Distilled SLMs

Knowledge distillation [40] transfers capability from a large teacher model to a smaller student. In the LLM era, distillation takes several forms. *White-box distillation* accesses the teacher's internal representations (intermediate hidden states, attention maps), enabling feature-level alignment. *Black-box distillation* uses only the teacher's outputs (token logits or generated text) as supervision. Orca [86] demonstrated that distilling reasoning *traces* from GPT-4 rather than just answers, enables a 13B student to acquire substantially stronger chain-of-thought capabilities. Reasoning distillation became a dominant 2025 strategy: DeepSeek-R1-Distill models [87] transfer long chain-of-thought from DeepSeek-R1 into Qwen-2.5 students at 1.5B–14B, and Phi-4-mini-reasoning [88] combines distillation from a large reasoning teacher with reinforcement learning to bring competition-mathematics performance to a 3.8B model. Gemma 3 and Gemma 4 [18, 41] distill every model size, including the largest, from a bigger teacher by default.

3.3.3 Pruned and Compressed SLMs

Rather than training a small model from scratch, pruning removes parameters from a large pre-trained model. NVIDIA's Minitron pipeline [89] first prunes Llama 3.1 8B by removing attention heads, MLP neurons, and layers guided by activation statistics, then distills from the unpruned teacher, achieving a 40× reduction in training

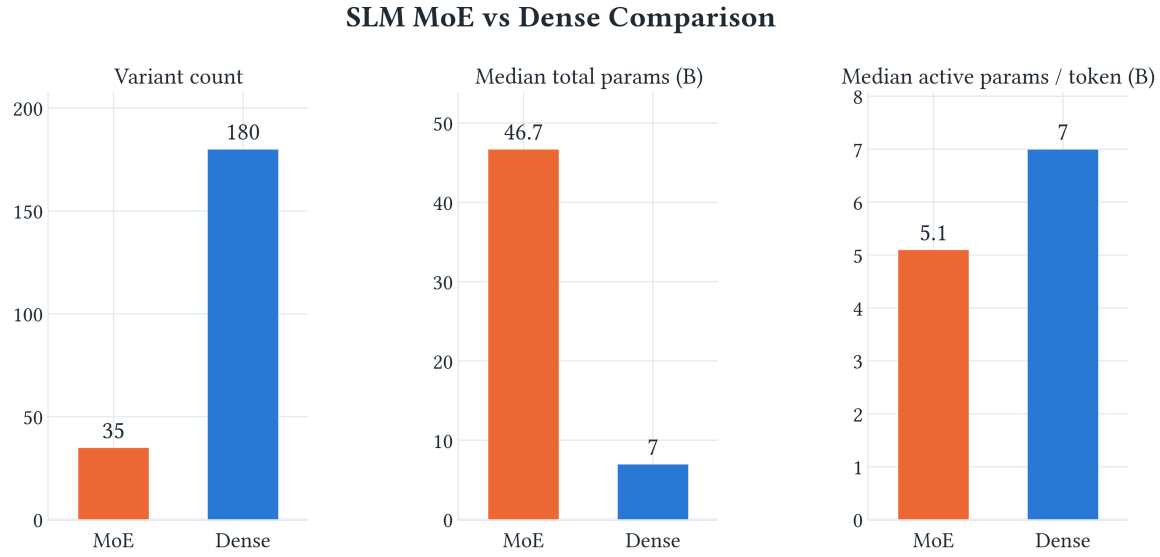


Figure 6. Mixture-of-experts versus dense SLMs in the inventory: variant count (left), median total parameters (centre), and median *active* parameters per token (right). MoE models carry a much larger total-parameter count but a per-token cost comparable to dense models.

compute relative to training from scratch while matching or exceeding the performance of same-size models trained from scratch.

3.4 Taxonomy by Modality

3.4.1 Text-Only SLMs

Text-only models still dominate the inventory (Figure 7), processing tokenized text as both input and output. This category encompasses all major SLM families: Phi, Gemma, LLaMA, Mistral, Qwen, Yi, InternLM, SmolLM, MiniCPM, OLMo, Pythia, and DeepSeek.

3.4.2 Multimodal (Vision-Language) SLMs

A growing subset accepts image inputs alongside text, enabling visual question answering, image captioning, document understanding, and visual reasoning. Notable examples include:

PaliGemma 3B [90]
: Google's 3B vision-language SLM combining a SigLIP-400M image encoder with a Gemma decoder; its release paper reports strong transfer to a wide set of downstream vision-language tasks after fine-tuning.

Phi-3-vision (4.2B)
: Microsoft's first vision-capable Phi model [12], adding a CLIP-family image encoder to Phi-3-mini's text backbone and targeting document and chart understanding.

Gemma 3 (4B/12B/27B)
: Google's Gemma 3 attaches a custom SigLIP vision encoder, operating on fixed 896×896 inputs with a Pan-and-Scan cropping scheme for other aspect ratios, to the 4B, 12B, and 27B variants [41]. The 1B variant is text-only and has a 32K rather than a 128K context window, so vision support in this family begins at 4B and is not, as is sometimes reported, present at every size.

Qwen2.5-VL 3B/7B
: Alibaba's multimodal SLMs [14], with native support for video, charts, and documents.

3.4.3 Audio and Multi-Modal SLMs

A smaller but rapidly growing category incorporates audio alongside text and vision. Gemma 3n [91] is built mobile-first: its MatFormer ("Matryoshka") architecture nests a 2B-effective sub-model inside a 4B-effective model so that one download yields a spectrum of sizes, and Per-Layer Embeddings cut the resident memory footprint of the 5B/8B raw models to roughly that of a 2B/4B model (2–3 GB). Gemma 4 [18] extends the family from 2.3B-effective to 31B with dense and MoE variants, an integrated thinking mode, and, for the 12B model, an encoder-free architecture that ingests raw audio and image patches directly. Apple's on-device Foundation Model [19] is a ~3B multimodal

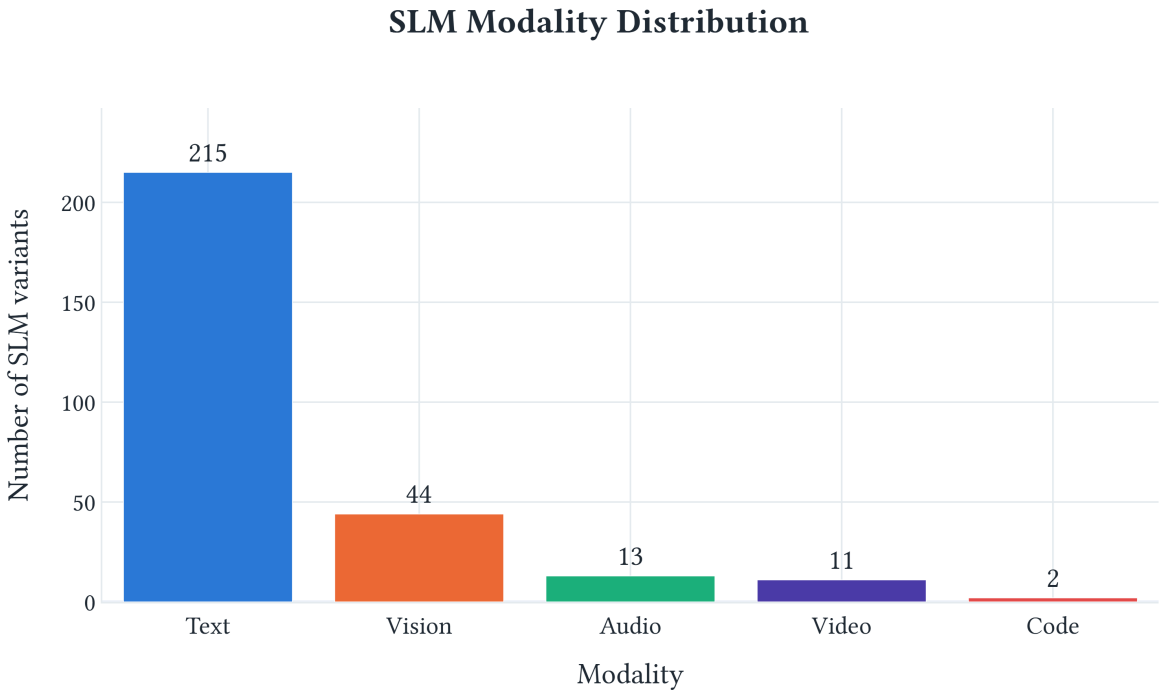


Figure 7. Input modalities supported by documented SLM variants (a variant may support several). Vision has become common; audio and video remain a small but growing minority.

model quantized with 2-bit quantization-aware training and KV-cache sharing. The full text+vision+audio frontier in a single sub-10B model is now a shipping capability rather than a research aspiration.

3.5 Taxonomy by Application Domain

3.5.1 General-Purpose SLMs

General-purpose SLMs are trained on diverse data and evaluated on multi-domain benchmarks (MMLU, HellaSwag, ARC, GSM8K). The majority of SLMs in our taxonomy, particularly the flagship families from Microsoft (Phi), Google (Gemma), Meta (LLaMA), Mistral AI, Alibaba (Qwen), and DeepSeek, fall into this category.

3.5.2 Domain-Specific SLMs

Domain-specific SLMs sacrifice breadth for depth, outperforming general-purpose models on narrow tasks:

Healthcare
: Meerkat (7B) [92], fine-tuned on synthesized clinical reasoning traces, was the first 7B model to clear the USMLE pass threshold (70.6% on MedQA) while running on a single A100 GPU; its 70B sibling edges GPT-4 on average across six medical-licensing exams. OptimCLM [93] applies combined compression (distillation + quantization + pruning) to clinical language models without sacrificing diagnostic accuracy.

Code
: Code Llama [94], StarCoder2 [95], and DeepSeek-Coder [96] represent a mature class of code-focused SLMs with strong HumanEval and MBPP performance.

Legal
: Domain-specific legal SLMs trained on corpora of statutory text, case law, and contracts target high-volume routine work such as clause classification, obligation extraction, and regulatory change triage, where on-premises deployment satisfies privilege and data-residency constraints that API-based LLMs cannot. Public evaluation in this domain remains sparse relative to the healthcare and code verticals.

Multilingual
: Cohere's Aya line illustrates a breadth-versus-depth trade-off that is specific to this size band. Aya-101 [97] is a 13B mT5-based model covering 101 languages, above the compact band; Aya 23 [98] is an 8B and 35B decoder-only pair that allocates the same capacity to 23 languages instead, and its 8B variant outperforms the 13B Aya-101 on the languages both cover. Qwen2.5 7B [14] is the most widely downloaded open-weight

SLM for multilingual deployments, with documented coverage of Chinese, Japanese, Korean, Arabic, and the major European languages.

Table 5 summarizes the four primary facets with representative models for each category. The representative models are examples, not exhaustive assignments, and several appear against more than one category because the scheme is multi-label.

Table 5. Summary of the four primary facets of the SLM taxonomy with representative models. Assignment is multi-label: a model may carry several categories on one facet, and the same model may therefore appear more than once in a block.

Facet	Category	Key Characteristics	Representative Models
Architecture	Dense Transformer	All params active per token; GQA, RoPE, SwiGLU	Phi-4-mini, Qwen3, SmolLM3, Gemma 3
	MoE	Sparse activation; total ≫ active params	Granite 4.0 Tiny (SLM); Mixtral 8×7B, Qwen3-30B-A3B (small-active MoE)
	Hybrid SSM-Transformer	Linear-time; interleaved SSM/attention blocks	Falcon-H1, Granite 4.0, Nemotron Nano 2
Training Strategy	From scratch	Curated/synthetic data; full pre-training	Phi, Qwen2.5/Qwen3, SmolLM3
	Distillation	Knowledge transfer from larger teacher	Gemma 3/4, DeepSeek-R1-Distill, Phi-4-mini-reasoning
	Pruning-based	Parameter removal from large model + distillation	Minitron, Sheared-LLaMA
Modality	Text-only	Tokenized text I/O; majority of production SLMs	Mistral 7B, SmolLM3, Qwen3
	Vision-Language	Image + text; CLIP/SigLIP encoder added	Phi-3-vision, Gemma 3 (4B and above), PaliGemma
	Audio/Multimodal	Text + vision + audio in one sub-10B model	Gemma 3n, Gemma 4, Apple AFM
Domain	General-purpose	Multi-domain training; evaluated on broad benchmarks	Phi, Gemma, Llama, Qwen, Mistral
	Healthcare	Clinical NLP; on-premises for data residency	Meerkat 7B, OptimCLM
	Code	Code corpora; FIM training; HumanEval/MBPP focus	Code Llama, StarCoder2, DeepSeek-Coder
	Legal/Multilingual	Domain corpora; regulatory and cross-lingual tasks	Aya 23 8B, Qwen2.5 multilingual

4 Model Compression Techniques for SLMs

Model compression is central to the SLM ecosystem: it is the primary pathway for reducing large pre-trained models to sizes deployable on edge hardware, and it provides a complementary route to training-from-scratch. The principal techniques are knowledge distillation, quantization, and pruning; in practice, compound pipelines combining all three have become the state of the art.

4.1 How Compression Results Are Reported Here

Comparisons of compression techniques are routinely presented as a single "compression ratio" set against a single "accuracy retention" percentage. An earlier version of this survey did the same, and the presentation does not survive scrutiny, so we set out here why and what replaces it.

There is no single compression ratio. Quantization reduces bits per weight, leaving the parameter count untouched; pruning reduces the parameter count, and reduces memory only where the sparsity pattern is one the runtime can actually exploit; distillation reduces the parameter count of the student but is better understood as reducing the training compute needed to reach a given quality, since a distilled 4B model and a from-scratch 4B model have the same deployment cost. Collapsing these into one number equates quantities that trade off against different budgets. Distillation in particular has no well-defined universal ratio at all: the student's size is chosen by the practitioner, so "10–100×" describes a range of design choices in the literature, not a property of the method.

There is no baseline-free accuracy retention. A statement that 4-bit quantization retains "about 96%" of accuracy is uninterpretable without the baseline precision, the model, the model's size, the task, the metric, and the rule by which per-task retentions were aggregated. The dependence on each of these is large: retention under post-training quantization (PTQ) falls with model size [99], differs by task family, with mathematical and multi-step reasoning degrading faster than knowledge recall, and differs by method at matched bit-width [46]. A single band averages across all of that variation and then presents the average with two significant figures.

We therefore replace the single table with two. Table 6 separates the reductions each technique delivers along five distinct axes, which are not interchangeable and are not summed. Table 7 reports what the reviewed literature establishes about quality cost, stated as a direction and a scope condition with the source attached, rather than as a number without a baseline. Where the literature supports a numeric claim under stated conditions, the number appears in the text with its conditions; where it does not, no number is given, and Section 10.7 specifies the experiment that would supply one.

4.2 Knowledge Distillation

knowledge distillation (KD) [40] transfers the "dark knowledge" embedded in a teacher model's soft output distribution to a smaller student model. The student is trained to minimize a combination of the standard cross-entropy loss on labeled data and a Kullback–Leibler (KL) divergence loss with respect to the teacher's output logits

Table 6. What each compression technique reduces, by axis. Entries are what the technique does by construction, not measurements. "Parameters" counts stored weights; "memory" is realized weight footprint on a runtime that can exploit the format; "training compute" is the cost of producing the compressed artifact relative to training an equivalent model from scratch; "latency" is decode-time speedup on memory-bandwidth-bound hardware. A dash means the technique does not act on that axis.

Technique	Parameter reduction	Weight-memory reduction	Training-compute reduction	Decode latency effect	Retraining required
PTQ, 8-bit	—	2× vs. FP16	—	Proportional to bandwidth saved	No
PTQ, 4-bit	—	3.2–3.4× realized (Table 11)	—	Proportional to bandwidth saved, batch 1	No
PTQ, 2-bit	—	~6× realized	—	As above	No; quantization-aware training (QAT) in practice
Unstructured pruning, 50%	2×	Only on sparse-capable runtimes	—	None on dense kernels	Calibration only
Structured pruning	Set by target width/depth	Proportional to parameters removed	—	Proportional to parameters removed	Yes, to recover quality
Knowledge distillation	Set by student size	Proportional to student size	Large; the student is trained rather than the teacher	Proportional to student size	Yes, student training
Prune + distill + quantize	Set by target	Product of the pruning and quantization factors	40× reported for Minitron [89]	Product of the above	Yes

at a temperature $T > 1$. This approach was pioneered for BERT compression (DistilBERT [10], TinyBERT [11]) and has since been substantially extended for generative SLMs.

4.2.1 White-Box vs. Black-Box KD

The survey by Yang, Lu, Zhu, et al. [101] classifies LLM distillation into two paradigms:

White-box KD has access to the teacher's intermediate representations. TinyBERT [11] pioneered layer-by-layer feature alignment, matching student hidden states, attention matrices, and embedding layers to teacher counterparts via MSE loss. In the decoder-only LLM setting, this requires pairing student and teacher layers, which is non-trivial when sizes differ substantially, and is computationally expensive but yields superior retention of teacher capabilities.

Black-box KD uses only the teacher's generated outputs, either token probabilities or free-form text, as the supervision signal. Orca [86] exemplifies text-output black-box KD: GPT-4 generates step-by-step reasoning traces, which a LLaMA-based student learns to replicate. DeepSeek-R1 [87] extends this to long chain-of-thought reasoning distillation, producing 7B and 14B students that dramatically outperform identically-sized from-scratch models on mathematical reasoning.

4.2.2 Chain-of-Thought Distillation

A particularly productive form of black-box KD is chain of thought (CoT) distillation [102]: the teacher model generates explicit reasoning steps, and the student learns to produce or internalize these steps. Feng, Li, Chenglin, Chen, Yu, and Zhang [103] demonstrate counterfactual distillation, which augments CoT traces with contrastive incorrect reasoning, as a way to sharpen the student's reasoning boundaries. Liao, Li, Liu, and Zhang [104] propose Neural-Symbolic Collaborative Distillation, using a neuro-symbolic teacher to provide structured reasoning supervision that is more transferable to small students than purely neural CoT.

4.2.3 Data Augmentation in KD

The survey by Xu, Li, Tao, et al. [105] highlights data augmentation (DA) as a central paradigm within KD frameworks. When the student lacks sufficient coverage of the teacher's knowledge space, the teacher can generate synthetic training instances targeting specific skill gaps, the mechanism underlying the Phi family's success [36]. The efficient KD approach of Ballout, Tresp, Divo, and Kersting [106] demonstrates that targeted data augmentation can reduce the student training compute by 60% while preserving 95% of the teacher's task-specific accuracy.

4.2.4 Self-Distillation

An emerging approach eliminates the external teacher entirely. Fu, Shi, and Yin [107] propose Dynamic Self-Distillation, in which the model uses its own predictions from previous mini-batches as soft targets during

Table 7. What the reviewed literature establishes about the quality cost of each technique in the sub-10B regime, with the scope condition under which it was established. Claims are graded by evidence class (Section 5.1.5): B independent evaluation, C vendor- or author-reported, D synthesis across sources by the present authors.

Technique	What the literature supports	Scope of the evidence	Grade
PTQ, 8-bit	Effectively lossless on knowledge and mathematical benchmarks; no consistent degradation reported	Widely reproduced across model families	B
PTQ, 4-bit	A small but real accuracy cost that grows as model size falls, and is larger on multi-step reasoning than on knowledge recall	Reported for the sub-4B band [99]	C
PTQ, 4-bit, method comparison	AWQ retains more accuracy than GPTQ at matched bit-width by protecting activation-salient channels	Author-reported, with independent reuse [46]	C
PTQ, 2-bit	PTQ degrades unacceptably; QAT recovers a meaningful part of the loss, and the margin over PTQ widens as bit-width falls	Vendor pipelines with full training-stack control [19]	C
Unstructured pruning, 50%	Perplexity increase below 1% at 175B scale in one pass [49]; the tolerable sparsity falls sharply as the base model shrinks	Established at large scale; extrapolation to sub-4B is not supported	C
Quantization vs. pruning	Quantization preserves fidelity, multilingual perplexity, and reasoning accuracy better than pruning at matched compression; AWQ 4-bit beats 50% Wanda on 6 of 7 tasks	Controlled study: 6 models, 0.5–3.8B, 7 languages, 7 tasks [28]	B
Prune + distill + quantize	A 4B Minitron matches from-scratch 4B models on commonsense, code, and math at 40× less training compute	Single pipeline, single teacher [89]	C
Domain compound pipeline	A clinical model under 1B retains reported accuracy within 5% of its 7B counterpart on ICD coding and discharge summarization	Single study, single domain [93]	C
Fairness under compression	Degradation is uneven across languages and dialects, and pruning is worse than quantization on multilingual tasks	Inferred across two sources [28, 100]	D

fine-tuning, providing a form of implicit regularization that reduces overfitting on small fine-tuning datasets, a critical challenge for SLMs.

4.3 Quantization

Quantization reduces the numerical precision of model weights and/or activations from 32-bit floats (FP32) or 16-bit (FP16/BF16) to integers or low-bit floats (INT8, INT4, FP4, NF4, INT2). This directly reduces memory footprint and, on hardware with native integer arithmetic (NPUs, modern CPUs), reduces inference latency.

4.3.1 Post-Training Quantization (PTQ)

PTQ is performed after training is complete, requiring only a small calibration dataset (typically 128–512 samples) and no gradient computation. It is the dominant quantization paradigm because it is cheap and applies to any model without access to training infrastructure.

GPTQ [45] applies second-order weight compression: it uses an approximation of the inverse Hessian of the loss with respect to each layer's weights to determine optimal rounding decisions, minimizing the layer-wise squared error introduced by quantization. For SLMs, GPTQ 4-bit quantization of a 7B model reduces video memory (VRAM) from ~14 GB to ~4 GB.

AWQ [46] observes that not all weights are equally important for activation output: 1% of weights (those corresponding to large-magnitude activation channels) account for disproportionate reconstruction error. AWQ identifies these "salient" channels using a small calibration set and applies per-channel scaling to protect them before quantization, consistently outperforming GPTQ on complex reasoning tasks and winning the MLSys 2024 Best Paper award.

GGUF is a quantization file format popularized by `llama.cpp` [47] that packages model weights, vocabulary, and architecture metadata in a single binary file. It supports a range of quantization levels (Q2_K through Q8_0), letting users trade off quality against memory footprint along a near-continuous spectrum, and it is the format in which most sub-4B models are actually distributed for CPU-only laptop and single-board deployment.

SqueezeLLM [48] introduced sensitivity-based non-uniform quantization with dense-and-sparse decomposition, showing that sensitivity varies dramatically across weight dimensions and enabling higher accuracy at ultra-low bit-widths.

4.3.2 Quantization-Aware Training (QAT)

QAT integrates simulated quantization into the training loop, allowing the model to learn weight values that are robust to quantization error. This is particularly valuable for aggressive quantization (2-bit, 3-bit) where PTQ

accuracy drops unacceptably. QAT recovers a meaningful part of the accuracy that PTQ loses at the same bit-width, and the margin widens as bit-width falls, but it requires access to training data and significant compute, which is why it appears mainly in vendor pipelines with control of the full training stack, such as Apple's 2-bit on-device quantization [19].

4.3.3 Quantization versus Pruning in the Sub-10B Regime

Zhou, Kurz, and Zhao [28] report a systematic comparison of PTQ quantization (GPTQ, AWQ) against post-training pruning (SparseGPT, Wanda) specifically in the SLM regime (0.5B–3.8B parameters, six models, seven languages, seven downstream tasks). Their primary conclusion: *quantization consistently outperforms pruning in preserving model fidelity, multilingual perplexity, and reasoning accuracy.* AWQ at 4-bit outperforms 50% Wanda pruning on 6 of 7 tasks. Quantization's advantage is most pronounced on language modeling and multilingual tasks; the gap narrows on complex knowledge tasks, where both methods show similar degradation under high compression.

4.4 Pruning

Pruning removes parameters from a pre-trained model, either individually (unstructured) or in structured groups (entire heads, layers, or channels). Structured pruning yields hardware-friendly sparse models compatible with standard matrix multiplication libraries; unstructured pruning achieves higher compression ratios but requires special sparse hardware or software to realize latency benefits.

4.4.1 Unstructured Pruning

SparseGPT [49] extends the GPTQ framework to weight removal: it prunes weights layer-by-layer using inverse-Hessian information to minimize the reconstruction error introduced by zeroing each weight. It can prune the 175B GPT-3 to 50% sparsity in a single forward pass without weight updates, with a perplexity increase below 1%.

Pruning by weights and activations (Wanda) [108] proposes a simpler criterion: remove the weights with the smallest product of weight magnitude and corresponding input-activation norm. This needs only a forward pass over the calibration data (no second-order information) and runs about 10× faster than SparseGPT while matching its accuracy at 50% sparsity.

4.4.2 Structured Pruning

Structured pruning removes entire attention heads, MLP channels, or transformer layers, yielding immediately deployable models without special sparsity libraries. Layer pruning, which removes the least important layers as measured by cosine similarity between adjacent layer outputs, can remove a modest fraction of a model's depth with limited benchmark degradation, though the tolerable fraction falls sharply as the base model gets smaller. The Minitron pipeline [89] applies depth and width pruning simultaneously, guided by importance scores derived from activation magnitudes over a calibration set, and is the best-documented instance of the approach at SLM scale.

4.5 Combined Compression Pipelines

The state of the art in SLM compression combines pruning, distillation, and quantization in a multi-stage pipeline. NVIDIA Minitron [89] achieves a 40× reduction in training compute relative to training Llama 3.1 8B from scratch by: (1) structured pruning to target size; (2) knowledge distillation from the original 8B teacher to the pruned student; (3) post-training quantization for deployment. The resulting 4B Minitron model matches the performance of 4B models trained from scratch on commonsense, code, and math benchmarks.

OptimCLM [93] applies a similar pipeline to clinical language models, combining all three compression stages to produce models under 1B that achieve within 5% of clinical NLP accuracy of their 7B counterparts. These results suggest that the combination of pruning (for architectural reduction) + distillation (for capability recovery) + quantization (for memory reduction) represents the practical frontier of SLM compression.

5 The Public Evaluation Record for Small Language Models

This section examines what the public record can support about SLM capability and deployment behavior. In contrast to an original benchmarking study, the performance values analyzed here are *reported* figures, synthesized from vendor technical reports, official model cards, community leaderboards, and third-party evaluations. We first document the protocol used to collect and reconcile these figures (Section 5.1), then present the consolidated synthesis (Sections 5.2–5.5). Appendix A records the provenance of these figures at the level of model and source class, together with the reference conditions under which they were reported; where sources disagree, the reconciliation rule of Section 5.1.4 applies.

5.1 Data-Collection Protocol

5.1.1 Model Selection

Model inclusion required: (1) public availability of weights or a publicly documented evaluation card; (2) parameter count below 10 billion, with Phi-4 at 14B retained as an upper-bound reference; (3) at least one reported score on an established benchmark used here; and (4) representation of diverse training methodologies (synthetic-data-centric, web-scale pretraining, distillation). Table 8 lists the resulting 18-model cohort with architectural specifications as reported by each model's release materials. The cohort spans from 0.5B to 14B parameters and eight developer organizations, enabling analysis of how training methodology, architecture, and scale interact to determine reported capability.

Table 8. Architectural specifications of reviewed models (as reported by each model's official release materials).

Model	Params	Layers	Hidden	Attention	Context	Vocab	Train Tokens
TinyLlama-1.1B	1.1B	22	2,048	MHA-32h	2K	32K	3.0T
SmolLM2-1.7B	1.7B	24	2,048	MHA-32h	2K	49K	11.0T
StableLM 2-1.6B	1.6B	24	2,048	MHA-32h	4K	100K	2.0T
Qwen2.5-0.5B	0.5B	24	896	GQA-14/2	32K	152K	18.0T
Qwen2.5-1.5B	1.5B	28	1,536	GQA-12/2	32K	152K	18.0T
Qwen2.5-3B	3.0B	36	2,048	GQA-16/2	32K	152K	18.0T
Qwen2.5-7B	7.6B	28	3,584	GQA-28/4	128K	152K	18.0T
Llama 3.2-1B	1.2B	16	2,048	GQA-32/8	128K	128K	9.0T
Llama 3.2-3B	3.2B	28	3,072	GQA-24/8	128K	128K	9.0T
Gemma 2-2B	2.6B	26	2,304	GQA-8/4	8K	256K	2.0T
Gemma 2-9B	9.2B	42	3,584	GQA-16/8	8K	256K	8.0T
Gemma 3-1B	1.0B	26	1,536	GQA-LG	32K	262K	2.0T
Gemma 3-4B	4.3B	34	2,560	GQA-LG	128K	262K	4.0T
Mistral 7B	7.3B	32	4,096	GQA-32/8	8K	32K	n/a
Phi-3-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.3T
Phi-3.5-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.4T
Phi-4-mini	3.8B	32	3,072	GQA-24/8	128K	200K	5.0T
Phi-4	14.0B	40	5,120	GQA	16K	100K	∼10T

Note: MHA = Multi-Head Attention; GQA = Grouped-Query Attention (Q/KV heads); GQA-LG = GQA with local–global interleaving. Every model in this cohort is dense, so total and active parameters coincide and a single parameter column suffices; no small-active-parameter MoE model (Section 1.2.2) is in the cohort, because none has public reporting complete enough to include.

On training-token counts. These are the least reliable column in the table, and their status differs by row. Marked *per-variant* where the source states a figure for that specific size: Gemma 2-2B (2.0T), Gemma 2-9B (8.0T), Gemma 3-1B (2.0T), Gemma 3-4B (4.0T), Phi-3-mini (3.3T), Phi-3.5-mini (3.4T), Phi-4-mini (5.0T), TinyLlama (3.0T), SmolLM2-1.7B (11.0T), StableLM 2 (2.0T). Marked *series-level* where the source reports one figure for the whole family and the individual sizes are not separately stated: Qwen2.5 (18.0T across 0.5B–7B) and Llama 3.2 (9.0T across 1B and 3B). Phi-4's ∼10T is an approximation given as such by its source. Mistral 7B's training-token count is not disclosed. We report series-level figures in the size rows they cover rather than omitting them, but they should not be read as independent per-model measurements, and no claim in this paper relies on comparing a series-level figure against a per-variant one.

On context length. The value is the maximum the primary source documents, which for several models is reached through position interpolation at inference rather than trained natively: Qwen2.5-7B is natively 32K and extended to 128K, and Phi-3-mini's 128K is a separate long-context checkpoint from the 4K base.

5.1.2 Benchmark Tasks and Metrics

The benchmarks that recur across the reviewed release documents span seven capability areas: MMLU [109] for broad knowledge; ARC-Easy and ARC-Challenge [110] and HellaSwag [111] for commonsense and language reasoning; GSM8K [112] and MATH [113] for mathematical reasoning; HumanEval [114] and MBPP [115] for code generation; TriviaQA [116] and SQuAD 2.0 [117] for question answering; MNLI [118] for natural language inference; and SST-2 [119] for sentiment classification. Each benchmark carries a standard metric: accuracy for multiple choice, exact match or F1 for question answering, pass@1 for code, exact match for GSM8K, and symbolic equivalence for MATH. Instruction-following (MT-Bench-style [120]) and summarization are reported too sparsely and too heterogeneously to compare at all, and the conventions behind them are described in Appendix A.5. Section 5.3 takes up separately the question of whether any of these benchmarks bears on agentic suitability.

Four conditions determine what a benchmark number means, and a source that omits any of them has not reported a comparable result. *Benchmark version:* MMLU exists in the original and in MMLU-Pro, ARC in Easy and Challenge splits, GSM8K in original and GSM8K-Platinum revisions, and HumanEval in the original and HumanEval+ forms, and these are not interchangeable. *Prompt template:* whether the model's own chat template is applied, whether options are lettered or restated, and whether a system prompt is present. *Answer extraction:* whether a multiple-choice answer is scored by comparing per-option log-likelihoods or by parsing a generated letter, which can differ by several points on the same model and benchmark, and for chain-of-thought tasks, which span of the generation is taken as the answer. *Decoding settings:* greedy versus sampled, temperature, and for pass@k the number of samples. *Harness:* the framework and its version, most often `lm-evaluation-harness` [121] or HELM [122], each with its own defaults for all of the above.

The coverage of these five conditions across the reviewed sources is what Section 5.2 reports on, and it is poor. Which benchmarks a vendor reports, in which version, under how many shots, and with which extraction rule varies enough to prevent a common table.

5.1.3 Hardware and Quantization Scope

The review covers reported behavior across six configurations spanning the cloud-to-edge spectrum (Table 9) and three precisions: FP16 (reference), INT8, and INT4, with GGUF Q4_K_M, GPTQ group-128, and AWQ group-128 treated as distinct INT4 subclasses when the source distinguishes them. Inference-framework attribution (vLLM [123], llama.cpp [47], ONNX Runtime [65], ExecuTorch [63], MLC-LLM [64]) follows the stack used by the source of each figure; cross-framework comparisons are made only when compared figures share a framework or are explicitly flagged.

Table 9. Hardware configurations covered by the review. For each reviewed model, reported performance on a configuration was included where publicly available.

Configuration	Hardware	RAM/VRAM	TDP	Use Case
Cloud GPU	NVIDIA A100 80GB	80 GB HBM2e	300W	Training + serving
High-end Consumer	NVIDIA RTX 4090	24 GB GDDR6X	450W	Local inference
CPU/iGPU Laptop	AMD Ryzen 7 7840HS	32 GB DDR5	35–54W	Laptop deployment
Edge GPU Module	NVIDIA Jetson Orin Nano	8 GB LPDDR5	25W	Edge AI / robotics
Edge SBC	Raspberry Pi 5	8 GB LPDDR4X	12W	Embedded IoT
Mobile SoC	Snapdragon 8 Gen 3	12 GB LPDDR5X	~8W sustained	Smartphone

5.1.4 Source Selection, Extraction, and Reconciliation

Reported values were collected in decreasing order of priority from: (1) official vendor technical reports and release papers; (2) official model cards on Hugging Face and vendor repositories, including linked evaluation appendices; (3) community-operated standardized evaluations (the Open LLM Leaderboard, HELM [122], and lm-evaluation-harness [121]-based community reports); (4) third-party independent evaluations (Artificial Analysis, LMSYS Chatbot Arena, peer-reviewed follow-up studies); and (5) vendor developer blogs, used only to fill gaps. All figures were extracted within a single collection window to preserve time-consistency across models.

When qualifying sources disagreed on the same model–task–condition tuple, the figure from the source higher in the priority list was retained and the alternative recorded in Appendix A; where the discrepancy exceeded 2.0 percentage points (or the metric equivalent), both figures were retained as a reported range or the tuple was excluded. A tuple was excluded from cross-model comparison when: (a) only a single self-reported value existed and could not be cross-checked; (b) the harness, prompt format, or shot count materially differed from the setting used by at least half the compared models and could not be normalized; or (c) the source did not specify base versus instruction-tuned status.

5.1.5 Evidence Grading

Because this survey mixes controlled studies, vendor self-reports, independent reproductions, and our own synthesis, and because those carry very different weight, every substantive quantitative claim in the paper is graded. The scheme has four levels, and the grade denotes the strongest class of source standing behind the claim, not the claim's importance:

A **Controlled**
: Measured under a single named harness, with the compared conditions differing only in the variable of interest, and a matched baseline reported. This is the standard a cross-model comparison would have to meet. *No claim in this paper carries grade* A, which is the principal finding of Section 5.2.

Table 10. Evidence grade for the principal claims of this survey. Grades follow Section 5.1.5: A controlled, B independent, C reported by the interested party, D our interpretation.

Claim	Grade	Where
A cross-model accuracy table cannot be assembled from current public reporting without misrepresenting its sources	B	§5.2
The strongest 3–4B models of 2026 report scores that were state of the art for 13B-class models in 2023	C	§5.2
Within families that publish ablations, corpus composition moves results more than architecture at fixed scale	D	§9.1
INT8 post-training quantization is effectively lossless on knowledge and mathematical benchmarks	B	§5.4
INT4 accuracy loss grows as model size falls and is larger for multi-step reasoning	C	§5.4
Quantization preserves fidelity better than pruning below 4B parameters at matched compression	B	§4.3
Weight footprint at 4 bits is 3.2–3.4× below FP16	Derived	§5.4
Sub-10B function-calling models are competitive with far larger models on function-calling benchmarks	B	§7.3
Serving a 7B model is 10–30× cheaper than serving a 70–175B model	C	§7.2.2
End-to-end agent success rates for sub-10B models on realistic multi-step benchmarks are low in absolute terms	B	§7.3
Hypothesis H: a specialized SLM is the appropriate default for bounded, recurrent, short-context, verifiable agent calls	D	§1.4

B **Independent**
: Reported by a party with no stake in the result: a peer-reviewed third-party study, a standardized community evaluation, or a reproduction by an unaffiliated group.

C **Reported**
: Stated by the developer of the artifact or the author of the technique, in a technical report, model card, or release paper, without independent reproduction.

D **Interpretation**
: Our synthesis across sources, or an inference from evidence that was not gathered to answer the question being asked. This grade is applied wherever we combine figures across sources, and it is the correct grade for most directional statements in this survey.

Grades appear in the tables that carry quantitative claims (Tables 7 and 14) and against the main claims of the paper in Table 10. Two consequences are worth stating plainly. First, a grade C claim is not thereby false; it is unverified, and vendor self-reports in this field are systematically more favorable than third-party reproductions, so the direction of the likely error is known. Second, the absence of any grade A claim is not a defect peculiar to this survey but a property of the field's current evidence base, and it is the reason Section 10.7 is written as a specification for experiments rather than a list of topics.

5.1.6 Limitations of the Evidence Base

Because the figures are compiled from heterogeneous public sources rather than a single controlled pipeline, four limitations bound the strength of every comparison below. **Prompt-format and harness variation:** different sources use different templates and answer-extraction heuristics, which can shift reported accuracy by several points under the same nominal benchmark. **Self-reporting bias:** vendor-reported scores are systematically more favorable than third-party reproductions; where third-party figures exist, comparisons preferentially use them and flag the divergence. **Condition-coverage sparsity:** not every model has reported scores for every task, tier, or precision. Missing values are never imputed, interpolated, or carried across from a sibling model size or from a base checkpoint to its instruction-tuned variant; a tuple with no qualifying source is recorded as absent, and absence is itself reported, since the pattern of what vendors decline to publish is part of the finding of Section 5.2. No statistic in this paper is computed over a set from which absent tuples were silently dropped. **Snapshot staleness:** the ecosystem evolves monthly, and figures reflect the state of public reporting at the collection window. These limitations motivate the empirical-verification agenda of Section 10.7.

5.2 Why a Common Benchmark Table Cannot Be Built from Public Reporting

Our original intent for this section was a single consolidated accuracy table placing all eighteen models side by side on a shared benchmark set. Assembling it from the public record proved impossible without misrepresenting

the sources, and the reasons are systematic rather than incidental. Because that finding bears directly on how practitioners should read vendor claims, we report it in place of the table.

Vendors do not report a common benchmark set. Microsoft reports MMLU, GSM8K, ARC-Challenge, HellaSwag and MATH for Phi-4-mini [39]. Alibaba's instruction-tuned Qwen2.5 evaluations report MMLU-Pro, GSM8K, MATH and HumanEval, but not ARC-Challenge or HellaSwag [14]. Google reports ARC-c, HellaSwag, MMLU, GSM8K, MATH, HumanEval and MBPP for Gemma, but its headline tables cover the pre-trained checkpoints [38, 41]. The intersection across any six vendors is close to empty, so a full matrix can only be completed by mixing sources of different provenance into the same row.

Shot counts differ per benchmark and per vendor. Within a single Phi-4-mini model card, MMLU is 5-shot, GSM8K is 8-shot with chain-of-thought, ARC-Challenge is 10-shot and HellaSwag is 5-shot. Gemma reports ARC-c at 25-shot, HellaSwag at 10-shot, MATH at 4-shot and MBPP at 3-shot. Llama 3.2 reports ARC-Challenge and HellaSwag zero-shot but MMLU at 5-shot. A column headed with a single evaluation protocol is therefore false for most of its cells, and the differences are large: shot count alone moves reported accuracy by several points on multiple-choice tasks.

Base and instruction-tuned figures are routinely conflated. Several families publish their most complete benchmark tables for the pre-trained checkpoint while shipping the instruction-tuned variant as the artifact practitioners actually deploy. Gemma 3-4B is reported at 59.6 MMLU and 38.4 GSM8K after pre-training [41]; the instruction-tuned model is described qualitatively as far stronger, without an equivalently detailed table. Any row that silently mixes the two overstates the weaker model and understates the stronger one.

Matched baselines are rarely published. Quantized, fine-tuned and long-context variants are seldom reported alongside an FP16 baseline evaluated under the same harness, so the deltas that matter most for deployment decisions cannot be recovered from the sources at all.

The consequence is that the widely circulated cross-model SLM comparison tables, including those in secondary sources and community leaderboards, should be treated as approximate at best. Where a genuine comparison is needed, the only sound basis is a controlled re-run under one harness, which is why Section 10.7 sets that out as the first item of the verification agenda rather than as an optional extension. What can be stated from the public record without distortion is qualitative and still substantial: the strongest 3–4B models of 2026 report scores on mathematics, code and knowledge benchmarks that were state of the art for 13B-class systems in 2023; no single SLM leads across all task families; and the largest single source of variation among models of near-identical size is the training corpus rather than the architecture, a pattern visible in every family's own ablations even though it cannot be quantified across families.

5.3 Why These Benchmarks Do Not Answer the Agentic Question

There is a second problem with the record, independent of its internal inconsistency, and it bears directly on the thesis of this paper. Every benchmark in Section 5.1.2 measures a model answering a question in isolation. None measures what an agent actually asks a model to do.

MMLU measures recall of facts under a multiple-choice format. GSM8K measures arithmetic word problems solved in one turn. HumanEval measures whether a standalone function passes unit tests. An agent, by contrast, asks the model to select one of several available tools, to emit arguments that satisfy a schema, to remain well-formed across a long tail of inputs, to recover when a tool returns an error, and to do so repeatedly without a single failure derailing the trajectory. A model can be strong on all three of the benchmarks above and still emit malformed JSON often enough to be unusable in a loop, and the reverse holds too: a small model tuned for function calling can be weak on MMLU and entirely adequate as an agent's executor.

The consequence is that the entire evaluation record examined in this section, even if it were internally consistent, would be the wrong evidence for hypothesis H. Section 7.3 therefore assembles a separate body of agent-specific evidence, drawn from function-calling, tool-use, structured-output, and end-to-end agent benchmarks, and grades it under the same scheme. We flag here that this second body of evidence has its own gaps, which are different from and in some respects worse than the ones documented above: the agent benchmarks that exist report frontier models far more often than sub-10B ones, and almost none reports cost per completed task, which is the quantity the economic half of the thesis turns on.

5.4 Quantization Impact

Across the compression literature reviewed in Section 4, the reported picture for the sub-10B regime is consistent on four points. INT8 post-training quantization is effectively lossless on knowledge and mathematical benchmarks. INT4 costs a small but real amount of accuracy, and the loss is *not* uniform across model sizes: smaller models degrade more, consistent with the argument that each weight in a small model carries proportionally more information [99]. Mathematical and multi-step reasoning degrade faster than knowledge recall under INT4, which

matters directly for agentic use, since arithmetic and tool-argument construction are exactly the operations agents delegate. Among INT4 methods, AWQ retains more accuracy than GPTQ by protecting the small fraction of activation-salient channels [46], while GGUF's Q4_K_M mixed-precision scheme gives the best size-quality balance on CPU and edge backends [47]. Across the sub-4B range, quantization preserves fidelity better than pruning at matched compression [28].

Per-model, per-benchmark degradation figures are not reported consistently enough across this cohort to tabulate: sources differ in harness, calibration set, and group size, and few publish a matched FP16 baseline alongside the quantized result. Measuring them under a single toolchain is one of the verification studies proposed in Section 10.7.

The memory consequence, by contrast, is firmly established, because it follows from the storage format rather than from measurement. Table 11 gives weight footprints computed from parameter count and bit-width. The realized reduction falls short of the nominal 4×, at roughly 3.2 to 3.4×, because embedding, normalization, and in mixed schemes a subset of attention tensors are retained at higher precision. A 7B model drops from about 15 GB to about 4.5 GB, and a 3.8B model from 7.6 GB to about 2.4 GB, which is what moves each of them across a hardware tier boundary.

Table 11. Model-weight memory footprint by precision (GB), computed from parameter count and storage bit-width. Four-bit columns include the higher-precision tensors each scheme retains, which is why the reduction is below the nominal 4×. Activation and KV-cache memory are additional and depend on context length and batch size.

Model	FP16	GPTQ-4bit	AWQ-4bit	Q4_K_M
Qwen2.5-7B	15.2	4.2	4.4	4.5
Phi-4-mini	7.6	2.2	2.3	2.4
Llama 3.2-3B	6.4	1.9	2.0	2.1
SmolLM2-1.7B	3.4	1.1	1.1	1.2
TinyLlama-1.1B	2.2	0.7	0.8	0.8

5.5 Multilingual and Safety Reporting

Multilingual and safety reporting is markedly sparser than benchmark reporting across the cohort, and the coverage that exists is not comparable across models; we therefore report the direction of the evidence rather than tabulating it. On multilingual capability, the models whose technical reports document large non-English pretraining mixtures, namely Qwen2.5 [14], Gemma 3 [41], and Aya [97], report substantially stronger non-English performance than the English-centric families, and the gap is widest on non-English mathematical reasoning, where both language coverage and reasoning capacity bind at once. On safety, per-model toxicity evaluations are reported inconsistently and are frequently absent altogether from research-grade small models, so the cohort cannot be ranked; what the literature does establish is that alignment is harder to preserve at reduced scale and that fine-tuning can erode what alignment exists [124], and that compression degrades performance unevenly across languages and dialects [28, 100]. The absence of comparable safety reporting is itself a finding, and it is one of the verification gaps named in Section 10.7. Both observations carry directly into Section 7: an agent that routes non-English or safety-sensitive calls to a small English-centric model inherits these gaps, which is an argument for capability-aware rather than size-only routing.

6 Deployment on Resource-Constrained Environments

6.1 On-Device and Mobile Deployment

On-device deployment removes the network round trip, the per-token charge, and the transmission of prompt content to a third party. It enables functionality in offline and low-bandwidth environments and is a precondition, though not on its own a guarantee, of a private personal assistant. The primary challenges are memory (RAM and storage), sustained inference throughput under thermal constraints, and battery impact.

6.1.1 What "On-Device" Does and Does Not Guarantee

The claim that on-device inference means no data leaves the device is made frequently, including in earlier drafts of this paper, and it is a property of the *application*, not of the model. Running weights locally establishes only that the prompt is not transmitted to an inference API. Whether user data leaves the device depends on four further things, none of which the model controls: whether the application emits telemetry, crash reports, or analytics that embed prompt content; whether it logs conversations to a cloud-synchronized store or a backup; whether the pipeline escalates to a cloud model, in which case the escalated calls are exactly the sensitive ones if routing is

difficulty-based rather than sensitivity-based; and whether other stages of the pipeline, such as retrieval over a hosted index or a tool call to a remote API, transmit the same content the local model was chosen to protect.

The correct statement is therefore conditional: on-device inference removes one specific egress channel, and eliminates data egress only in an application whose whole pipeline is local and whose telemetry is either absent or content-free. Apple's design [19] is instructive precisely because it treats the escalation path as the hard part and builds Private Cloud Compute around it rather than claiming the on-device model makes the question moot. Every privacy claim in this paper should be read under this scope condition, including those in Sections 8 and 9.3.

6.1.2 Apple Intelligence (AFM)

Apple's Foundation Models [19, 125] are the most fully documented on-device SLM deployment. The 2025 report [19] describes a ~3B on-device model, optimized for Apple silicon through KV-cache sharing (dividing the model so the second block reuses the first block's cache, a reported 37.5% memory saving) and 2-bit quantization-aware training, paired with a Parallel-Track Mixture-of-Experts server model on Private Cloud Compute. The on-device model is reported competitive with Qwen3-4B and Gemma-3-4B on its target tasks; the server model with Llama 4 Scout. The third-generation framework [126] adds a natively multimodal, sparsely activated on-device model and opens the same API surface to third-party cloud models, so that an application calls the local SLM by default and escalates to a frontier LLM only on demand. Google's Gemini Nano and Samsung's Gauss [127] occupy the same design point on Android and Galaxy devices.

6.1.3 Samsung Gauss

Samsung Gauss [127] is a family of language models for Galaxy devices, including a ~1.7B on-device model targeting the Gauss1 chip and Exynos 2400 NPU. Samsung reports that Gauss enables on-device email summarization, translation, and writing assistance without requiring internet connectivity.

6.1.4 General Mobile Deployment Considerations

SLMs targeting mobile deployment must navigate three constraints. (1) *Memory*: Consumer smartphones typically provide 6–12 GB RAM, of which the OS uses 2–4 GB, leaving 4–8 GB for applications. A 3B model at Q4 requires ~2 GB, comfortable for most flagship devices. (2) *Thermal*: Sustained inference on mobile NPUs is throttled by thermals; realistic sustained throughput is lower than peak throughput benchmarks suggest. (3) *Battery*: vendor guidance for mobile-targeted SLMs places per-prompt battery cost low enough to permit hundreds of interactions per charge, though published per-device figures remain scarce and are not reported under a common protocol. The fundamental constraint in edge deployment is memory bandwidth rather than compute: flagship mobile and laptop devices offer roughly 50–90 GB/s of memory bandwidth (single-board computers well under 30 GB/s) versus 2–3 TB/s for datacenter GPUs [68], making INT4 quantization not merely a storage optimization but a critical performance enabler.

6.2 Edge and IoT Deployment

Below smartphone-class hardware lies a diverse ecosystem of edge devices: Raspberry Pi single-board computers, NVIDIA Jetson modules, and microcontrollers. SLMs targeting this tier typically require:

Sub-1B or extreme quantization
: A Raspberry Pi 5 (8 GB RAM, Cortex-A76 CPU) runs Q4-quantized 1B models at roughly ten tokens per second, with sub-500M models comfortably faster. Benchmarking shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models on single-board computers [62].

Jetson Orin
: NVIDIA's Jetson Orin modules with TensorRT-LLM [128] serve quantized 3 to 7B models at interactive rates, with the larger-memory NX and AGX parts faster than the Orin Nano, enabling real-time local inference for industrial and robotics applications.

Microcontrollers (MCU)
: Sub-100M models using aggressive quantization (INT4/INT2) can run on ARM Cortex-M series MCUs with 512 KB–2 MB SRAM for specialized tasks like keyword detection and sensor data analysis.

Table 12 details the specifications of the four representative devices referenced throughout this section, from the Raspberry Pi 5 up to a laptop-class CPU.

6.3 Deployment Frameworks

A mature ecosystem of open-source frameworks enables SLM deployment across hardware targets. Table 13 provides a structured comparison.

Table 12. Detailed specifications of the devices referenced in this section.

Specification	Raspberry Pi 5	Jetson Orin Nano	Galaxy S24+	Laptop
CPU	BCM2712 4×A76 @ 2.4GHz	6×A78AE @ 1.5GHz	Snapdragon 8 Gen 3	Ryzen 7 7840HS
GPU	VideoCore VII	1024 CUDA (Ampere)	Adreno 750	Radeon 780M
NPU	None	None	Hexagon	Ryzen AI
RAM	8 GB LPDDR4X	8 GB LPDDR5 (unified)	12 GB LPDDR5X	32 GB DDR5-5600
Memory bandwidth	~17 GB/s	68 GB/s	51.2 GB/s	89.6 GB/s
Storage	microSD / NVMe	NVMe	UFS 4.0	NVMe SSD
TDP	12W	25W	~8W sustained	35–54W
Inference engine	llama.cpp	MLC LLM	ExecuTorch	llama.cpp

Table 13. Key SLM deployment frameworks, their primary targets, and characteristics. Throughput figures in the Notes column are vendor- or project-reported on the hardware each names and are not comparable across rows.

Framework	Primary Target	Quantization	Notes
llama.cpp [47]	CPU, Metal, CUDA, ROCm	GGUF (Q2–Q8)	De facto standard; minimal dependencies; ecosystem includes Ollama, llamafile
ONNX Runtime [65]	GPU, CPU, NPU, WASM	INT4/INT8	Cross-platform; DirectML/CoreML/CUDA backends; Phi-3-mini at 35 tok/s on Snapdragon NPU
MLX [129]	Apple Silicon	4-bit	Unified memory; eliminates GPU transfers; 55+ tok/s for Llama 3.2-3B on M3 Pro
TensorRT-LLM [128]	NVIDIA GPUs	INT4/INT8/FP16	Continuous batching; kernel fusion; 2–3× over PyTorch for 7B models
ExecuTorch [63]	Mobile (iOS, Android)	INT4	50 KB base runtime; 12+ hardware backends; deployed in Instagram, WhatsApp
MLC-LLM [64]	iOS, Android, web, MCU	4-bit	TVM compilation; WebGPU support; universal platform coverage
WebLLM [66]	Web browsers	4-bit WASM	WebGPU acceleration; 80% of native performance; OpenAI-compatible API

6.4 Why the Deployment Gap Matters

The practical consequence of the preceding sections is a gap that is structural rather than marginal. A quantized 3B model occupies roughly 2 GB of weight memory and runs on hardware the user already owns, so the operator pays no per-query charge for it and, subject to the scope conditions of Section 6.1.1, the prompt is not transmitted to an inference provider. A 70B model at FP16 needs roughly 140 GB, which places it on multi-accelerator server hardware, and a frontier API model adds a network round trip and a per-token charge to every call. Between those two conditions there is little continuum for a given task: the small model is either adequate for it or it is not, and where it is adequate the large model buys nothing that the task can use.

Published cost estimates put the gap at one to two orders of magnitude. Belcak, Heinrich, Diao, et al. [20] estimate that serving a 7B SLM is 10 to 30× cheaper than serving a 70 to 175B LLM once latency, energy, and FLOPs are accounted for. On the latency side, the binding constraint on edge hardware is prefill rather than generation, because time-to-first-token dominates perceived responsiveness in interactive use, and prefill cost scales with context length. This is why GQA and local-global attention interleaving, which cut KV-cache traffic, matter more for on-device deployment than for server deployment.

6.4.1 Calculated Quantities Versus Measured Ones

Because the distinction is easy to lose and consequential, we state which of the quantities in this paper are computed and which would have to be measured.

Computed, and therefore reliable within their stated assumptions: weight memory at a given precision (Table 11), which follows from parameter count and bit-width; KV-cache size per token, which follows from the head configuration and context length; and the ratio between them. These are arithmetic, and we present them as such.

Measured, and therefore dependent on conditions we do not control: throughput, time-to-first-token, sustained versus peak rates under thermal load, energy per token, battery cost per interaction, and total cost of ownership. Every one of these depends on batch size, context length, kernel implementation, achieved memory-bandwidth utilization, driver and runtime version, thermal envelope, and, for energy, the grid mix and the amortization period assumed for device manufacture. Published figures for them are not comparable across sources, and we do not

tabulate per-scenario latency, energy, or cost figures. Where a throughput figure appears in this paper it is attributed to the source and the hardware it names (Table 13), and no cross-row comparison is drawn.

One claim in this family needs particular care, because it is widely repeated and we have repeated it. Quantizing from 16 to 4 bits is often described as giving a near-linear 4× speedup. That holds under a specific regime: single-stream autoregressive *decoding*, at batch size 1, where the generation of each token requires reading the full weight set and the hardware is memory-bandwidth-bound rather than compute-bound. Outside that regime it does not. Under batching, arithmetic intensity rises and the bottleneck shifts toward compute, so the speedup shrinks. During prefill, which is compute-bound, it largely disappears. On hardware without native low-precision kernels the dequantization overhead can consume much of it. And the realized memory reduction is itself 3.2–3.4×, not 4×, because mixed schemes retain higher-precision tensors. The defensible statement is that 4-bit quantization converts a storage saving into a decode-time saving in proportion to the bandwidth it frees, and that on edge hardware in single-stream interactive use this is the dominant regime.

Subject to all of this, the claim the argument of Section 7 actually needs is ordinal and robust: for a bounded, repetitive call that a small model can complete correctly, the small model is cheaper by a wide margin, lower in latency once the network hop is counted, and available offline.

7 Small Language Models as the Substrate of Agentic AI

The preceding sections establish three things about the sub-10B regime: modern SLMs report capability levels that were state-of-the-art for far larger models only two years earlier (Section 5); they can be compressed to run on commodity and edge hardware at a modest and characterized accuracy cost (Sections 4–6); and a mature ecosystem now exists to deploy them (Section 6.3). This section assesses hypothesis H of Section 1.4: that for agent calls which are bounded, recurrent, short-context, and verifiable, a specialized SLM is the appropriate default execution substrate. The argument follows and extends the position of Belcak, Heinrich, Diao, et al. [20]; we situate it within the broader literature on LLM–SLM collaboration [21–23] and connect it to the deployment evidence consolidated in this paper.

The section is organized so that the case and its limits are separable. Section 7.1 characterizes the workload. Section 7.2 states the three premises. Section 7.3 assembles the agent-specific evidence for each premise and grades it, which is the part of the argument that general benchmark scores cannot supply. Section 7.4 states the workload classes for which the hypothesis fails. Sections 7.5–7.7 then treat routing, conversion, and adoption.

7.1 What Agents Delegate to Language Models

An AI agent is a software system in which one or more language models operate under a control loop that also includes tools, memory, and a scaffold of prompts and parsers [20]. The language model in such a system is rarely asked to hold an open-ended conversation. It is asked to perform bounded sub-tasks: decide which tool to call and with what arguments, extract fields from a document, classify an intent, rewrite a query, summarize a retrieved passage, produce a short code patch, or check whether a step succeeded. These calls share three properties. They are *narrow*: each exercises a small slice of the model's competence. They are *repetitive*: the same handful of call types recur thousands of times in production. And they are *format-constrained*: the output must parse as JSON, match a function signature, or fall within a fixed label set, or the surrounding code breaks [20].

None of these properties rewards the breadth of a frontier LLM. A model that can also write a sonnet, debate philosophy, and reason across a 200-page contract is carrying capacity that the agent never invokes on the call in question. Belcak, Heinrich, Diao, et al. [20] formalize this as the observation that agents use only a narrow subset of language-model functionality, and that agentic systems demand strict behavioral alignment to code interfaces, which is exactly the setting in which a specialized small model can match or exceed a general large one.

7.2 The Case for SLMs in Agents

7.2.1 Capability Sufficiency

The first premise is that SLMs are good enough for the delegated sub-task, where "the sub-task" means the individual bounded call and not the completed trajectory. Section 7.3.1 sets out the evidence, which for tool selection and argument filling is independent and favorable. Unified-reasoning SLMs extend the range of what can be delegated: Qwen3 [17] exposes a controllable "thinking budget" so that an agent can spend inference compute only on the calls that need it, and distilled reasoning models such as Phi-4-mini-reasoning [88] and the DeepSeek-R1 distillations [87] bring competition-mathematics performance into the 3–8B range; Srivastava, Cao, and Wang [27] benchmark 72 small models across 14 reasoning benchmarks and conclude that reasoning ability in this band tracks training method and data quality more than scale. Belcak, Heinrich, Diao, et al. [20] argue that objections based on scaling laws understate SLM capability because scaling-law studies hold architecture fixed, exclude task-specific fine-tuning,

and predate inference-time reasoning, all three of which disproportionately help small models. We accept that argument for per-call competence and note that it does not extend to the multi-step results of Section 7.3.3.

7.2.2 Economics

The second premise is cost. Belcak, Heinrich, Diao, et al. [20] estimate that serving a 7B SLM is 10–30× cheaper than serving a 70–175B LLM once latency, energy, and FLOPs are accounted for, and that parameter-efficient fine-tuning lets a small model be specialized for a new agent role in GPU-hours rather than GPU-weeks. This is a grade-C estimate rather than a measurement, and its width is not incidental: realized cost ratios depend on batch size, context length, achieved hardware utilization, and the serving stack, and the same two models can differ by a factor of three in cost ratio across two deployments. What is robust across those conditions is the ordinal claim and its rough magnitude, that the gap is one to two orders and not a few percent, and it is that ordinal claim the argument uses.

Three qualifications matter. First, the per-token comparison is the wrong unit: what a product pays is cost per *completed task*, and as Section 7.3.4 records, no source we found reports it for a small-model agent against a large-model agent on one workload. A small model that needs retries and escalations can be more expensive per completed task while being far cheaper per token. Second, "zero marginal cost" for on-device inference means no per-query cash cost to the operator; it is not zero cost to the user, who pays in battery, thermal budget, and device wear, nor zero cost in aggregate once device manufacture is amortized. Third, in offline settings the comparison does not arise: the large model cannot run at all, which makes local inference a capability question rather than an economic one. Subject to those, because an agent amortizes model cost over many calls per user task, a constant-factor saving per call compounds into the difference between a viable and an unviable margin.

7.2.3 Behavioral Alignment and Operational Fit

The third premise is control. An agent needs its model to emit exactly the format the scaffold expects, and to do so reliably across a long tail of inputs. Two mechanisms deliver this, and they are worth separating. Constrained decoding [130] enforces well-formedness outside the model, so schema adherence is not a function of model size at all; this removes the strongest structural objection to using a small model in a loop, but guarantees only syntax. Specialization supplies the rest: constraining a small model to a narrow output *distribution* is easier than suppressing the much larger behavioral range of a frontier model, and fine-tuning on the agent's own logged traces closes the residual gap [20]. Belcak, Heinrich, Diao, et al. [20] note a virtuous cycle here (their argument A7): agent interactions generate high-quality, in-distribution training data essentially for free, which is precisely what specialization of a small model requires. Operationally, small models are also easier to version, roll back, run on-premises for data-residency reasons, and co-locate with the tools they call.

7.3 Agent-Specific Evidence

The three premises above are commonly supported with general benchmark scores. As Section 5.3 argues, those scores are the wrong instrument: MMLU, GSM8K, and HumanEval measure a model answering in isolation, not a model operating inside a loop. This subsection therefore assembles the evidence that does bear on the question, organized by the capability an agent actually exercises, and grades each line under the scheme of Section 5.1.5. Table 14 summarizes.

7.3.1 Function Calling and Tool Selection

This is the best-evidenced part of the case and the only one with a standardized, independently operated evaluation. The Berkeley Function-Calling Leaderboard [131] evaluates a model's ability to select the right function from a supplied set and populate its arguments, across simple, multiple-function, parallel, and, in later versions, multi-turn and agentic settings, with executable checks rather than string matching. Its predecessors in this line, Gorilla [132], ToolLLM's ToolBench [133], and API-Bank [134], established that API-call generation is a distinct skill on which general capability transfers only partially: Patil, Zhang, Wang, and Gonzalez [132] report that a fine-tuned 7B model paired with document retrieval exceeds GPT-4 on API-call generation, and Li, Zhao, Yu, et al. [134] report a 26-point gain on tool use from targeted training of a small model.

The most direct evidence for premise (i) is that models in this size class hold competitive positions on this leaderboard. Salesforce's xLAM family [71] reports its 7B function-calling model at the top of the leaderboard at the time of its release, and its 1B variant surpassing GPT-3.5-Turbo and Claude-3 Haiku on the same task. This is a genuine grade-B result, since the leaderboard is independently operated, and it is the strongest single piece of evidence in this paper for the substrate hypothesis. Two qualifications belong with it. Leaderboard standings are a moving target, so the claim to make is that sub-10B models are *competitive* on function calling, not that a particular one leads. And function-calling accuracy is a per-call metric; Section 7.3.3 takes up what happens when such calls are chained.

7.3.2 Structured-Output Validity

An agent's scaffold breaks on malformed output regardless of whether the content was right, so schema adherence is a first-class requirement rather than a presentational one. The relevant finding here is architectural rather than comparative: constrained decoding makes the question largely moot. Reformulating generation as traversal of a finite-state machine derived from a regular expression or grammar [130] guarantees that output parses, at negligible overhead and independently of the underlying model. Because the guarantee is enforced outside the model, it applies equally to a 1B and a 400B model, which removes what would otherwise be a structural disadvantage for small models and is a real, if indirect, argument for premise (iii). What constrained decoding does not do is make the *content* correct, and the gap between well-formed and correct output is where small models still lose ground.

7.3.3 End-to-End Agent Success

This is where the case is weakest, and it needs saying plainly. On realistic multi-step agent benchmarks, absolute success rates are low for all models, and the reporting for sub-10B models is sparse. WebArena [135] reports a best GPT-4-based agent at 14.41% end-to-end success against 78.24% for humans. GAIA [136] reports GPT-4 with plugins at 15% against 92% for humans. SWE-bench [137] reports the best model of its time resolving about 2% of real GitHub issues. AgentBench [138] finds a large gap between commercial and open models across eight environments, attributing it specifically to long-horizon reasoning and instruction following rather than to knowledge.

Two things follow. First, these are frontier-model numbers, so the failures are not attributable to model size, and the honest reading is that end-to-end agentic competence is an open problem for the field rather than a settled advantage for either size class. Second, and more damaging to a naive reading of our thesis, the failure mode these benchmarks expose is precisely the one that per-call metrics hide. τ-bench [139] makes this quantitative: it introduces a pass^k metric over repeated trials of the same task and reports that even strong models succeed on under half of its tasks and degrade sharply as k grows, which is a measurement of behavioral consistency, not of capability. An agent that chains n calls at per-call reliability p completes the trajectory at p^n absent recovery, so the difference between 0.98 and 0.95 per-call reliability is the difference between 82% and 60% over ten steps. Hypothesis H's requirement (iv), that the scaffold can cheaply verify each step, is what breaks this compounding, and it is a requirement rather than a nicety for exactly this reason.

7.3.4 Routing, Escalation, and Cost per Completed Task

The heterogeneous design of Section 7.5 stands or falls on router quality, and the evidence here is thinner than the architecture diagrams suggest. RouteLLM [69] reports more than 2× cost reduction at matched quality on standard benchmarks, with transfer of a trained router to unseen model pairs, which is a real result but one obtained on single-turn query benchmarks rather than inside an agent loop. Division-of-Thoughts [70] routes decomposed sub-questions between a local SLM and a cloud LLM for on-device agents. What is missing across this literature is calibration reporting: how often the router is confidently wrong, how the escalation rate moves under distribution shift, and what the seam costs when a trajectory switches models mid-task.

Most consequentially, almost no source reports *cost per completed task*, as opposed to cost per token or cost per call. That is the quantity the economic premise actually requires, because a cheaper model that needs three attempts and one escalation is not cheaper. We were unable to find a study that measures it for a small-model agent against a large-model agent on the same workload, and we name it as the single most valuable missing measurement in this area (Section 10.7).

7.3.5 What This Evidence Supports

Read together, the evidence supports a narrower claim than the one usually made. It is well supported (B) that a specialized sub-10B model can perform the individual delegated operation, tool selection and argument filling, at a standard competitive with much larger models. It is well supported (B) that well-formedness can be guaranteed independently of model size. It is *not* established that a small-model agent completes realistic multi-step tasks at a rate comparable to a large-model agent, because that comparison has not been run; what is established is that both do poorly in absolute terms and that reliability compounds badly. And the economic premise, though plausible on per-token cost, rests on grade-C estimates and lacks the per-completed-task measurement that would confirm it.

This is why hypothesis H is conditional and why condition (iv), verifiability, is part of it rather than an implementation detail. The case for SLMs inside agents is strong exactly where the scaffold can check the work, and it is currently unproven where it cannot.

7.4 Where the Hypothesis Does Not Hold

A design hypothesis is only useful if it says when it does not apply. The following workload classes fall outside hypothesis H, and for each of them a frontier model remains the right default.

Table 14. Agent-specific evidence for the premises of hypothesis H, by capability. Grades follow Section 5.1.5. The pattern is that the evidence is strong and independent at the level of the individual call and thin to absent at the level of the completed task.

Capability	What the evidence shows	Source class	Grade
Function calling, tool selection	Sub-10B models trained for the task are competitive with much larger models on an independently operated leaderboard	Standardized leaderboard [131]; model reports [71]	B
API-call generation	Retrieval-paired 7B models exceed GPT-4 on API-call generation; targeted training yields large gains on tool use	Author-reported [132, 134]	C
Tool-use breadth	Instruction data over thousands of real APIs transfers to unseen tools	Author-reported [133]	C
Structured-output validity	Constrained decoding guarantees well-formed output at negligible cost, model-independently	Method paper [130]	B
Multi-step reliability	Success degrades sharply over repeated trials of the same task; consistency is distinct from capability	Independent benchmark [139]	B
End-to-end agent success	Absolute success is low for all model sizes on realistic environments; sub-10B coverage is sparse	Independent benchmarks [135–138]	B
Router quality	Learned routing cuts cost at matched quality on single-turn benchmarks	Author-reported [69, 70]	C
Router calibration, escalation rate	Not reported in the sources reviewed	—	None
Cost per completed task	Not reported in the sources reviewed; the decisive quantity for the economic premise	—	None
Privacy-sensitive routing	Production systems implement it; no published evaluation of routing accuracy on privacy grounds	Vendor report [19]	C

Open-ended planning over an unbounded action space
When the agent must decide *what* to do rather than execute a decided step, the call is not bounded in the sense of condition (i): there is no fixed output schema, and the space of acceptable plans is large. AgentBench [138] locates the open-model gap specifically in long-horizon reasoning, which is the capability planning needs. This is the escalation case in every routing architecture in Section 7.5, and it is not a residual case: in agent traces it is the call that determines whether the whole trajectory succeeds.

Synthesis over long or heterogeneous context
Condition (iii) fails when the input is a large document set, a long trajectory history, or a multi-source retrieval result requiring cross-document reasoning. Long context in a small model is a memory and prefill problem before it is a capability problem (Section 6.4), and quality on long-context synthesis degrades faster with scale reduction than quality on short-context extraction.

Tasks whose correctness cannot be cheaply checked
Condition (iv) fails for open-ended generation, judgement calls, and anything where verification is as hard as the task. Without a cheap check, per-call errors compound silently across the trajectory, which is exactly the regime the pass^k results of Yao, Shinn, Razavi, and Narasimhan [139] describe.

Low-resource-language and safety-sensitive calls
The multilingual and safety deficits documented in Section 5.5 are concentrated in small models, and the reporting needed to know which small model is safe for a given language is largely absent. A router that sends such calls to a small model on size grounds alone degrades them silently.

Sparse or unstable call distributions
Condition (ii) fails when a call type is rare, or when the traffic distribution shifts faster than the specialization loop of Section 7.6 can track. Specialization has a fixed cost per call type; below some call volume it does not repay, and a generalist model is simply cheaper in total.

Cold-start deployments
Before an agent is in production there are no logged traces, so the conversion pipeline has no input. The correct opening move is a frontier model with instrumentation, and migration afterwards.

Two of these deserve emphasis because they are not transitional. Planning and uncheckable generation are not gaps that a better 4B model closes; they are categories of work for which the properties that make small models attractive, narrow competence and constrained output, are the wrong properties. The heterogeneous architecture of the next subsection exists because of them, not in spite of them.

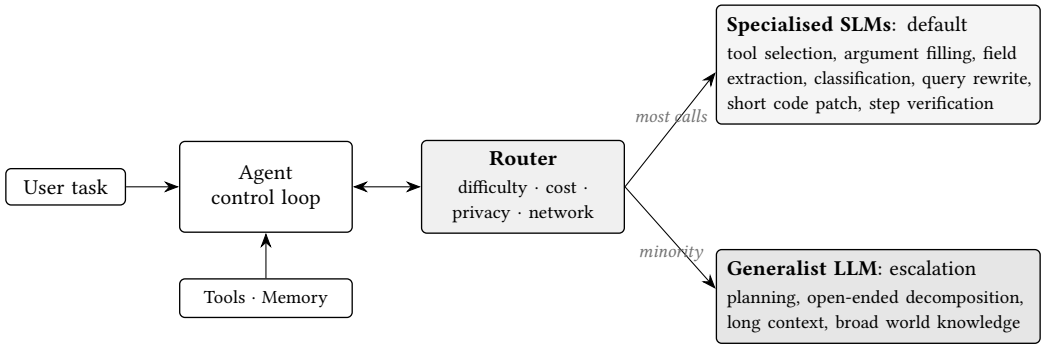


Figure 8. Heterogeneous SLM–LLM agent. A per-call router sends the majority of delegated sub-tasks to specialised small models and escalates only planning-like or long-context calls to a generalist LLM; tools and memory are shared. After Belcak, Heinrich, Diao, et al. [20].

7.5 Heterogeneous SLM–LLM Systems

The position is not that LLMs disappear from agents but that the default inverts: SLMs handle the repetitive language errands, and a strong generalist model is invoked only for planning, open-ended decomposition, or calls that genuinely need long context or broad world knowledge [20]. Figure 8 sketches the arrangement. Realizing it requires a mechanism to decide, per call, which model to use. Chen, Zhao, and Han [23] organize the collaboration literature into five interaction patterns (pipeline, routing, auxiliary, distillation, and fusion), of which routing is the most direct fit for agents.

7.5.1 Learned Routing

RouteLLM [69] trains a lightweight router on human-preference data to predict, for each query, whether a weak model's answer would be judged as good as a strong model's, and dispatches accordingly; the authors report cost reductions of more than 2× on standard benchmarks with no measurable quality loss, and transfer of the trained router to unseen strong/weak model pairs. Division-of-Thoughts [70] pushes routing below the query level, decomposing a task into sub-questions and assigning each to a local SLM or a cloud LLM according to estimated difficulty, for on-device agents. The open problems are difficulty estimation without an oracle, routing objectives that jointly weigh accuracy, latency, privacy sensitivity, and network availability, and handoff that does not produce a visible seam in output quality [140].

7.5.2 Production Instances

Two large deployments already embody the heterogeneous pattern. Apple's Foundation Models framework pairs an on-device model of roughly 3B parameters, quantized with 2-bit quantization-aware training and KV-cache sharing, with a Private Cloud Compute server model built on a Parallel-Track Mixture-of-Experts transformer, and routes to the server only when the on-device model is expected to fall short [19]. The on-device model is reported to be competitive with Qwen3-4B and Gemma-3-4B on its target tasks. Third-party providers can now register cloud models behind the same API surface as the local model, making the SLM the default and the LLM an opt-in escalation [126]. Samsung's Gauss and Google's Gemini Nano occupy the same design point on their respective platforms [127].

7.6 Converting an LLM Agent to an SLM Agent

For teams with an existing LLM-only agent, Belcak, Heinrich, Diao, et al. [20] propose a six-step conversion pipeline that we summarize here because it operationalizes the position. **S1, instrumentation:** log every non-interface model call (inputs, outputs, latency) from the running agent, with encryption and access control. **S2, curation:** strip personally identifiable and protected health information and application secrets from the logs. **S3, clustering:** apply unsupervised clustering to the curated calls to surface the recurring request types, which are the specialization targets. **S4, selection:** for each cluster, choose a candidate SLM by matching required capability against benchmarks, license, and deployment constraints. **S5, specialization:** fine-tune the chosen SLM on the cluster's curated data, typically with LoRA or QLoRA. **S6, iteration:** re-run the loop periodically as the traffic distribution drifts. The pipeline is attractive precisely because the data it needs is a by-product of running the agent, and because each cluster converted removes a recurring frontier-model bill.

7.7 Barriers to Adoption

Belcak, Heinrich, Diao, et al. [20] identify three reasons the shift has lagged the technical case. First, large centralized-inference investments create sunk-cost inertia and an operational bias toward one big model behind one API (B1).

Second, SLMs are still evaluated on generalist leaderboards (MMLU, broad chat benchmarks) that are misaligned with agentic utility, so their fitness for delegated sub-tasks is under-measured (B2). Section 5 shows the gap to be worse than a matter of emphasis: public reporting on these generalist benchmarks is itself too inconsistent to support cross-model comparison, so agentic fitness is not merely under-measured but currently unmeasurable from published sources. Third, SLM releases receive far less attention than frontier launches, so awareness lags capability (B3). The first and third are not technical obstacles and are eroding as routing frameworks mature and on-device platforms ship SLMs by default. The second is different, and Section 7.3 shows it to be partly a real evidential gap rather than only a misdirected one: agent-specific evaluation exists and is maturing [131, 138, 139], but it reports sub-10B models sparsely and reports cost per completed task essentially not at all. Until that changes, the strength of the case for hypothesis H will remain uneven across its premises in the way Table 14 records.

8 Domain-Specific Applications

The domains below are where SLM deployment is most advanced, and each is an agentic setting in the sense of Section 7: a workflow of bounded, repeated language operations (extract a diagnosis code, classify a clause, complete a function, translate a passage) executed under regulatory or cost constraints that make a cloud frontier model impractical. Domain fine-tuning of a small model is the LLM-to-SLM conversion pipeline (Section 7.6) applied to a vertical.

8.1 Healthcare and Clinical NLP

Healthcare NLP presents demanding requirements: high factual accuracy, interpretability, compatibility with clinical workflows, and operation within a regulatory regime that constrains where identifiable patient data may be processed. That regime is the pull toward on-premises models, and it needs stating precisely, because it is frequently overstated.

On-premises deployment is not compliance. Neither HIPAA nor the GDPR is satisfied by the location of a model's weights. HIPAA compliance is a property of a covered entity's administrative, physical, and technical safeguards taken together: access control, audit logging, integrity controls, transmission security, workforce training, breach notification, and business-associate agreements where a third party is involved. The GDPR requires a lawful basis for processing, purpose limitation, data minimization, a data protection impact assessment for high-risk processing, and, for automated decisions with legal or similarly significant effects, the safeguards of Article 22. A locally hosted model can fail every one of these. What on-premises deployment does is remove one specific difficulty, the transfer of identifiable data to a processor outside the entity's control, which under HIPAA otherwise requires a business-associate agreement and under the GDPR raises transfer and sub-processor questions. That is a real and often decisive simplification, and it is the reason the pull exists. It is not a compliance property of the model, and no claim in this paper should be read as asserting one. The same scope conditions as Section 6.1.1 apply: an on-premises model whose application logs prompts to a cloud service has reintroduced the transfer it was chosen to avoid.

The Meerkat family [92] demonstrates the potential of domain-specific SLM fine-tuning. Built on a 7B base with instruction fine-tuning on curated clinical QA data and chain-of-thought traces synthesized from medical textbooks, Meerkat-7B reports 70.6% on MedQA (USMLE-style) and a 64.5% average across six medical-licensing benchmarks, making it the first 7B model to clear the 60% USMLE pass threshold, and well ahead of other open 7–8B models and GPT-3.5 (54.8%), though still below GPT-4. The larger Meerkat-70B edges GPT-4 by roughly 1 point on the same average and matches it on a set of 38 complex clinical vignettes (21 vs. 21.8 correct), with both models well above the physician baseline. Either variant runs on a single A100 GPU rather than a multi-GPU cloud cluster.

OptimCLM [93] applies the full compression pipeline of knowledge distillation, structured pruning, and 4-bit quantization to clinical language models, reducing a 7B model to under 1B while preserving 95% of its accuracy on ICD coding and discharge summary generation tasks. This makes clinical NLP feasible on hospital workstations without GPU infrastructure.

The broader SLM healthcare ecosystem includes domain-pretrained biomedical models in the 2–3B range, clinical note generation models, and radiology-specific models. A key trend is the use of retrieval-augmented generation (RAG) [141] to anchor SLM outputs to verified clinical knowledge bases, reducing hallucination rates in patient-facing applications.

8.1.1 The State of the Evidence in This Domain

Healthcare is the best-evidenced of the verticals surveyed here, and it is still weaker evidence than it appears. Three qualifications apply to everything above.

Licensing-exam benchmarks are not deployment evidence. MedQA and its siblings are multiple-choice examinations designed to test candidate physicians, and performance on them establishes that a model can answer exam questions.

It does not establish accuracy on the tasks clinical systems actually perform: coding an encounter, drafting a discharge summary, extracting a medication list from a note, or flagging a contraindication. The Meerkat results are grade-C, author-reported on public benchmarks with a documented method, which is a good standard for the model literature and not the standard a clinical deployment decision requires.

Independent prospective evaluation is largely absent. We found no prospective study evaluating a sub-10B model in a live clinical workflow with outcome measurement, and no head-to-head comparison of an on-premises small model against a cloud frontier model on the same institution's data under the same protocol. Retrospective benchmark performance and deployment performance are known to diverge in clinical NLP, and nothing in the current SLM literature closes that gap.

Compression results are single-study. The OptimCLM finding [93] that a sub-1B model retains accuracy within 5% of its 7B counterpart on ICD coding and discharge summarization is one pipeline on one set of tasks. It has not been independently reproduced, and the tasks it covers are among the most template-driven in the domain, which is where compression should be expected to do best.

The honest summary is that healthcare provides the strongest published case for domain-specific small models and that the case rests on exam benchmarks and single studies rather than on deployment evidence. Establishing per-task clinical benchmarks with independent evaluation, and reporting per-group performance floors of the kind Section 10.5 argues for, is an open need in this vertical specifically.

8.2 Legal NLP

Legal document processing, which covers contract review, compliance monitoring, regulatory change analysis, and case law search, involves long and precisely structured documents where hallucination is particularly dangerous. SLMs address legal NLP deployment constraints in two ways: (1) by enabling on-premises deployment to satisfy attorney-client privilege and data security requirements; and (2) by enabling cost-effective processing of the massive volumes of routine legal work that would be prohibitively expensive with frontier LLMs.

The argument for domain-adapted SLMs (3B–7B, fine-tuned on contract corpora) in this vertical is structural rather than empirical: contract review, compliance triage, and regulatory monitoring consist of bounded, highly repetitive classification and extraction operations over documents that frequently cannot leave the client's infrastructure, which is precisely the profile the conversion pipeline of Section 7.6 targets. Narrow fine-tuning on a specific regulatory regime concentrates in-domain knowledge that a generalist model must retrieve or infer.

We state the evidential position for this vertical explicitly, because it is weaker than for the other three and it would be easy to read the structural argument above as though it were a result. Legal NLP lacks the standardized, independently operated public benchmarks that healthcare and code have. There is no legal equivalent of MedQA or HumanEval in general use, and none of the task-level benchmarks that do exist for clause classification and obligation extraction is reported across a range of small models under a common protocol. The consequence is that we can find no evidence of the grade this survey would require for a capability claim in this domain: published comparisons rest on vendor pilot reports that are not independently verifiable, and often not reproducible at all because the corpora are client-confidential. Two structural features of the domain make this worse: an error in contract or compliance work can be both consequential and hard to detect, which is exactly the condition under which hypothesis H's verifiability requirement (Section 1.4) fails; and long, cross-referential documents strain the short-context condition. Legal work is therefore a domain where the economic case for small models is strong and the capability evidence is absent, and we present the section as such rather than as a claim of parity. Establishing open legal-domain benchmarks with independent evaluation is a prerequisite for saying anything stronger.

8.3 Code Generation

Code-focused SLMs have matured into a category distinct from general SLMs, with specialized pre-training corpora (GitHub, Stack Overflow, documentation), code-aware tokenization, and evaluation on function-level benchmarks (HumanEval, MBPP, SWE-Bench).

Code Llama [94]
: Meta's family of code models fine-tuned on Llama 2 (7B, 13B, 34B). Code Llama 7B achieves 33.5% HumanEval pass@1.

StarCoder2 [95]
: BigCode's 3B/7B/15B models trained on The Stack v2 (600+ programming languages). StarCoder2-7B achieves 46.3% HumanEval pass@1.

DeepSeek-Coder [96]
: A 1B–33B family trained from scratch on 2T code tokens. The 6.7B variant achieves 47.6% HumanEval pass@1, outperforming GPT-3.5 on code completion.

Codestral [142]

Mistral's 22B code model trained on 80+ programming languages with fill-in-the-middle (FIM) training for IDE integration.

8.4 Education and Multilingual NLP

Multilingual SLMs address the linguistic coverage gap in AI assistants: the majority of LLM pre-training data is English, leaving speakers of lower-resource languages with markedly worse performance. Aya 8B [97] is fine-tuned on 204 million multilingual instructions spanning 101 languages, with strong performance across African, South Asian, and Southeast Asian languages that are severely underrepresented in standard benchmarks.

Qwen2.5 7B [14] is pre-trained on 18T tokens with strong multilingual coverage including Chinese, Japanese, Korean, Arabic, and multiple European languages, and is among the most widely downloaded open-weight models in its size class for multilingual deployment. Gemma 3's multilingual extension [41] adds instruction-tuning data for 40+ languages at the 4B and 12B scales.

Educational applications include on-device tutoring systems (leveraging Phi-3 or SmolLM on tablets), interactive language learning assistants with real-time feedback, and low-connectivity educational tools for rural and developing-world deployments. The key requirement for educational SLMs is not just multilingual capability but also low hallucination rates and age-appropriate safety filtering.

9 Discussion

9.1 What the Evidence Supports

Read together, the taxonomy (Section 3), the compression review (Section 4), and the audit of the evaluation record (Section 5) support four claims.

Capability per parameter has risen sharply. The strongest 3–4B models of 2026 report scores on mathematics, code, and knowledge benchmarks that were state of the art for 13B-class systems in 2023. Within families that publish controlled ablations, the dominant explanatory variable is training data rather than architecture: corpus composition and curation move results more than any architectural choice available at this scale. The practical implication for model builders is that data pipelines dominate architectural novelty as a lever on SLM quality.

Compression is a first-class capability, not an afterthought. INT8 is effectively lossless (B) and INT4 costs a small, non-uniform amount of accuracy, with smaller models and mathematical reasoning at the higher end (C). Because single-stream autoregressive decoding is memory-bandwidth-bound, 4-bit quantization is simultaneously a storage optimization and a decode-time speedup on edge hardware, in proportion to the bandwidth it frees and subject to the regime conditions of Section 6.4.1. Quantization preserves fidelity better than pruning below 4B parameters (B, [28]), which simplifies the practitioner's decision: quantize aggressively, and prune only when a structured size reduction is independently required. What the literature does not support, and what Section 4.1 withdraws, is a single accuracy-retention figure per technique: retention depends on baseline, model, size, task, and aggregation rule, and a number stated without them is not interpretable.

Right-sizing beats scaling for bounded tasks. A quantized 3.8B model that clears a task's accuracy threshold, running on hardware that already exists in the user's hand, is preferable on cost, latency, energy, and data exposure to a larger model that also clears it. A 3–4B model at INT4 occupies roughly 2 GB and runs at interactive speed on a phone or a Jetson-class module. The scaling-era assumption that a larger model is always the safer default is inverted for any task whose requirements can be stated in advance *and checked*. Both halves of that condition matter: where the requirement cannot be stated in advance, the task is not bounded, and where success cannot be checked, per-call errors compound silently across an agent trajectory (Section 7.3.3). Section 7.4 lists the workload classes where this claim does not apply.

The evaluation record is the weak point, and it is weaker than the field acknowledges. Section 5 documents why a cross-model accuracy comparison cannot be built from current public reporting: benchmark sets do not intersect across vendors, shot counts vary per benchmark and per vendor, base and instruction-tuned figures are conflated, and matched baselines are rarely published. This is not a limitation peculiar to this survey. It applies equally to the comparison tables circulated in secondary sources, community leaderboards, and prior surveys, which are widely reproduced without these caveats and which practitioners use to make procurement decisions. Establishing a shared, independently run evaluation for this model class would be a more valuable contribution to the field than another survey, and Section 10.7 specifies what it would require.

9.2 Implications for Agentic Systems

The agentic argument of Section 7 rests on these claims rather than restating them. Capability-per-parameter growth is what makes an SLM *sufficient* for a delegated sub-task; compression and edge deployability are what

make it *cheap* and *private*; right-sizing is the design discipline that turns a heterogeneous SLM–LLM system from an option into the default. Two cross-cutting observations sharpen the picture. First, routing must be capability-aware, not size-aware: the reported multilingual and safety gaps (Section 5.5) mean that a naive "small model unless hard" router will silently degrade non-English and safety-sensitive calls. Second, adaptation matters more as models get smaller, so the economical unit in an agent is often a *fine-tuned* small model per recurring call type rather than a single general SLM, which is exactly what the LLM-to-SLM conversion pipeline of Section 7.6 produces.

9.3 Democratization and Sustainability

Deploying a 3B model that answers grade-school mathematics correctly on an $80 single-board computer is a qualitative change in access. Organizations facing cloud-API cost, data-sovereignty rules, or unreliable connectivity can now run capable NLP locally with no per-query charge and, subject to the scope conditions of Section 6.1.1, without transferring prompt content to a processor outside their control.

The environmental case is narrower than it is usually stated and we state it narrowly. Reported per-inference energy for a small model on modest hardware is one to two orders of magnitude below a frontier-model query in a datacenter, so for a task the small model completes correctly, serving it small uses substantially less inference energy. Whether a wholesale shift reduces total system energy is a different question, and one no published accounting answers: it turns on teacher-training energy, the compression pipeline, device embodied carbon, regional grid intensity, update distribution, and the rebound effect, none of which the available reporting covers. Section 10.6.1 sets out that accounting boundary in full. We therefore make the per-query claim and decline the aggregate one.

9.4 Limitations

Beyond the evidence-base limitations already stated (Section 5.1.6), six bound this work.

No original measurement. No benchmark was executed for this survey. Every quantitative claim is therefore at best grade B (Section 5.1.5), and the paper contains no grade A evidence. This is a deliberate choice, since we judged an audit of the record more useful than adding another unreplicated table to it, but it is a limitation and Section 10.7 is written to be acted on rather than cited.

The definitional boundaries are conventions. The twenty-billion threshold, the split into conditions C1 and C2, and the sub-10B restriction on the evaluation cohort are pragmatic choices, not natural kinds. They are stated precisely (Section 1.2) so that a reader can recompute under different ones, which is the most a threshold of this sort can offer.

The thesis is a hypothesis. The agentic argument is grounded in a production-scale position paper [20], two large deployments, and the agent-specific evidence of Section 7.3, and its premises are graded individually in Table 14. The end-to-end claim, that a small-model agent completes realistic tasks at comparable rate and lower cost per completed task, is the one premise for which we found no evidence either way. It is a projection, and we mark it as one.

Coverage is English-centric. The task coverage, the benchmarks audited, and most of the compression literature are English-centric, which matters more here than usual because compression degrades non-English performance disproportionately (Section 9.5).

Domain sections are uneven. Code and healthcare have public benchmarks and are treated evidentially; legal has neither, and Section 8.2 states a structural argument rather than a capability claim. Readers should not read the four verticals as equally supported.

The snapshot ages. The ecosystem moves faster than a review cycle. The update policy of Section 1.6.5 and the versioned deposit are how we bound this, not eliminate it.

9.5 Ethical Considerations

Safety alignment is proportionally harder at small scale: several research-grade SLMs lack the safety training of their larger siblings, and fine-tuning can erode what alignment exists [124]. Practitioners deploying SLMs, and especially those wiring them into agents with tool access, should treat application-level safety filtering and output monitoring as mandatory, not optional. Compression carries its own risk: pruning and quantization can degrade performance unevenly across dialects, languages, and demographic groups [100], and the reported advantage of quantization over pruning is largest precisely on multilingual tasks [28], which suggests pruning disproportionately harms non-English users. Fairness-aware compression objectives and per-group performance floors for high-stakes applications are open requirements. Against these risks, on-device SLMs advance privacy (one egress channel removed, subject to Section 6.1.1), accessibility (no connectivity required), and cost (no subscription), which matters most for the healthcare and educational applications of Section 8 in under-resourced settings.

10 Challenges and Future Directions

Despite remarkable progress, SLMs face persistent challenges that define the frontier of the field, and the agentic framing of Section 7 adds a set of its own. We organize these into research clusters, closing with the empirical-verification agenda for this paper's consolidated evidence base.

10.1 Hallucination Mitigation in Small Models

Hallucination, the confident generation of factually incorrect content, is an intrinsic failure mode of autoregressive language models, and it is more pronounced in smaller models [143]. The core issue is information capacity: a 3B model's parameters can encode far fewer facts than a 175B model, and when queried about knowledge at the boundary of its training distribution, small models are more likely to confabulate plausible-sounding but incorrect responses.

Mitigation strategies include: (1) *RAG* [141], which anchors SLM generation to retrieved documents rather than parametric memory; (2) *uncertainty quantification* through sampling and ensemble methods; (3) *RLHF with truthfulness rewards* to penalize confident incorrect responses; and (4) *calibration post-processing* to align model confidence to actual accuracy.

10.2 Reasoning Limitations Under Parameter Constraints

Complex multi-step reasoning, particularly in mathematics, formal logic, and scientific inference, improves roughly as a power law with parameter count [6]. At the 1B–7B scale, SLMs struggle with problems that require holding many intermediate states simultaneously or with deep deductive chains.

Emerging directions include: (1) *Test-time scaling*: allocating additional inference compute (more generation tokens, repeated sampling, tree-of-thought search) to substitute for model capacity; DeepSeek-R1 [87] achieves frontier-class reasoning in its 7B distilled variant through extended chain-of-thought generation. (2) *Neuro-symbolic integration*: augmenting SLMs with symbolic solvers (SAT solvers, theorem provers, calculators) to handle formal reasoning externally. (3) *Speculative decoding* [144] using a small draft SLM to propose tokens verified by a larger model, enabling faster inference from larger models when latency permits.

10.3 Open Problems for Agentic SLM Systems

Section 7 argues that SLMs are the appropriate default substrate for the class of agent calls hypothesis H names; several problems stand between that conditional position and routine practice. **Difficulty estimation without an oracle:** a router must decide, cheaply and before generation, whether a given call exceeds the local model's competence; current approaches use learned win-prediction [69] or sub-task decomposition [70], but calibration under distribution shift is unsolved. **Multi-objective routing:** the decision must jointly weigh accuracy, latency, energy, monetary cost, privacy sensitivity, and network availability, and the reported multilingual and safety gaps (Section 5.5) mean a size-only router silently degrades non-English and safety-critical calls. **Handoff without a seam:** escalation from SLM to LLM mid-task must not produce a visible discontinuity in style or quality. **Agent-appropriate evaluation:** generalist leaderboards do not measure the tool-selection, argument-filling, and format-adherence skills agents actually use [20]; function-calling leaderboards [71, 131] and interaction benchmarks [139] are a start, but they report sub-10B models sparsely, and none reports cost per completed task (Section 7.3.4). **Fleet management:** an agent built from many fine-tuned sub-3B specialists (Section 7.6) raises questions of versioning, drift detection, and retraining cadence that single-model deployments do not face. Apple's privacy-preserving cloud escalation [19, 125] shows the pattern is production-viable; generalizing it beyond a single vendor's stack is open.

10.4 Hardware Co-Design and NPU Optimization

The efficiency of SLM inference depends as much on hardware as on model architecture. Modern consumer hardware increasingly includes dedicated AI accelerators: Apple ANE, Qualcomm Hexagon NPU [145], Samsung Exynos NPU, and Intel NPU. These accelerators are optimized for specific operation types (INT4/INT8 matrix multiplication) and data layouts.

A critical open problem is *hardware-model co-design*: developing SLM architectures whose attention patterns, FFN widths, and layer counts align with the data bus widths and tile sizes of target NPUs. GQA head counts of 8 or 16 align well with SIMD widths on ARM Cortex-A processors, while head counts of 32 may be suboptimal. Future SLMs trained with hardware constraints as first-class objectives, rather than retrofitting quantization after the fact, will likely yield significant efficiency improvements.

10.5 Ethical Considerations: Bias Amplification in Compressed Models

Model compression introduces an underexplored ethical risk: *bias amplification*. Pruning and quantization may disproportionately affect model performance on underrepresented demographic groups, dialects, and languages, as the model's remaining capacity may preferentially serve the majority distributions present in training data [100]. Zhou, Kurz, and Zhao [28] observe that quantization's advantage over pruning is most pronounced for multilingual tasks, suggesting that pruning disproportionately degrades non-English performance.

Research priorities include: (1) *fairness-aware compression* that includes demographic parity metrics as part of the compression objective; (2) *representation auditing* of compressed models across language, dialect, gender, and race; and (3) *minimum performance floors* for safety-critical applications, ensuring compressed models meet baseline accuracy thresholds for all user populations.

10.6 Sustainable AI: SLMs as a Green AI Alternative

The energy consumption of training and serving frontier LLMs has become a significant environmental and economic concern [146, 147]. Training GPT-3 emitted an estimated 552 metric tons of CO_2 equivalent [8]; training a model two orders of magnitude smaller from scratch consumes correspondingly less, and per-query inference energy is lower in proportion to the compute each query performs.

10.6.1 What the Environmental Claim Covers

Claims of this kind are routinely made at a level of generality the accounting does not support, and we withdraw one we made in an earlier draft. The statement that 10,000 concurrent small-model instances on consumer hardware consume less total energy than a single multi-GPU server is not one we can source or derive: its truth depends entirely on utilization on both sides, and a server at high batch utilization is considerably more energy-efficient per token than an idle consumer device performing occasional single-stream inference. We remove the claim rather than qualify it.

What can be said requires the boundary of the accounting to be stated. A complete comparison of a small-model deployment against a large-model one has to include: the training energy of the small model itself; the training energy of any *teacher* used to distill it, which for a distilled model can dominate and which vendors do not report; the energy of the distillation, quantization, or pruning pipeline that produced the deployed artifact; the embodied carbon of manufacturing the devices inference runs on, amortized over their service life and over the share of that life spent on inference; the marginal grid intensity where each side runs, which differs by more than a factor of ten between regions; and the energy of model updates over the deployment's lifetime, which for on-device models means redistributing weights to every device rather than updating one server.

Almost none of these terms is reported in the literature. What the field has instead is per-query inference energy on the two sides, which is a single term in a much longer sum, and which does favor the small model by one to two orders of magnitude for a query both can answer. The defensible claim is therefore narrow: *for a task a small model completes correctly, serving it with the small model uses substantially less inference energy.* Whether a wholesale shift to small models reduces total system energy is a question no published accounting answers, and the rebound effect, that cheaper inference induces more of it, cuts against it. As AI regulation incorporates carbon disclosure requirements, this accounting will have to be done properly, and the inputs for it do not currently exist.

10.7 Empirical-Verification Opportunities

Because this review synthesizes reported performance rather than producing original benchmark runs, the most immediately valuable follow-up work is targeted empirical verification of the reported claims that do the most work in practitioner decisions. Four empirical studies would disproportionately reduce the residual uncertainty in the evidence base reviewed here:

Harness-consistent reruns of the headline SLMs on the headline benchmarks
: A controlled re-run of the seven most-cited SLMs (Phi-4-mini, Phi-4, Qwen2.5-3B, Qwen2.5-7B, Gemma-3-4B, Llama 3.2-3B, Mistral-7B) on MMLU, GSM8K, HumanEval, and MBPP under a single `lm-evaluation-harness` configuration would anchor every vendor-reported figure to an independent reproduction.

Quantization-degradation measurement
: A controlled comparison of FP16 vs. INT8 vs. INT4 accuracy across the seven headline SLMs and four benchmarks, using the GGUF, GPTQ, and AWQ toolchains referenced in the vendor reports, would sharpen the 1–4-point degradation figure reported in the literature.

Edge-throughput reproduction
: Timed runs of the same seven SLMs in their best-supported INT4 format on a Raspberry Pi 5, Jetson Orin

Nano, Snapdragon reference device, and recent laptop CPU would calibrate the reported throughput ranges against modern silicon.

QLoRA-gain reproduction
A controlled QLoRA (rank 64, $\alpha = 128$) adaptation of the seven SLMs on SST-2, MNLI, ARC-C, GSM8K, and SQuAD 2.0 would quantify the reported fine-tuning gains under a single adaptation recipe.

Cost per completed task, small-model against large-model agent
The measurement this survey most needs and could not find (Section 7.3.4). Fix one agentic workload with a checkable success criterion, for instance a τ-bench domain [139] or a WebArena subset [135]. Run three configurations end to end: a frontier model throughout; a specialized SLM throughout; and a routed heterogeneous system. Report, per configuration, end-to-end task success, the number of model calls per completed task including retries, escalation rate, wall-clock latency, and total monetary and energy cost per *completed* task rather than per call. This single study would move the economic premise of hypothesis H from grade C to grade A, and it would settle whether per-token savings survive the retry and escalation overhead that per-call accounting hides.

Agent-specific evaluation of the sub-10B cohort
Run the seven headline SLMs on the Berkeley Function-Calling Leaderboard [131] and on τ-bench, reporting the pass^k consistency metric alongside single-attempt accuracy, with and without constrained decoding [130]. This would fill the sparse-coverage gap identified in Table 14 and would separate well-formedness failures from content failures, which current reporting conflates.

The first four studies are practical on a single modern GPU on the time scale of weeks. The last two require an agent harness and a modest API budget but no unusual resources, and they are the two that bear on the paper's central hypothesis rather than on its survey material. Together they would convert several consolidated claims from "reported by sources" to "empirically verified," and would supply the first grade-A evidence in an area that currently has none. They do not replace the review; they strengthen the claims that practitioners depend on most.

10.8 Open Research Directions Beyond Empirical Verification

Beyond direct empirical verification, several promising research directions emerge from this review:

Multilingual SLM evaluation
Extending the consolidated performance evidence to non-English benchmarks would reveal whether the English-centric performance hierarchy reported in the literature holds across languages, particularly for models with multilingual training data (Qwen2.5, Gemma 3, BLOOM).

On-device continual learning
Developing methods for incremental adaptation of deployed SLMs to new data without full retraining would enable personalization and domain adaptation at the edge.

SLM ensembles vs. single LLM
Investigating whether ensembles of specialized SLMs (one for math, one for code, one for general knowledge) with a lightweight router can match single-LLM performance at lower aggregate cost.

Multimodal SLMs
Evaluating vision-language SLMs such as Phi-3-vision, Gemma 3 with vision, and Gemma 3n [91] on multimodal benchmarks would extend the comparison to the rapidly growing multimodal inference domain.

Hybrid architectures
The emergence of non-transformer SLM architectures (SambaY [148], Gated Delta Networks [149]) suggests that the transformer's dominance may not extend to the edge, where linear-time inference is particularly valuable.

Federated fine-tuning
Developing privacy-preserving fine-tuning protocols that aggregate gradient updates across edge devices would enable collaborative model improvement without centralized data collection.

11 Conclusion

This paper set out to do two things at once: survey the small-language-model landscape as it stands in 2026, and argue a position about where it is going. On the first, we fixed a two-condition definition that separates the per-token cost of a model from its residency cost, cataloged 215 documented SLM variants under a multi-label faceted taxonomy, tracked the 2025–2026 shift toward hybrid state-space architectures, unified reasoning models, and mobile-first multimodal families, reviewed the compression techniques specific to the sub-10B regime, where

quantization outperforms pruning [28], and audited the public evaluation record for an 18-model cohort. That audit returned a negative result: the record does not currently support the cross-model accuracy comparison that surveys in this area routinely present.

On the second: the "less is more" thesis is no longer only about efficiency, and it is not a universal claim. The consumer of language intelligence that is growing fastest is the AI agent, and for a large share of what an agent delegates, a generalist is the wrong instrument. Where a call is bounded, recurrent, short-context, and cheaply verifiable, a small specialized model is sufficiently capable, substantially cheaper to serve, cheaper to adapt, and easier to hold to a code interface [20]. Where it is not, and the counterexamples of Section 7.4 are neither few nor transitional, a frontier model remains the right choice. The evidence assembled here supports the premises unevenly, and we have graded them rather than averaging over them: independent leaderboards support per-call competence at this size, and no published study measures cost per completed task for a small-model agent against a large-model one, which is the measurement the economic premise turns on. Apple's on-device and server split [19], together with the routing systems surveyed in Section 7, embodies the heterogeneous design at production scale. The barriers that remain are those of sunk infrastructure, measurement, and attention, and the measurement barrier is the one this paper finds to be a real evidential gap rather than only a misdirected emphasis.

The work ahead is concrete. Below roughly 3B parameters, task-specific adaptation stops being optional and becomes a precondition for reliable delegated work, and inference-time compute only partly closes the remaining reasoning gap; hallucination is worse at small scale; compression can amplify bias; hardware and model architecture are still designed separately; and the consolidated evidence base needs controlled verification of the handful of claims that practitioners most depend on (Section 10). None of these is a reason to wait, and the six experiments specified in Section 10.7 are the shortest route from a survey of what has been reported to a body of what has been established. The future of applied NLP is not one frontier model answering every query through a cloud API, and neither is it small models everywhere. It is a fabric of small, specialized, mostly on-device models doing the ordinary work, calling a large model on the planning, the long-context synthesis, and the calls nobody can check. In that architecture, less is not a limitation. It is the design.

Declarations

Funding

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU), grant number IMSIU-DDRSP2602.

Acknowledgements

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU), grant number IMSIU-DDRSP2602.

Competing interests

The authors declare that they have no competing interests.

Ethics approval and consent to participate

Not applicable. This study is a literature review and analysis of publicly available model documentation; it involves no human participants, human data, or animals.

Consent for publication

Not applicable.

Data availability

This paper is a review. All performance figures are *reported* values compiled from public sources (vendor technical reports, official model cards, community leaderboards, and third-party evaluations) and are not the product of original benchmark runs. The provenance of the consolidated figures is documented in Appendix A.

Four artifacts supporting this review are deposited in a versioned archival repository under the persistent identifier **[TO SUPPLY: archival DOI for the deposited artifacts]**, and are also provided with the manuscript source files:

The variant inventory
: `merged_llm_database.json`, one record per documented variant, carrying every field of Table 1 together with the source URL, the date that source was retrieved, the licence, the C1/C2 status (Section 1.2.1), and the inclusion decision. This dataset generates the inventory tables of Appendix B and every generated figure in the paper.

The screening ledger
every record retrieved by the searches of Section 1.6.2, with its deduplication key, screening decision, exclusion reason where applicable, and quality-appraisal scores on the four criteria of Section 1.6.3. This is the source of the counts in Table 2.

The source ledger
for the 18-model evaluation cohort, the primary document, model card, and any third-party evaluation consulted per model, with retrieval dates and the reconciliation decisions of Section 5.1.4.

A changelog
keyed to snapshot dates, recording every addition, removal, and field correction between released versions, so that the update policy of Section 1.6.5 can be audited rather than taken on trust.

Code availability

The scripts that generate the ecosystem inventory tables and the figures (`generate_model_tables.py`, `generate_figures_v2.py`) are deposited alongside the data under the identifier above and provided with the manuscript source files, so that every generated table and figure in this paper can be reproduced from the released inventory. No model-training or model-evaluation code was used to produce the results in this paper, which are compiled from the published literature.

Author contributions

M.G.R.: Conceptualization, Methodology, Investigation, Data curation, Software, Writing (original draft), Visualization. **H.A.:** Conceptualization, Methodology, Writing (review and editing), Supervision, Project administration. **S.J.A.:** Investigation, Validation, Writing (review and editing). **W.E.A.:** Validation, Writing (review and editing), Funding acquisition. All authors read and approved the final manuscript.

A Data Provenance and Supplementary Protocols

This appendix supports the reproducibility of the *review itself*. It records where the quantitative material in Sections 5 and 6 comes from, and documents the evaluation and compression conventions the reviewed sources report, so that a reader can judge any individual figure against the conditions behind it. Detailed device specifications appear in Section 6, Table 12. The appendix reports no original experimental runs.

A.1 Provenance of the Consolidated Figures

Provenance is recorded at two levels. *Model-level* provenance fixes, for each model in the cohort, the primary release document from which its architectural specification and headline benchmark scores derive (Table 15). *Source-class* provenance records, for each consolidated table, the categories of source the figures are drawn from and the reconciliation rules applied (Section A.2). Benchmark definitions and metric variants follow the originating task papers cited in Section 5.1.2.

Table 15. Model-level provenance for the reviewed cohort: the primary release document from which each model's architectural specification and headline reported scores derive.

Model(s)	Developer	Primary source	Source type
TinyLlama-1.1B	StatNLP / SUTD	Zhang, Zeng, Wang, and Lu [55]	Technical report
SmolLM2-1.7B, -360M	Hugging Face	Allal, Lozhkov, Penedo, et al. [57]	Technical report
StableLM 2-1.6B	Stability AI	Stability AI [60]	Technical report
Qwen2.5-0.5B/1.5B/3B/7B	Alibaba	Qwen Team, Alibaba Group [14]	Technical report
Llama 3.2-1B/3B	Meta	Llama Team, AI @ Meta [4]	Technical report
Gemma 2-2B/9B	Google DeepMind	Gemma Team, Google DeepMind [38]	Technical report
Gemma 3-1B/4B	Google DeepMind	Gemma Team, Google DeepMind [41]	Technical report
Mistral 7B	Mistral AI	Jiang, Sablayrolles, Mensch, et al. [56]	Technical report
Phi-3-mini	Microsoft	Abdin, Aneja, Awadalla, et al. [12]	Technical report
Phi-3.5-mini	Microsoft	Abdin et al. [52]	Technical report
Phi-4-mini	Microsoft	Microsoft Research [39]	Technical report
Phi-4	Microsoft	Abdin et al. [44]	Technical report
Ministral 8B	Mistral AI	Vendor model card	Model card

A.2 Provenance Status by Table

Every quantitative table retained in this paper falls into one of three categories, and the category determines how much weight it can carry.

Sourced from primary release documents. The architectural specifications in Table 8 are taken from the release document listed for each model in Table 15. Layer counts, hidden sizes, attention configurations, context lengths, vocabulary sizes and training-token counts are stated directly in those documents and are stable across printings.

Computed from stated quantities. The weight footprints in Table 11 follow arithmetically from parameter count and storage bit-width and are not measurements. The device specifications in Table 9 and Table 12 come from manufacturer datasheets.

Structural properties of a technique. Table 6 states what each compression technique reduces by construction. Its entries are consequences of how the technique works, not measurements: the memory factor of n-bit quantization follows from the bit-width, and the parameter factor of s-fraction pruning follows from the sparsity level. Where a cell reports a figure from a specific study, that study is cited in the cell.

Qualitative synthesis of the reviewed literature. The quality-cost statements in Table 7, the framework comparison in Table 13, the survey comparison in Table 3, the taxonomy summary in Table 5, and the agent-evidence summary in Table 14 summarize the works cited in their respective sections. They are positional rather than metric, and each row of the two graded tables carries the class of evidence standing behind it (Section 5.1.5).

Withdrawn. An earlier version of this paper carried a table giving a single "compression ratio" and a single "accuracy retention" percentage per technique. Both columns have been withdrawn. Section 4.1 sets out the reasons: the compression ratio conflates reductions in parameters, memory, and training compute, which trade against different budgets, and is undefined for distillation, where the student size is a design choice rather than a property of the method; and an accuracy-retention percentage is uninterpretable without a baseline precision, model, size, task, metric, and aggregation rule, all of which the retained figure had averaged over. They are replaced by Tables 6 and 7. They should not be reinstated without per-cell provenance of the kind Section A.1 requires.

No table in this paper reports a cross-model accuracy comparison. As Section 5.2 sets out, such a table cannot be assembled from current public reporting without mixing incompatible shot counts, inconsistent base and instruction-tuned checkpoints, and benchmark sets that do not intersect. We considered it preferable to state that finding and omit the table than to publish a matrix whose cells could not each be traced to a source evaluated under the conditions its column header claims.

A.3 Prompt Formats

The reviewed source materials commonly preserve each model's own instruction wrapper while following a shared task-payload structure, which is summarized here in prose. **Multiple-choice question answering** presents the question followed by the enumerated options (A–D) and an "Answer:" cue, with the model expected to emit a single option letter. **Mathematical reasoning** prepends an instruction to solve the problem step by step and appends a "Let's think step by step" cue, so that a chain-of-thought precedes the final numeric answer. **Extractive question answering** supplies the context passage, then the question, then an instruction to answer from the context and to respond "unanswerable" when the context does not contain the answer. **Abstractive summarization** supplies the source article preceded by an instruction to produce a two- to three-sentence summary. Where a source reports few-shot results, the same fixed set of exemplars is prepended to the payload for every model on a given task. Because sources differ in the exact wording of these wrappers and in answer-extraction heuristics, the consolidated figures are compared only within tasks and, wherever possible, within a single reporting source.

A.4 Quantization and Compression Pipeline

To complement the methodology section, the compression workflows reported across the reviewed sources are summarized here. The reviewed literature reports five precision or compression settings across the consolidated comparison:

FP16 baseline
: Standard half-precision inference used as the reference condition.

GGUF Q4_K_M
: Mixed-precision 4-bit quantization via `llama.cpp`, assigning higher precision to the most sensitive tensors.

AWQ 4-bit
: Activation-aware weight quantization with group size 128 and calibration on held-out text sequences.

GPTQ 4-bit
: Layer-wise post-training quantization using second-order curvature approximations.

bitsandbytes NF4
: 4-bit NormalFloat quantization with double quantization, matching the QLoRA training regime.

For compression stress tests, the sources report SparseGPT [49]-induced 50% unstructured sparsity applied before 4-bit quantization. Reported figures for this combined pruning-plus-quantization setting are retained in the provenance tables for sensitivity analysis rather than the main comparative tables, because support for sparse inference remains hardware dependent and is not uniformly reported across the reviewed sources.

A.5 Human-Evaluation Conventions in the Source Literature

For subjective tasks such as summarization quality and instruction following, the reviewed sources that report human judgments typically rate outputs along three axes (**relevance**, **coherence**, and **factual accuracy**) on a 1–5 Likert scale, with model identity hidden from annotators and inter-annotator reliability reported via a Krippendorff-style coefficient. MT-Bench-style results substitute a strong LLM judge for human raters. Because these protocols differ across sources in rubric wording and judge model, and because per-model coverage is sparse, instruction-following and summarization are not tabulated in this review and are discussed only qualitatively.

B Broader Small-Model Ecosystem Context

This appendix situates the reviewed SLM cohort within the wider model ecosystem, which matters because the 2025–2026 release cycle moved compact open models, open-weight reasoning models, and frontier API systems in parallel rather than in sequence. The summary below records the ecosystem context needed to interpret the cohort, followed by the full generated variant inventory.

B.1 Landscape Snapshot

Three ecosystem trends are especially relevant to the present study:

Capability compression
: Reasoning, long-context handling, and multimodality have moved into the 1B–8B range, narrowing the gap between deployable SLMs and cloud-scale LLMs.

Open-weight acceleration
: Families such as Qwen, Phi, Gemma, SmolLM, DeepSeek distills, Granite, and MiniCPM have expanded the design space for self-hosted deployment (Figure 11).

Deployment diversification
: Inference targets now span cloud GPUs, consumer GPUs, laptops, smartphones, browsers, and embedded edge boards, making model choice inseparable from hardware choice.

The small-active-parameter turn
: The most distinctive feature of the mid-2026 wave is how much of it activates 3–13B parameters against totals of 30B to 284B: Nemotron 3.5 Lightning (30B/3B), Hy-MT2-30B-A3B, North Mini Code (30B/3B), Laguna XS 2.1 (33B/3B), Ling 3.0 Flash (124B/5.1B) and its domain variants, and DeepSeek V4 Flash (284B/13B). These are the models that make the C1/C2 distinction load-bearing rather than pedantic, since each is cheap per token and none is resident on a phone.

The May–September 2026 additions to the inventory illustrate both ends of the range. At the small end, IBM shipped Granite 4.1 and then Granite 4.2 in 3B, 8B, and 30B dense sizes with a thinking switch, Liquid AI extended LFM2.5 from 230M to an 8.3B/1.5B on-device MoE, and Tencent released the Hy-MT2 translation family at 1.8B and 7B. At the frontier, OpenAI moved from GPT-5.6 (Sol, Terra, and Luna tiers) to GPT-6 Astra, Anthropic shipped Opus 4.8, Fable 5, Sonnet 5, Opus 5, and Fable 5.1 in a single quarter, Google iterated Gemini Flash from 3.1 to 3.8, and Meta's restructured Superintelligence Labs released the proprietary Muse Spark line alongside Muse Glimmer 30B, its first open-weight model since Llama 4.

Figure 9 gives a size-proportional overview of the inventory by developer and family, and Figure 10 tracks the entry of new developers into the open-SLM space.

B.2 Representative Family Reference

B.3 Classification Conventions

The inventory applies conditions C1 and C2 of Section 1.2.1 mechanically, from the parameter counts each record carries, rather than by editorial judgement:

- A record enters the SLM inventory when its *active* parameter count is at or below roughly 20B (C1). Dense models, for which active equals total, therefore enter at or below 20B total.



Figure 9. Documented SLM variants grouped by developer and family; area is proportional to the number of variants. Generated from `merged_llm_database.json` (snapshot 5 September 2026).

- Within the inventory, C2 (total at or below 20B) separates SLMs proper from small-active-parameter MoE models. Both are listed; only the former carry the paper's storage, resident-memory, and on-device claims.
- Where a source states no active-parameter count for a model it describes as mixture-of-experts, the record is marked undisclosed rather than having active silently set equal to total, which would misclassify a small-active model as a large one on no evidence.
- Closed API models with undisclosed parameter counts are treated as frontier LLM baselines, with five exceptions retained on explicit vendor positioning as small models and named in Section B.4.

Applying these conditions mechanically is what changed the headline count between drafts, from 126 to 215; Section 1.2.2 records the reconciliation and the released changelog lists it per model.

B.4 Generated Variant Inventory

The following inventory is generated directly from `merged_llm_database.json` using `scripts/generate_model_tables.py`. The database covers 215 SLM variants and 224 LLM variants across 113 families (439 entries in total, snapshot 5 September 2026); only the SLM inventory is reproduced here, organized chronologically by release year and then by family, with parameter size as an explicit comparison column. The corresponding LLM inventory is available in the source database.

Each record in the released database carries the fields `meets_c1_active_le_20b`, `meets_c2_total_le_20b` and `slm_class`, so a reader can recompute the inventory under a different threshold or restrict it to models satisfying both conditions. Of the 215 entries reproduced here, 183 carry `slm_class = slm`, 27 carry `small-active-moe`, and 5 carry `undisclosed`. The `undisclosed` entries are vendor-positioned small models (Amazon Nova Micro and Lite, GPT-4o mini, Voxtral Mini, Nex-N2-Mini) for which no parameter count has been published; they are listed by name in the database changelog rather than assigned an assumed size, and no memory or on-device claim in this paper rests on them.

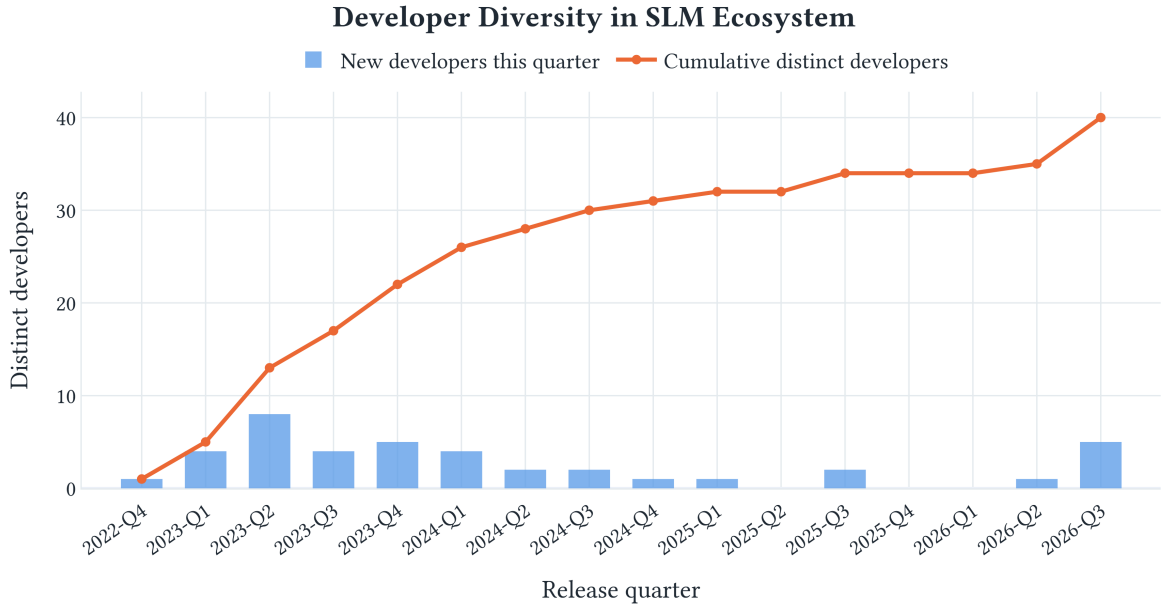


Figure 10. Cumulative distinct developers releasing documented SLM variants (line) and new developers per quarter (bars).

Table 16. Representative model families tracked in the inventory.

Family	Developer	Representative Versions	Access	Typical Role
Qwen2.5 / Qwen3	Alibaba	0.5B–14B dense, MoE variants	Open weights	Strong multilingual, math, and code performance
Phi-3 / Phi-4	Microsoft	3.8B mini variants, 14B reasoning	Open weights	Data-curated reasoning-focused SLMs
Gemma 2 / 3 / 3n	Google DeepMind	1B–12B dense, mobile-first multimodal variants	Open weights	Distilled compact models with strong safety tuning
Llama 3.2 / 4	Meta	1B–3B edge models, larger MoE frontier models	Open weights	Broad deployment ecosystem from edge to server
SmolLM2 / SmolLM3	Hugging Face	135M–3B	Open weights	Lightweight, fully open research and edge baselines
DeepSeek distills	DeepSeek	1.5B–14B distilled reasoning models	Open weights	Reasoning transfer into self-hosted sizes
Granite 3 / 4	IBM	2–8B dense; 4.0 hybrid Mamba-2/MoE (350M–32B, 1–9B active)	Open weights	Enterprise-friendly, tool-use and RAG focus
Falcon-H1	TII	0.5–34B parallel attention/Mamba-2 hybrid	Open weights	Long-context edge-to-server
LFM2 / LFM2.5	Liquid AI	230M–8.3B hybrid conv/attention, incl. 8B-A1B MoE	Open weights	Fastest-on-CPU on-device models
MiniCPM	OpenBMB	Compact multimodal variants	Open weights	Edge multimodal use cases
Ling / Nemotron 3.5 / Laguna	inclusionAI / NVIDIA / Poolside	Small-active MoE, 3–13B active	Open weights	SLM-like per-token cost at LLM-like residency
GPT-5.6 / GPT-6 / Claude 5 / Gemini 3.x	OpenAI / Anthropic / Google	API frontier models	API only	High-end proprietary baselines and teacher models
Grok / Kimi / GLM / Muse / Nova	xAI / Moonshot / Zhipu / Meta / AWS	Large API or MoE systems	Mixed	Frontier alternatives with long context or agent focus

Chronology. SLM inventory: 2026, 2025, 2024, 2023 and earlier.

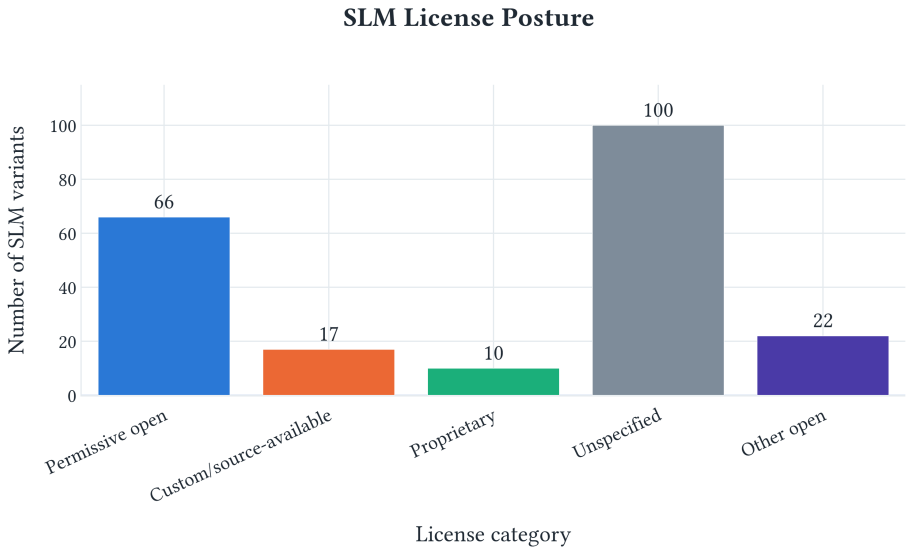


Figure 11. License posture of documented SLM variants. Permissive open licenses (Apache-2.0, MIT) account for the plurality; "unspecified" entries are typically research checkpoints without an attached license file.

Table 17. Generated chronological small language model inventory from `merged_llm_database.json`, grouped by release year and family.

Model	Access	Params	Context	Modalities	License	Release	Notes
2026							
Qwen3.5 — Alibaba (Qwen3.5 (Mar 2026)) [family page]							
Qwen3.5-0.8B	OW	0.8B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	ultra-lightweight multimodal; on-device
Qwen3.5-2B	OW	2B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	on-device multimodal chat; edge AI
Qwen3.5-4B	OW	4B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	mobile multimodal assistant; agentic coding
Qwen3.5-9B	OW	9B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	strong multimodal SLM; coding
Qwen3.5-35B-A3B	OW	35B (3B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	efficient MoE for edge/server; multimodal reasoning
Qwen3.5-122B-A10B	OW	122B (10B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	strong MoE reasoning; locally runnable flagship
Qwen3.5-397B-A17B	Both	397B (17B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	flagship open-weight multimodal; 201 languages
AFM — Apple (AFM) [family page]							
Apple AFM 3 Core Advanced	API	20B (1-4B act.)	64K	Text+Vision	Proprietary	2026-06-08	on-device assistant; multimodal understanding
Command — Cohere (Command) [family page]							
North Mini Code 1.0	OW	30B (3B act.)	250K	Text	CC-BY-NC (Cohere Labs)	2026-06-17	coding; agentic coding
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek V4 Flash Vision Exp	OW	284B (13B act.)	1M	Text+Vision	See source	2026-08-21	vision; long context
DeepSeek V4 Flash 0731	OW	284B (13B act.)	1280K	Text	See source	2026-07-31	general; long context
Dots — Dots Studio (Dots3-Note Preview (Aug 2026)) [family page]							
Dots3-Note Preview	OW	280B (16B act.)	500K	Text	See source	2026-08-14	note taking; general
Gemma 4 — Google DeepMind							
Gemma 4 E2B	OW	~5B (~2B effective act.)	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	on-device; multimodal
Gemma 4 26B (MoE)	OW	26B (3.8B act.)	128K	Text+Vision	Apache 2.0	2026-04-02	reasoning; multimodal
Gemma 4 E4B	OW	~8B (~4B effective act.)	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Gemma 4 12B	OW	12B	256K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Granite 4 — IBM							
Granite 4.2 3B	OW	3B	128K	Text	Apache 2.0	2026-08-25	reasoning; on-device
Granite 4.2 8B	OW	8B	128K	Text	Apache 2.0	2026-08-25	reasoning; agentic
Granite 4.1 3B	OW	3B	128K	Text	Apache 2.0	2026-04-29	enterprise; on-device
Granite 4.1 8B	OW	8B	128K	Text	Apache 2.0	2026-04-29	enterprise; agentic

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
LFM2 — Liquid AI							
LFM2.5-2.6B	Both	2.69B	128K	Text	LFM1.0	2026-08-04	agentic workflows; data extraction
LFM2.5-8B-A1B	OW	8.3B (1.5B act.)	128K	Text	LFM1.0	2026-05-28	on-device; tool calling
LFM2.5-350M	OW	350M	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-1.2B	OW	1.2B	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-Audio-1.5B	OW	1.5B	–	Text+Audio	LFM1.0	2026	on-device speech; transcription
LFM2.5-VL-3B	OW	3B	–	Text+Vision	LFM1.0	2026	on-device VLM; document extraction
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral Small 4 (119B)	Both	119B (6.5B act.)	256K	Text+Vision	Apache 2.0	2026-03-16	hybrid reasoning; coding
NVIDIA Nemotron — NVIDIA (Nemotron 3 Super (Mar 2026)) [family page]							
NemotronLabs VoiceChat 11B	OW	11B	128K	Text+Audio	NVIDIA Custom	2026-08-03	voice interaction; on-device speech
Nemotron 3 Super	OW	120B (12B act.)	1M	Text	NVIDIA Open Model License	2026-03	open hybrid Mamba-Transformer MoE; agentic reasoning
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron 3.5 Lightning 30B-A3B	OW	30B (3B act.)	256K	Text	NVIDIA Open Model License	2026-08-11	agentic execution layer; high-volume tool calling
Nemotron 3.5 Content Safety	OW	4B	128K	Text+Vision	NVIDIA Open Model License	2026-06-04	guardrails; content moderation
Nex-N2 — Nex AGI (Nex-N2-Mini (Jun 2026)) [family page]							
Nex-N2-Mini	OW	Undisclosed	256K	Text+Vision	See source	2026-06-24	agentic; coding
Nex-N2-Pro	OW	397B (17B act.)	256K	Text+Vision	See source	2026-06-08	agentic; coding
Laguna — Poolside (Laguna S 2.1 (Jul 2026)) [family page]							
Laguna S 2.1	OW	118B (8B act.)	1M	Text	See source	2026-07-21	coding agent
Laguna XS 2.1	OW	33B (3B act.)	256K	Text	See source	2026-07-02	coding agent
Hunyuan — Tencent (Hunyuan) [family page]							
Hy-MT2-1.8B	OW	1.8B	8K	Text	Tencent Hunyuan License	2026-08-20	translation; on-device
Hy-MT2-30B-A3B	OW	30B (3B act.)	8K	Text	Tencent Hunyuan License	2026-08-20	translation
Hy-MT2-7B	OW	7B	8K	Text	Tencent Hunyuan License	2026-08-19	translation; Lang: 33 language pairs + 5 dialect/minority pairs
Inkling — Thinking Machines Lab (Inkling Small (Jul 2026)) [family page]							
Inkling Small	OW	276B (12B act.)	1M	Text+Vision	See source	2026-07-30	general; multimodal
MiMo — Xiaomi (MiMo-V2-Pro (Mar 2026)) [family page]							
MiMo-V2-Flash	OW	309B (15B act.)	256K	Text	Undisclosed	2026	high-throughput coding agents; production serving
Ling — inclusionAI (Ling-1T (2026)) [family page]							
Ling 3.0 Flash Sante	API	124B (5.1B act.)	256K	Text	See source	2026-09-04	health; medicine

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Ling 3.0 Flash Fin	API	124B (5.1B act.)	256K	Text	See source	2026-08-27	finance
Ling 3.0 Flash	OW	124B (5.1B act.)	256K	Text	See source	2026-07-23	general; agentic
2025							
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Omni-7B	OW	7B	–	Text+Audio+Video+Image	See source	2025-03-26	omni-modal; text and audio output
Qwen2.5-VL-7B	OW	7B	–	Text+Video+Image	See source	2025-01-22	active
Qwen3 — Alibaba (Qwen3 (Apr 2025)) [family page]							
Qwen3-4B-Instruct-2507	OW	4B	128K	Text	Apache 2.0	2025-07	improved instruction following; non-thinking mode optimised
Qwen3-0.6B	OW	0.6B	32K	Text	Apache 2.0	2025-04-29	edge inference; IoT
Qwen3-1.7B	OW	1.7B	32K	Text	Apache 2.0	2025-04-29	mobile inference; Q&A
Qwen3-30B-A3B	OW	30B (3B act.)	128K	Text	Apache 2.0	2025-04-29	MoE efficiency; broad capability in tiny footprint
Qwen3-4B-Instruct	OW	4B	128K	Text	Apache 2.0	2025-04-29	on-device chat; coding
Qwen3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04-29	single-GPU inference; reasoning
Qwen3-14B-Instruct	OW	14B	128K	Text	Apache 2.0	2025-04-29	high-quality on-premise chat; coding
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device, 2nd gen)	API	~3B	64K	Text+Vision	Proprietary	2025-06-09	on-device assistant; writing tools
DeepSeek R1-Distill — DeepSeek (R1-Distill (Jan 2025)) [family page]							
DeepSeek-R1-Distill-Qwen-1.5B	OW	1.5B	128K	Text	MIT	2025-01-20	reasoning at ultra-small scale; math
DeepSeek-R1-Distill-Qwen-7B	OW	7B	128K	Text	MIT	2025-01-20	chain-of-thought reasoning; coding
DeepSeek-R1-Distill-Llama-8B	OW	8B	128K	Text	MIT	2025-01-20	reasoning on Llama 3 base; on-premise
DeepSeek-R1-Distill-Qwen-14B	OW	14B	128K	Text	MIT	2025-01-20	strong reasoning 14B; top distill for coding
Gemma 3 — Google DeepMind (Gemma 3 (Mar 2025)) [family page]							
Gemma 3 270M-IT	OW	270M	32K	Text	Google Gemma License	2025-03	ultra-lightweight prototyping; experiments
Gemma 3 1B-IT	OW	1B	32K	Text	Google Gemma License	2025-03	on-device mobile inference; lightweight Q&A
Gemma 3 4B-IT	OW	4B	128K	Text+Vision	Google Gemma License	2025-03	multimodal on-device; image understanding
Gemma 3 12B-IT	OW	12B	128K	Text+Vision	Google Gemma License	2025-03	high-quality assistant; 140+ languages
Gemma 3n — Google DeepMind (Gemma-3n (2025)) [family page]							
Gemma-3n-E2B-IT	OW	~5B (~2B effective act.)	32K	Text+Vision+Audio+Video	Google Gemma License	2025	mobile-first multimodal; on-device speech/vision/text

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Gemma-3n-E4B-IT	OW	~8B (~4B effective act.)	32K	Text+Vision+Audio+Video	Google Gemma License	2025	higher-accuracy mobile multimodal; edge devices
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM3-3B	OW	3B	64K	Text	Apache 2.0	2025	dual-mode reasoning; tool calling
Granite — IBM (Granite) [family page]							
Granite 3.2-2B	OW	2B	125K	Text	See source	2025-02-26	Thinking mode; 128K context
Granite 3.2-8B	OW	8B	125K	Text	See source	2025-02-26	Thinking mode (chain-of-thought); 128K context
IBM Granite — IBM (Granite 3.3 (Apr 2025)) [family page]							
Granite 4.0 Nano 350M	OW	350M	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 Nano 1B	OW	1B	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 H-Tiny	OW	7B (1B act.)	128K	Text	Apache 2.0	2025-10-02	low-latency; agentic
Granite 4.0 H-Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; tool use
Granite 4.0 Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; RAG
Granite 4.0 H-Small	OW	32B (9B act.)	128K	Text	Apache 2.0	2025-10-02	enterprise; agentic
Granite 3.3-2B-Instruct	OW	2B	128K	Text	Apache 2.0	2025-04	enterprise NLP; governed AI
Granite 3.3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04	enterprise reasoning; Fill-in-Middle code
Jan — Jan.ai (Jan-v1-4B (2025)) [family page]							
Jan-v1-4B	OW	4B	128K	Text	Apache 2.0	2025	agentic reasoning; tool use
LFM2 — Liquid AI							
LFM2-8B-A1.5B (MoE)	OW	8.3B (1.5B act.)	32K	Text	LFM Open License v1.0	2025-08	on-device; agentic
LFM2-2.6B	OW	2.6B	32K	Text	LFM Open License v1.0	2025-08	on-device
LFM2-350M	OW	350M	32K	Text	LFM Open License v1.0	2025-07-10	on-device; low latency
LFM2-700M	OW	700M	32K	Text	LFM Open License v1.0	2025-07-10	on-device
LFM2-1.2B	OW	1.2B	32K	Text	LFM Open License v1.0	2025-07-10	on-device; agentic
Llama 4 — Meta AI (Llama 4 Maverick (Apr 2025)) [family page]							
Llama 4 Maverick	Both	400B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	image understanding; coding
Llama 4 Scout	Both	109B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	extreme long-context; single H100 runnable
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4-mini-flash-reasoning	OW	3.8B	128K	Text	MIT	2025-04	fast reasoning; edge agents
Phi-4-mini-reasoning	OW	3.8B	128K	Text	MIT	2025-04	chain-of-thought reasoning; math
Phi-4-reasoning	OW	14B	128K	Text	MIT	2025-04	complex reasoning; STEM
Phi-4-reasoning-plus	OW	14B	128K	Text	MIT	2025-04	RL-trained reasoning; math olympiad-level
Phi-4-mini-instruct	OW	3.8B	128K	Text	MIT	2025-02	on-device chat; function calling

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral 3 (14B)	OW	14B	–	Text+Vision	See source	2025-12-02	edge deployment; multimodal
Ministral-3B-Instruct-2512	OW	3.8B	256K	Text+Vision	Apache 2.0	2025-12	tiny multimodal; function calling
Voxtral — Mistral AI (Voxtral) [family page]							
Voxtral Mini	API	undisclosed	–	Text+Audio	Proprietary	2025-09-15	speech understanding; transcription
Nemotron — NVIDIA (Nemotron) [family page]							
NVIDIA Nemotron Nano 2	OW	9B	128K	Text	See source	2025-08-18	on-device; efficient
GPT-OSS — OpenAI (gpt-oss-120b (2025)) [family page]							
gpt-oss-20b	OW	21B (3.6B act.)	128K	Text	Apache 2.0	2025	edge reasoning; 16GB device inference
gpt-oss-120b	OW	117B (5.1B act.)	128K	Text	Apache 2.0	2025	open-weight reasoning; near o4-mini parity
MiniCPM — OpenBMB (MiniCPM-V 4.5 (2025)) [family page]							
MiniCPM-V 4.5	OW	~8B	32K	Text+Vision	Apache 2.0	2025	mobile vision-language; on-device VQA
MiniCPM-o 3.0	OW	~9B	32K	Text+Vision+Audio	Apache 2.0	2025	end-to-end multimodal on-device; live streaming
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM3-8B	OW	8B	32K	Text	See source	2025-01-09	Reasoning via deep thinking; Knowledge-rich training pipeline
Falcon-H1 — Technology Innovation Institute							
Falcon-H1-0.5B	OW	0.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-1.5B	OW	1.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-3B	OW	3B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
Falcon-H1-7B	OW	7B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
2024							
Yi — 01.AI (Yi) [family page]							
Yi-1.5-6B	OW	6B	–	Text	See source	2024-05-13	legacy
Yi-1.5-9B	OW	9B	–	Text	See source	2024-05-13	legacy
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Coder-1.5B	OW	1.5B	–	Text	See source	2024-11-12	compact coding model
Qwen2.5-14B	OW	14B	–	Text	See source	2024-09-19	active
Qwen2-VL-7B	OW	7B	–	Text+Video+Image	See source	2024-08-28	vision-language; video understanding
Qwen2-Audio-7B	OW	7B	–	Text+Audio	See source	2024-08-09	audio understanding; speech recognition
Qwen2-0.5B	OW	0.5B	–	Text	See source	2024-06-07	active
Qwen2-1.5B	OW	1.5B	–	Text	See source	2024-06-07	active
Qwen2-7B	OW	7B	–	Text	See source	2024-06-07	active
Qwen1.5-MoE-A2.7B	OW	14.3B	–	Text	See source	2024-03-28	MoE architecture; efficient

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Qwen1.5-0.5B	OW	0.5B	–	Text	See source	2024-02-05	legacy
Qwen1.5-7B	OW	7B	–	Text	See source	2024-02-05	legacy
Qwen1.5-14B	OW	14B	–	Text	See source	2024-02-05	legacy
Qwen2.5 — Alibaba (Qwen2.5 (Sep 2024)) [family page]							
Qwen2.5-0.5B-Instruct	OW	0.5B	32K	Text	Apache 2.0	2024-09-19	extreme edge/IoT; classification
Qwen2.5-1.5B-Instruct	OW	1.5B	32K	Text	Apache 2.0	2024-09-19	edge NLP; lightweight assistants
Qwen2.5-3B-Instruct	OW	3B	32K	Text	Apache 2.0	2024-09-19	mobile chat; summarization
Qwen2.5-7B-Instruct	OW	7B	128K	Text	Apache 2.0	2024-09-19	general assistant; coding
Qwen2.5-Coder — Alibaba (Qwen2.5-Coder (Nov 2024)) [family page]							
Qwen2.5-Coder-7B-Instruct	OW	7B	128K	Text+Code	Apache 2.0	2024-11	code generation; debugging
Qwen2.5-Coder-14B-Instruct	OW	14B	128K	Text+Code	Apache 2.0	2024-11	advanced code assistant; agentic code tasks
OLMo — Allen Institute for AI (Ai2) (OLMo) [family page]							
OLMo 2-7B	OW	7B	–	Text	See source	2024-11-26	fully open; training code included
OLMo 2-13B	OW	13B	–	Text	See source	2024-11-26	active
OLMo-1B	OW	1B	2K	Text	See source	2024-02-01	Fully open (data, code, weights, logs); Dolma training data
OLMo-7B	OW	7B	–	Text	See source	2024-02-01	fully open; training data/code/logs released
Tülu — Allen Institute for AI (Ai2) (Tülu) [family page]							
Tülu 3-8B	OW	8B	–	Text	See source	2024-11-22	instruction following; DPO
Amazon Nova — Amazon (Nova Premier (2025)) [family page]							
Amazon Nova Lite	API	Undisclosed	300K	Text+Vision	Proprietary (AWS)	2024-12-03	multimodal understanding; cost-efficient AWS inference
Amazon Nova Micro	API	Undisclosed	128K	Text	Proprietary (AWS)	2024-12-03	AWS-native low-latency text tasks; Lambda AI apps
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device)	API	~3B	4K	Text	Proprietary	2024-06-10	On-device Apple Intelligence; Privacy-preserving
OpenELM — Apple (OpenELM) [family page]							
OpenELM	OW	3B	–	Text	See source	2024-04-24	layer-wise scaling; on-device
Aya — Cohere (Aya) [family page]							
Aya Expanse-8B	OW	8B	8K	Text	See source	2024-10-22	Improved multilingual performance; Data arbitrage and synthetic data strategies
Aya 23-8B	OW	8B	8K	Text	See source	2024-05-23	23 language support; Multilingual instruction following
Cohere Command — Cohere (Command A (Mar 2025)) [family page]							
Command R7B	Both	7B	128K	Text	Apache 2.0	2024-12	RAG; multilingual enterprise tasks

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
TinyLlama — Community (TinyLlama-1.1B (2024)) [family page]							
TinyLlama-1.1B-Chat	OW	1.1B	2K	Text	Apache 2.0	2024	ultra-light on-device chat; prototyping
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-VL-7B	OW	7B	4K	Text+Vision	See source	2024-03-13	Vision-language model; Hybrid vision encoder
DeepSeek-Math-7B	OW	7B	4K	Text	See source	2024-02-05	Math-specialized; Group Relative Policy Optimization (GRPO)
Gemma — Google DeepMind (Gemma) [family page]							
Gemma 2-2B	OW	2B	8K	Text	See source	2024-07-31	Knowledge distillation from larger model; Sliding window attention
Gemma 2-9B	OW	9B	–	Text	See source	2024-06-27	improved efficiency; knowledge distillation
PaliGemma	OW	3B	8K	Text+Vision	See source	2024-05-14	Vision-language model; SigLIP vision encoder + Gemma 2B
RecurrentGemma-2B	OW	2B	8K	Text	See source	2024-04-09	Griffin architecture with linear recurrence; Efficient inference
CodeGemma-7B	OW	7B	–	Text	See source	2024-04-09	code generation; code completion
Gemma-2B	OW	2B	–	Text	See source	2024-02-21	legacy
Gemma-7B	OW	7B	–	Text	See source	2024-02-21	built from Gemini technology; efficient
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM-135M	OW	135M	2K	Text	See source	2024-07-17	Ultra-small LM; On-device capable
SmolLM	OW	1.7B	–	Text	See source	2024-07-16	tiny but capable; on-device
SmolLM2-135M	OW	135M	2K	Text	Apache 2.0	2024	ultra-micro inference; browser/WASM
SmolLM2-360M-Instruct	OW	360M	8K	Text	Apache 2.0	2024	on-device/browser inference
SmolLM2-1.7B-Instruct	OW	1.7B	8K	Text	Apache 2.0	2024	efficient on-device chat; coding assist
LLaMA — Meta AI (LLaMA) [family page]							
Llama-3-8B	OW	8B	–	Text	See source	2024-04-18	improved tokenizer; grouped query attention
Llama 3.1 — Meta AI (Llama 3.1 (Jul 2024)) [family page]							
Llama 3.1-8B-Instruct	OW	8B	128K	Text	Llama 3.1 Community License	2024-07-23	most fine-tuned SLM base globally; RAG
Llama 3.2 — Meta AI (Llama 3.2 (Sep 2024)) [family page]							
Llama 3.2-1B-Instruct	OW	1B	128K	Text	Llama 3.2 Community License	2024-09-25	ultra-small on-device; Android/iOS apps
Llama 3.2-3B-Instruct	OW	3B	128K	Text	Llama 3.2 Community License	2024-09-25	mobile AI; default on-device Qualcomm
Llama 3.2-11B-Vision-Instruct	OW	11B	128K	Text+Vision	Llama 3.2 Community License	2024-09-25	vision-language tasks; document understanding

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
BitNet — Microsoft (BitNet) [family page]							
BitNet b1.58	OW	3B	–	Text	See source	2024-02-27	1-bit LLM (ternary weights: -1, 0, 1); Extreme efficiency
Phi-3 — Microsoft (Phi-3.5 (Aug 2024)) [family page]							
Phi-3.5-mini-instruct	OW	3.8B	128K	Text	MIT	2024-08	multilingual instruction following
Phi-3-mini-128k-instruct	OW	3.8B	128K	Text	MIT	2024-04-23	on-device chat; instruction following
Phi-3-small-128k-instruct	OW	7B	128K	Text	MIT	2024-04-23	balanced reasoning & coding
Phi-3-medium-128k-instruct	OW	14B	128K	Text	MIT	2024-04-23	on-premise deployment; complex reasoning
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4	OW	14B	16K	Text	MIT	2024-12-12	STEM reasoning; math
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral-8B-Instruct-2410	OW	8B	128K	Text	Apache 2.0	2024-10	edge-friendly assistant; function calling
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral NeMo 12B	OW	12B	128K	Text	Apache 2.0	2024-07	enterprise RAG; long-document summarization
Pixtral — Mistral AI (Pixtral) [family page]							
Pixtral-12B	OW	12B	–	Text+Image	See source	2024-09-17	vision-language; variable image resolution
Nemotron — NVIDIA (Nemotron) [family page]							
Minitron-8B	OW	8B	8K	Text	See source	2024-07-17	Width and depth pruning; Knowledge distillation
Nemotron-4-15B	OW	15B	8K	Text	See source	2024-03-14	High-quality training data; Strong downstream task performance
Hermes — Nous Research (Hermes) [family page]							
Nous Hermes 2 Mixtral	OW	46.7B (12.9B act.)	32K	Text	See source	2024-01-17	Fine-tuned Mixtral-8x7B; Function calling
GPT — OpenAI (GPT) [family page]							
GPT-4o mini	API	undisclosed	–	Text+Vision	Proprietary	2024-07-18	cost-efficient; fast
Reka — Reka AI (Reka) [family page]							
Reka Edge	API	7B	–	Text+Vision	Proprietary	2024-02-12	compact; multimodal
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM2.5-7B	OW	7B	–	Text	See source	2024-07-03	1M context; tool use
InternLM2.5-20B	OW	20B	1M	Text	See source	2024-07-03	1M context with LMDeploy; Strong reasoning
InternLM2-20B	OW	20B	–	Text	See source	2024-01-17	200K context; tool use
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM 2 12B	OW	12B	4K	Text	See source	2024-02-20	Multilingual (7 languages); Strong code capability

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
StableLM 2 1.6B	OW	1.6B	4K	Text	Stability AI License	2024-01-19	lightweight creative writing; basic chat
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon 3	OW	10B	–	Text+Audio+Video+Image	See source	2024-12-12	multimodal; runs on laptops
Falcon Mamba-7B	OW	7B	–	Text	See source	2024-08-12	SSM architecture; constant memory usage
Falcon 2-11B	OW	11B	–	Text	See source	2024-05-14	multimodal variant; vision-to-language
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
GLM-4-9B	OW	9B	–	Text	See source	2024-06-06	128K-1M context; tool use
CogVLM2-19B	OW	19B	8K	Text+Video+Image	See source	2024-05-24	Visual understanding; Video understanding
2023 and earlier							
Yi — 01.AI (Yi) [family page]							
Yi-6B	OW	6B	–	Text	See source	2023-11-02	bilingual; strong reasoning
Fuyu — Adept AI (Fuyu) [family page]							
Fuyu-8B	OW	8B	–	Text+Vision	See source	2023-10-17	Simplified multimodal architecture; No separate vision encoder
Qwen — Alibaba (Qwen) [family page]							
Qwen-Audio	OW	7B	–	Text+Audio	See source	2023-11-30	Audio understanding; Multi-task audio processing
Qwen-14B	OW	14B	–	Text	See source	2023-09-25	legacy
Qwen-VL	OW	9.6B	–	Text+Image	See source	2023-08-24	vision-language; image understanding
Qwen-7B	OW	7B	–	Text	See source	2023-08-03	bilingual; tool use
Baichuan — Baichuan Inc. (Baichuan) [family page]							
Baichuan-2-13B	OW	13B	–	Text	See source	2023-09-06	legacy
Baichuan-13B	OW	13B	–	Text	See source	2023-07-11	legacy
Baichuan-7B	OW	7B	–	Text	See source	2023-06-15	bilingual; strong Chinese performance
Cerebras-GPT — Cerebras Systems (Cerebras-GPT) [family page]							
Cerebras-GPT-13B	OW	13B	2K	Text	See source	2023-03-28	Chinchilla compute-optimal training; Trained on Cerebras CS-2 hardware
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-Coder-1.3B	OW	1.3B	–	Text	See source	2023-11-02	compact code model; fill-in-the-middle
DeepSeek-Coder-6.7B	OW	6.7B	–	Text	See source	2023-11-02	code generation; fill-in-the-middle
Pythia — EleutherAI (Pythia) [family page]							
Pythia-12B	OW	12B	2K	Text	See source	2023-04-03	154 intermediate checkpoints; Reproducible training

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
T5 — Google (T5) [family page]							
Flan-T5	OW	11B	–	Text	See source	2022-10-20	instruction tuned; encoder-decoder
Gemini — Google DeepMind (Gemini) [family page]							
Gemini 1.0 Nano	API	3.25B	–	Text	Proprietary	2023-12-06	on-device; efficient
StarCoder — Hugging Face / BigCode (StarCoder) [family page]							
StarCoder	OW	15.5B	–	Text	See source	2023-05-04	code generation; 80+ programming languages
Jais — Inception / Core42 / Cerebras (Jais) [family page]							
Jais-13B	OW	13B	2K	Text	See source	2023-08-30	Best Arabic LLM at launch; Bilingual Arabic-English
Vicuna — LMSYS (Vicuna) [family page]							
Vicuna-7B v1.5	OW	7B	–	Text	See source	2023-07-29	fine-tuned on ShareGPT; chat assistant
Vicuna-13B	OW	13B	–	Text	See source	2023-03-30	fine-tuned on ShareGPT data; chat assistant
Code Llama — Meta AI (Code Llama) [family page]							
Code Llama-7B	OW	7B	–	Text	See source	2023-08-24	code generation; infilling
LLaMA — Meta AI (LLaMA) [family page]							
Llama-2-7B	OW	7B	–	Text	See source	2023-07-18	open weights; commercial use allowed
Llama-2-13B	OW	13B	–	Text	See source	2023-07-18	legacy
LLaMA-7B	OW	7B	–	Text	Research only	2023-02-24	efficient training; strong few-shot performance
LLaMA-13B	OW	13B	–	Text	Research only	2023-02-24	legacy
Phi — Microsoft (Phi) [family page]							
Phi-2	OW	2.7B	–	Text	See source	2023-12-13	language comprehension; reasoning
Phi-1.5	OW	1.3B	–	Text	See source	2023-09-11	reasoning; language understanding
Phi-1	OW	1.3B	–	Text	See source	2023-06-20	Python coding; textbook-quality data
Mistral — Mistral AI (Mistral) [family page]							
Mistral Tiny (Mistral-7B-Instruct)	API	7B	32K	Text	Proprietary	2023-12-11	API-served Mistral 7B Instruct; Cost-effective API option
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral 7B v0.3	OW	7.3B	32K	Text	Apache 2.0	2023-09-27	general chat; RAG
MPT — MosaicML (Databricks) (MPT) [family page]							
MPT-7B	OW	7B	–	Text	See source	2023-05-05	ALiBi positional encoding; long context
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM-20B	OW	20B	–	Text	See source	2023-09-20	legacy
InternLM-7B	OW	7B	–	Text	See source	2023-07-06	bilingual; long context
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM-Zephyr-3B	OW	3B	–	Text	See source	2023-11-29	chat assistant; compact

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
StableLM-Alpha	OW	7B	–	Text	See source	2023-04-19	open-source; conversation
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon-7B	OW	7B	–	Text	See source	2023-06-05	RefinedWeb dataset; high-quality pretraining
RedPajama — Together AI (RedPajama) [family page]							
RedPajama-INCITE-7B	OW	7B	2K	Text	See source	2023-05-05	Fully open training data; Reproducible
SOLAR — Upstage (SOLAR) [family page]							
SOLAR-10.7B	OW	10.7B	4K	Text	See source	2023-12-13	Depth Up-Scaling (DUS); Merged and scaled from Llama 2 architecture
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
ChatGLM3-6B	OW	6B	–	Text	See source	2023-10-27	tool use; code interpreter
ChatGLM2-6B	OW	6B	–	Text	See source	2023-06-25	legacy
ChatGLM-6B	OW	6B	–	Text	See source	2023-03-14	bilingual; quantized for consumer GPUs

B.5 Monitoring Sources and Maintenance

Table 18. High-value monitoring sources retained from the inventory files.

Source	Use
Hugging Face model hub and leaderboards	New releases, model cards, and benchmark submissions
Artificial Analysis	Context windows, pricing, and comparative capability snapshots
LLM Stats	Consolidated model metadata and API comparisons
LMSYS / Chatbot Arena	Human preference rankings
BentoML and vendor engineering blogs	Practical deployment write-ups and model updates
Official vendor changelogs	Release confirmation for OpenAI, Anthropic, Google, Meta, Mistral, Alibaba, and DeepSeek

Recommended maintenance queries from the inventory drafts included combinations such as "best small language model [year]", "new open-weight reasoning model", and vendor-specific searches around context window, model card, and release notes. In practice, the highest-signal workflow is to combine vendor documentation with model cards and a benchmark tracker rather than relying on any single static list.

References

1. OpenAI. GPT-4 Technical Report. arXiv:2303.08774. 2023. URL: https://arxiv.org/abs/2303.08774.
2. Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. Anthropic technical report. 2024. URL: https://www.anthropic.com/news/claude-3-family.
3. Gemini Team, Google. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805. 2023. URL: https://arxiv.org/abs/2312.11805.
4. Llama Team, AI @ Meta. The Llama 3 Herd of Models. arXiv:2407.21783. 2024. URL: https://arxiv.org/abs/2407.21783.
5. Brown TB, Mann B, Ryder N, et al. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems 2020;33:1877–901.
6. Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models. arXiv:2001.08361. 2020. URL: https://arxiv.org/abs/2001.08361.
7. Hoffmann J, Borgeaud S, Mensch A, et al. Training Compute-Optimal Large Language Models. In: *Advances in Neural Information Processing Systems*. Vol. 35. New Orleans, Louisiana, USA: Curran Associates, Inc., 2022. URL: https://arxiv.org/abs/2203.15556.
8. Patterson D, Gonzalez J, Le Q, et al. Carbon Emissions and Large Neural Network Training. arXiv:2104.10350. 2021. URL: https://arxiv.org/abs/2104.10350.
9. Vries A de. The Growing Energy Footprint of Artificial Intelligence. Joule 2023;7:2191–4.
10. Sanh V, Debut L, Chaumond J, and Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. 2019. URL: https://arxiv.org/abs/1910.01108.
11. Jiao X, Yin Y, Shang L, Jiang X, Chen X, Li L, Wang F, and Liu Q. TinyBERT: Distilling BERT for Natural Language Understanding. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, 2020:4163–74. DOI: 10.18653/v1/2020.findings-emnlp.372. URL: https://aclanthology.org/2020.findings-emnlp.372/.
12. Abdin M, Aneja J, Awadalla H, et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219. 2024. URL: https://arxiv.org/abs/2404.14219.
13. Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295. 2024. URL: https://arxiv.org/abs/2403.08295.
14. Qwen Team, Alibaba Group. Qwen2.5 Technical Report. arXiv:2412.15115. 2024. URL: https://arxiv.org/abs/2412.15115.
15. MIT Technology Review. 10 Breakthrough Technologies 2025. MIT Technology Review. 2025. URL: https://www.technologyreview.com/2025/01/03/1108883/10-breakthrough-technologies-2025/.

16. Technology Innovation Institute. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448. 2025. URL: https://arxiv.org/abs/2507.22448.

17. Qwen Team, Alibaba Group. Qwen3 Technical Report. arXiv:2505.09388. 2025. URL: https://arxiv.org/abs/2505.09388.

18. Gemma Team, Google DeepMind. Gemma 4 Technical Report. arXiv:2607.02770. Google DeepMind; models released 2 April 2026 (E2B–31B, dense and MoE). 2026. URL: https://arxiv.org/abs/2607.02770.

19. Apple. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv:2507.13575. 2025. URL: https://arxiv.org/abs/2507.13575.

20. Belcak P, Heinrich G, Diao S, Fu Y, Dong X, Muralidharan S, Lin YC, and Molchanov P. Small Language Models are the Future of Agentic AI. arXiv:2506.02153. 2025. URL: https://arxiv.org/abs/2506.02153.

21. Chen S, Liu Y, Han W, Zhang W, and Liu T. A Survey on LLM-Based Multi-Agent System: Recent Advances and New Frontiers in Application. arXiv:2412.17481. 2024. URL: https://arxiv.org/abs/2412.17481.

22. Chen L and Varoquaux G. What is the Role of Small Models in the LLM Era: A Survey. arXiv:2409.06857. 2024. URL: https://arxiv.org/abs/2409.06857.

23. Chen Y, Zhao J, and Han H. A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv:2505.07460. 2025. URL: https://arxiv.org/abs/2505.07460.

24. Lu Z, Li X, Cai D, Yi R, Liu F, Zhang X, Lane ND, and Xu M. Small Language Models: Survey, Measurements, and Insights. arXiv:2409.15790. 2024. URL: https://arxiv.org/abs/2409.15790.

25. Nguyen CV, Shen X, Aponte R, Xia Y, Basu S, Hu Z, Chen J, Parmar M, Kunapuli S, Barrow J, Wu J, Singh A, Wang Y, Gu J, Dernoncourt F, Ahmed NK, Lipka N, Zhang R, Chen X, Yu T, Kim S, Deilamsalehy H, Park N, Rimer M, Zhang Z, Yang H, Rossi RA, and Nguyen TH. A Survey of Small Language Models. arXiv:2410.20011. 2024. URL: https://arxiv.org/abs/2410.20011.

26. Wang F, Zhang Z, Zhang X, Wu Z, Mo T, Lu Q, Wang W, Li R, Xu J, Tang X, He Q, Ma Y, Huang M, and Wang S. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. arXiv:2411.03350. 2024. URL: https://arxiv.org/abs/2411.03350.

27. Srivastava G, Cao S, and Wang X. Towards Reasoning Ability of Small Language Models. arXiv:2502.11569. 2025. URL: https://arxiv.org/abs/2502.11569.

28. Zhou Z, Kurz S, and Zhao Z. Revisiting Pruning vs Quantization for Small Language Models. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, 2025:12055–70. DOI: 10.18653/v1/2025.findings-emnlp.645. URL: https://aclanthology.org/2025.findings-emnlp.645/.

29. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, and Polosukhin I. Attention is All You Need. In: *Advances in Neural Information Processing Systems*. Vol. 30. Long Beach, California, USA: Curran Associates, Inc., 2017:5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

30. Devlin J, Chang MW, Lee K, and Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019:4171–86. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

31. Radford A, Wu J, Child R, Luan D, Amodei D, and Sutskever I. Language Models are Unsupervised Multitask Learners. OpenAI technical report. 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

32. Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways. Journal of Machine Learning Research 2023;24:1–113.

33. Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, Rodriguez A, Joulin A, Grave E, and Lample G. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971. 2023. URL: https://arxiv.org/abs/2302.13971.

34. Touvron H, Martin L, Stone K, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. 2023. URL: https://arxiv.org/abs/2307.09288.

35. Almazrouei E, Alobeidli H, Alshamsi A, et al. The Falcon Series of Open Language Models. arXiv:2311.16867. 2023. URL: https://arxiv.org/abs/2311.16867.
36. Gunasekar S, Zhang Y, Aneja J, Mendes CCT, Giorno AD, Gopi S, Javaheripi M, Kauffmann P, Rosa G de, Saarikivi O, et al. Textbooks Are All You Need. arXiv:2306.11644. 2023. URL: https://arxiv.org/abs/2306.11644.
37. Microsoft Research. Phi-2: The Surprising Power of Small Language Models. Microsoft Research Blog. 2023. URL: https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/.
38. Gemma Team, Google DeepMind. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. 2024. URL: https://arxiv.org/abs/2408.00118.
39. Microsoft Research. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv:2503.01743. 2025. URL: https://arxiv.org/abs/2503.01743.
40. Hinton G, Vinyals O, and Dean J. Distilling the Knowledge in a Neural Network. arXiv:1503.02531. 2015. URL: https://arxiv.org/abs/1503.02531.
41. Gemma Team, Google DeepMind. Gemma 3 Technical Report. arXiv:2503.19786. 2025. URL: https://arxiv.org/abs/2503.19786.
42. Romero A, Ballas N, Kahou SE, Chassang A, Gatta C, and Bengio Y. FitNets: Hints for Thin Deep Nets. arXiv:1412.6550. 2015. URL: https://arxiv.org/abs/1412.6550.
43. Mitra A, Del Corro L, et al. Orca 2: Teaching Small Language Models How to Reason. arXiv:2311.11045. 2023. URL: https://arxiv.org/abs/2311.11045.
44. Abdin M et al. Phi-4 Technical Report. arXiv:2412.08905. 2024. URL: https://arxiv.org/abs/2412.08905.
45. Frantar E, Ashkboos S, Hoefler T, and Alistarh D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In: *11th International Conference on Learning Representations (ICLR)*. Kigali, Rwanda: OpenReview.net, 2023. URL: https://openreview.net/forum?id=tcbBPnfwxS.
46. Lin J, Tang J, Tang H, Yang S, Chen WM, Wang WC, Xiao G, Dang X, Gan C, and Han S. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In: *Proceedings of Machine Learning and Systems*. Vol. 6. Santa Clara, California, USA: MLSys, 2024. URL: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.
47. Gerganov G. llama.cpp. GitHub repository: https://github.com/ggerganov/llama.cpp. 2023.
48. Kim S, Hooper C, et al. SqueezeLLM: Dense-and-Sparse Quantization. arXiv:2306.07629. 2024. URL: https://arxiv.org/abs/2306.07629.
49. Frantar E and Alistarh D. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. Honolulu, Hawaii, USA: PMLR, 2023:10323–37. URL: https://proceedings.mlr.press/v202/frantar23a.html.
50. Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, and Chen W. LoRA: Low-Rank Adaptation of Large Language Models. In: *10th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.
51. Dettmers T, Pagnoni A, Holtzman A, and Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs. In: *Advances in Neural Information Processing Systems*. Vol. 36. New Orleans, Louisiana, USA: Curran Associates, Inc., 2023. URL: https://arxiv.org/abs/2305.14314.
52. Abdin M et al. Phi-3.5: Small Language Models with Extended Context and Multilingual Support. Microsoft technical report. 2024. URL: https://huggingface.co/microsoft/Phi-3.5-mini-instruct.
53. Abdin M et al. Phi-4-Reasoning Technical Report. arXiv:2504.21318. 2025. URL: https://arxiv.org/abs/2504.21318.
54. Yang A, Yang B, Hui B, et al. Qwen2 Technical Report. arXiv:2407.10671. 2024. URL: https://arxiv.org/abs/2407.10671.
55. Zhang P, Zeng G, Wang T, and Lu W. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385. 2024. URL: https://arxiv.org/abs/2401.02385.

56. Jiang AQ, Sablayrolles A, Mensch A, et al. Mistral 7B. arXiv:2310.06825. 2023. URL: https://arxiv.org/abs/2310.06825.

57. Allal LB, Lozhkov A, Penedo G, et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737. 2025. URL: https://arxiv.org/abs/2502.02737.

58. Singer P, Gissin D, et al. H2O-Danube-1.8B Technical Report. arXiv:2401.16818. 2024. URL: https://arxiv.org/abs/2401.16818.

59. AI2 Team. OLMo 2: Open Language Model. arXiv:2501.00656. 2025. URL: https://arxiv.org/abs/2501.00656.

60. Stability AI. StableLM 2 1.6B Technical Report. arXiv:2402.17834. 2024. URL: https://arxiv.org/abs/2402.17834.

61. BigScience Workshop. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv:2211.05100. 2022. URL: https://arxiv.org/abs/2211.05100.

62. Nguyen T and Nguyen T. An Evaluation of LLMs Inference on Popular Single-board Computers. arXiv:2511.07425. 2025. URL: https://arxiv.org/abs/2511.07425.

63. Meta AI. ExecuTorch: End-to-End Solution for Enabling On-Device AI. PyTorch Blog. 2024. URL: https://pytorch.org/executorch/.

64. MLC Team. MLC LLM: Universal LLM Deployment Engine. GitHub repository: https://github.com/mlc-ai/mlc-llm. 2024.

65. Microsoft. ONNX Runtime. GitHub repository: https://github.com/microsoft/onnxruntime. 2019.

66. MLC Team. WebLLM: High-Performance In-Browser LLM Inference Engine. GitHub repository. 2024. URL: https://arxiv.org/abs/2412.15803.

67. Google Developers. LiteRT with QNN Acceleration. Google AI Blog. 2025. URL: https://ai.google.dev/edge/litert.

68. Xu J, Liu C, Tan M, et al. On-Device Language Models: A Comprehensive Review. arXiv:2409.00088. 2024. URL: https://arxiv.org/abs/2409.00088.

69. Ong I, Almahairi A, Wu V, Chiang WL, Wu T, Gonzalez JE, Kadous MW, and Stoica I. RouteLLM: Learning to Route LLMs with Preference Data. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: https://arxiv.org/abs/2406.18665.

70. Shao C, Hu X, Lin Y, and Xu F. Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. arXiv:2502.04392. 2025. URL: https://arxiv.org/abs/2502.04392.

71. Zhang J, Lan T, Zhu M, Liu Z, Hoang T, Kokane S, Yao W, Tan J, Prabhakar A, Chen H, Liu Z, Feng Y, Awalgaonkar T, Murthy R, Hu E, Chen Z, Xu R, Niebles JC, Heinecke S, Wang H, Savarese S, and Xiong C. xLAM: A Family of Large Action Models to Empower AI Agent Systems. arXiv:2409.03215. 2024. URL: https://arxiv.org/abs/2409.03215.

72. Feng S, Fang G, Ma X, and Wang X. Efficient Reasoning Models: A Survey. arXiv:2504.10903. 2025. URL: https://arxiv.org/abs/2504.10903.

73. Bakouch E, Bakouch L, Lozhkov A, Werra L von, Wolf T, et al. SmolLM3: smol, Multilingual, Long-Context Reasoner. Hugging Face blog, https://huggingface.co/blog/smollm3. 2025.

74. Shazeer N, Mirhoseini A, Maziarz K, Davis A, Le Q, Hinton G, and Dean J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In: *5th International Conference on Learning Representations (ICLR)*. Toulon, France: OpenReview.net, 2017. URL: https://openreview.net/forum?id=B1ckMDqlg.

75. Fedus W, Zoph B, and Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. Journal of Machine Learning Research 2022;23:1–39.

76. Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, Chaplot DS, Casas D de las, Hanna EB, Bressand F, et al. Mixtral of Experts. arXiv:2401.04088. 2024. URL: https://arxiv.org/abs/2401.04088.

77. Dai D, Deng C, Zhao C, Xu RX, Gao H, Chen D, Li J, Zeng W, Yu X, Wu Y, et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066. 2024. URL: https://arxiv.org/abs/2401.06066.

78. Meta AI. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/. 2025.

79. Gu A and Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. 2023. URL: https://arxiv.org/abs/2312.00752.

80. Lieber O, Lenz B, Bata H, Cohen G, Osin J, Dalmedigos I, Safahi E, Meirom S, Belinkov Y, Shalev-Shwartz S, et al. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887. 2024. URL: https://arxiv.org/abs/2403.19887.

81. IBM Granite Team. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models for Enterprise. https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models. 2025.

82. NVIDIA. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. arXiv:2508.14444. 2025. URL: https://arxiv.org/abs/2508.14444.

83. Qwen Team, Alibaba Group. Qwen3.5 Small Models for On-Device Applications. https://qwen.ai/blog?id=qwen3.5. Alibaba; 0.8B–9B dense multimodal, Gated DeltaNet hybrid, released 2 March 2026. 2026.

84. Li Y, Bubeck S, Eldan R, Giorno AD, Gunasekar S, and Lee YT. Textbooks Are All You Need II: phi-1.5 technical report. arXiv:2309.05463. 2023. URL: https://arxiv.org/abs/2309.05463.

85. Eldan R and Li Y. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759. 2023. URL: https://arxiv.org/abs/2305.07759.

86. Mukherjee S, Mitra A, Jawahar G, Agarwal S, Palangi H, and Awadallah A. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707. 2023. URL: https://arxiv.org/abs/2306.02707.

87. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. 2025. URL: https://arxiv.org/abs/2501.12948.

88. Xu H, Peng B, Awadalla H, Chen D, Chen YC, Gao M, Kim YJ, Li Y, Ren L, Shen Y, Wang S, Xu W, Gao J, and Chen W. Phi-4-Mini-Reasoning: Exploring the Limits of Small Reasoning Language Models in Math. arXiv:2504.21233. 2025. URL: https://arxiv.org/abs/2504.21233.

89. Sreenivas ST, Muralidharan S, Joshi R, Chochowski M, Patwary M, Shoeybi M, Catanzaro B, Kautz J, and Molchanov P. Minitron: Efficient LLM Compression via Pruning and Knowledge Distillation. arXiv:2407.14679. 2024. URL: https://arxiv.org/abs/2407.14679.

90. Beyer L, Steiner A, Pinto AS, Kolesnikov A, Wang X, and Zhai X. PaliGemma: A Versatile 3B VLM for Transfer. arXiv:2407.07726. 2024. URL: https://arxiv.org/abs/2407.07726.

91. Google. Gemma 3n: On-Device Multimodal Models. Google AI Blog. 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/.

92. Kim H, Hwang H, Lee J, Park S, Kim D, Lee T, Yoon C, Sohn J, Park J, Reykhart O, Fetherston T, Choi D, Kwak SH, Chen Q, and Kang J. Small Language Models Learn Enhanced Reasoning Skills from Medical Textbooks. npj Digital Medicine 2025;8:240.

93. Hasan MJ, Rahman F, and Mohammed N. OptimCLM: Optimizing Clinical Language Models for Predicting Patient Outcomes via Knowledge Distillation, Pruning, and Quantization. International Journal of Medical Informatics 2025;195:105764.

94. Rozière B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T, et al. Code Llama: Open Foundation Models for Code. arXiv:2308.12950. 2023. URL: https://arxiv.org/abs/2308.12950.

95. Lozhkov A, Li R, Allal LB, Cassano F, Lamy-Poirier J, Tazi N, Tang A, Pykhtar D, Liu J, Wei Y, et al. StarCoder2 and The Stack v2: The Next Generation. arXiv:2402.19173. 2024. URL: https://arxiv.org/abs/2402.19173.

96. Liu Z et al. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196. 2024. URL: https://arxiv.org/abs/2401.14196.

97. Üstün A, Aryabumi V, Yong ZX, Ko WY, D'souza D, Onilude G, Bhandari N, Singh S, Ooi HL, Kayid A, et al. Aya Model: An Instruction Finetuned Open-Access Multilingual Language Model. arXiv:2402.07827. 2024. URL: https://arxiv.org/abs/2402.07827.

98. Aryabumi V, Dang J, Talupuru D, Dash S, Cairuz D, Lin H, Venkitesh B, Smith M, Campos JA, Tan YC, Marchisio K, Bartolo M, Ruder S, Locatelli A, Kreutzer J, Frosst N, Gomez A, Blunsom P, Fadaee M, Üstün A, and Hooker S. Aya 23: Open Weight Releases to Further Multilingual Progress. arXiv:2405.15032. 2024. URL: https://arxiv.org/abs/2405.15032.

99. Wang J, Zeng Y, Guo J, Ma Y, Liu A, and Liu X. SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment. arXiv:2511.13023. 2025. URL: https://arxiv.org/abs/2511.13023.

100. Blodgett SL, Barocas S, Daumé III H, and Wallach H. Language (Technology) is Power: A Critical Survey of "Bias" in NLP. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, 2020:5454–76. DOI: 10.18653/v1/2020.acl-main.485. URL: https://aclanthology.org/2020.acl-main.485/.

101. Yang C, Lu W, Zhu Y, Wang Y, Chen Q, Gao C, Yan B, and Chen Y. Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application. arXiv:2407.01885. 2024. URL: https://arxiv.org/abs/2407.01885.

102. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, and Zhou D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*. Vol. 35. New Orleans, Louisiana, USA: Curran Associates, Inc., 2022. URL: https://arxiv.org/abs/2201.11903.

103. Feng T, Li Y, Chenglin L, Chen H, Yu F, and Zhang Y. Teaching Small Language Models Reasoning through Counterfactual Distillation. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, 2024:5831–42. DOI: 10.18653/v1/2024.emnlp-main.333. URL: https://aclanthology.org/2024.emnlp-main.333/.

104. Liao R, Li Q, Liu X, and Zhang Y. Neural-Symbolic Collaborative Distillation: Advancing Small Language Models for Complex Reasoning Tasks. arXiv:2409.13203. 2024. URL: https://arxiv.org/abs/2409.13203.

105. Xu X, Li M, Tao C, Shen T, Cheng R, Li J, Xu C, Tao D, and Zhou T. A Survey on Knowledge Distillation of Large Language Models. arXiv:2402.13116. 2024. URL: https://arxiv.org/abs/2402.13116.

106. Ballout M, Tresp V, Divo F, and Kersting K. Efficient Knowledge Distillation: Empowering Small Language Models with Teacher Model Insights. arXiv:2409.12586. 2024. URL: https://arxiv.org/abs/2409.12586.

107. Fu Y, Shi Q, and Yin J. Dynamic Self-Distillation via Previous Mini-batches for Fine-tuning Small Language Models. arXiv:2411.16991. 2024. URL: https://arxiv.org/abs/2411.16991.

108. Sun M, Liu Z, Bair A, and Kolter JZ. A Simple and Effective Pruning Approach for Large Language Models. In: *12th International Conference on Learning Representations (ICLR)*. Vienna, Austria: OpenReview.net, 2024. URL: https://openreview.net/forum?id=PxoFut3dWW.

109. Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, and Steinhardt J. Measuring Massive Multitask Language Understanding. In: *9th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2021. URL: https://openreview.net/forum?id=d7KBjmI3GmQ.

110. Clark P, Cowhey I, Etzioni O, Khot T, Sabharwal A, Schoenick C, and Tafjord O. Think You Have Solved Question Answering? Try ARC. arXiv:1803.05457. 2018. URL: https://arxiv.org/abs/1803.05457.

111. Zellers R, Holtzman A, Bisk Y, Farhadi A, and Choi Y. HellaSwag: Can a Machine Really Finish Your Sentence? In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019:4791–800. DOI: 10.18653/v1/P19-1472. URL: https://aclanthology.org/P19-1472/.

112. Cobbe K, Kosaraju V, Bavarian M, Chen M, Jun H, Kaiser L, Plappert M, Tworek J, Hilton J, Nakano R, Hesse C, and Schulman J. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168. 2021. URL: https://arxiv.org/abs/2110.14168.

113. Hendrycks D, Burns C, Kadavath S, Arora A, Basart S, Tang E, Song D, and Steinhardt J. Measuring Mathematical Problem Solving With the MATH Dataset. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 34. Curran Associates, Inc., 2021. URL: https://arxiv.org/abs/2103.03874.

114. Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. 2021. URL: https://arxiv.org/abs/2107.03374.

115. Austin J, Odena A, Nye M, et al. Program Synthesis with Large Language Models. arXiv:2108.07732. 2021. URL: https://arxiv.org/abs/2108.07732.

116. Joshi M, Choi E, Weld DS, and Zettlemoyer L. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, 2017:1601–11. DOI: 10.18653/v1/P17-1147. URL: https://aclanthology.org/P17-1147/.

117. Rajpurkar P, Jia R, and Liang P. Know What You Don't Know: Unanswerable Questions for SQuAD. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Melbourne, Australia: Association for Computational Linguistics, 2018:784–9. DOI: 10.18653/v1/P18-2124. URL: https://aclanthology.org/P18-2124/.

118. Williams A, Nangia N, and Bowman SR. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, 2018:1112–22. DOI: 10.18653/v1/N18-1101. URL: https://aclanthology.org/N18-1101/.

119. Socher R, Perelygin A, Wu J, Chuang J, Manning CD, Ng A, and Potts C. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In: *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Seattle, Washington, USA: Association for Computational Linguistics, 2013:1631–42. DOI: 10.18653/v1/D13-1170. URL: https://aclanthology.org/D13-1170/.

120. Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, Lin Z, Li Z, Li D, Xing EP, Zhang H, Gonzalez JE, and Stoica I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 36. New Orleans, Louisiana, USA: Curran Associates, Inc., 2023. URL: https://arxiv.org/abs/2306.05685.

121. Gao L, Tow J, Abbasi B, et al. A Framework for Few-Shot Language Model Evaluation. Zenodo. 2023. URL: https://github.com/EleutherAI/lm-evaluation-harness.

122. Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, et al. Holistic Evaluation of Language Models. Transactions on Machine Learning Research 2023.

123. Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez JE, Zhang H, and Stoica I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz, Germany: Association for Computing Machinery, 2023:611–26. DOI: 10.1145/3600006.3613165. URL: https://doi.org/10.1145/3600006.3613165.

124. Qi X, Zeng Y, Xie T, Chen PY, Jia R, Mittal P, and Henderson P. Fine-tuning Aligned Language Models Compromises Safety. arXiv:2310.03693. 2023. URL: https://arxiv.org/abs/2310.03693.

125. Apple Inc. Apple Intelligence Foundation Language Models. Tech. rep. Apple Machine Learning Research, 2024. URL: https://arxiv.org/abs/2407.21075.

126. Apple Machine Learning Research. Introducing the Third Generation of Apple's Foundation Models. https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models. 2026.

127. Samsung Research. Samsung Gauss: Language Model for Everyday Life. Samsung AI report. 2023. URL: https://arxiv.org/abs/2312.04913.

128. NVIDIA Corporation. TensorRT-LLM: Accelerating Large Language Model Inference. NVIDIA Developer documentation. 2024. URL: https://developer.nvidia.com/tensorrt.

129. Apple Machine Learning Research. MLX: An Array Framework for Apple Silicon. Software, GitHub repository. 2023. URL: https://github.com/ml-explore/mlx.

130. Willard BT and Louf R. Efficient Guided Generation for Large Language Models. arXiv:2307.09702. 2023. URL: https://arxiv.org/abs/2307.09702.

131. Patil SG, Mao H, Yan F, Ji CCJ, Suresh V, Stoica I, and Gonzalez JE. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. URL: https://proceedings.mlr.press/v267/patil25a.html.

132. Patil SG, Zhang T, Wang X, and Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334. 2023. URL: https://arxiv.org/abs/2305.15334.

133. Qin Y, Liang S, Ye Y, Zhu K, Yan L, Lu Y, Lin Y, Cong X, Tang X, Qian B, Zhao S, Hong L, Tian R, Xie R, Zhou J, Gerstein M, Li D, Liu Z, and Sun M. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. URL: https://arxiv.org/abs/2307.16789.

134. Li M, Zhao Y, Yu B, Song F, Li H, Yu H, Li Z, Huang F, and Li Y. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2023:3102–16. DOI: 10.18653/v1/2023.emnlp-main.187. URL: https://aclanthology.org/2023.emnlp-main.187/.

135. Zhou S, Xu FF, Zhu H, Zhou X, Lo R, Sridhar A, Cheng X, Ou T, Bisk Y, Fried D, Alon U, and Neubig G. WebArena: A Realistic Web Environment for Building Autonomous Agents. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. URL: https://arxiv.org/abs/2307.13854.

136. Mialon G, Fourrier C, Swift C, Wolf T, LeCun Y, and Scialom T. GAIA: A Benchmark for General AI Assistants. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. URL: https://arxiv.org/abs/2311.12983.

137. Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, and Narasimhan K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. URL: https://arxiv.org/abs/2310.06770.

138. Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, Gu Y, Ding H, Men K, Yang K, Zhang S, Deng X, Zeng A, Du Z, Zhang C, Shen S, Zhang T, Su Y, Sun H, Huang M, Dong Y, and Tang J. AgentBench: Evaluating LLMs as Agents. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. URL: https://arxiv.org/abs/2308.03688.

139. Yao S, Shinn N, Razavi P, and Narasimhan K. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. 2024. URL: https://arxiv.org/abs/2406.12045.

140. Lu Z, Li X, Cai D, Yi R, Liu F, Liu W, Luan J, Zhang X, Lane ND, and Xu M. Demystifying Small Language Models for Edge Deployment. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025:14747–64. DOI: 10.18653/v1/2025.acl-long.718. URL: https://aclanthology.org/2025.acl-long.718/.

141. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih Wt, Rocktäschel T, Riedel S, and Kiela D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020:9459–74. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

142. Mistral AI. Codestral: Hello, World! Mistral AI Blog. 2024. URL: https://mistral.ai/news/codestral.

143. Zhang Y, Li Y, Cui L, Cai D, Liu L, Fu T, Huang X, Zhao E, Zhang Y, Chen Y, et al. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. arXiv:2309.01219. 2023. URL: https://arxiv.org/abs/2309.01219.

144. Leviathan Y, Kalman M, and Matias Y. Fast Inference from Transformers via Speculative Decoding. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. Honolulu, Hawaii, USA: PMLR, 2023:19274–86. URL: https://proceedings.mlr.press/v202/leviathan23a.html.

145. Qualcomm Technologies. On-Device AI: Enabling the Next Generation of Mobile Intelligence. Qualcomm white paper. 2024. URL: https://www.qualcomm.com/research/artificial-intelligence/on-device-ai.

146. Strubell E, Ganesh A, and McCallum A. Energy and Policy Considerations for Deep Learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019:3645–50. DOI: 10.18653/v1/P19-1355. URL: https://aclanthology.org/P19-1355/.

147. Patterson D, Gonzalez J, Hölzle U, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, and Dean J. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. Computer 2022;55:18–28.

148. Ren L, Chen C, Xu H, Kim YJ, Atkinson A, Zhan Z, Sun J, Peng B, Liu L, Wang S, Cheng H, Gao J, Chen W, and Shen Y. Decoder-Hybrid-Decoder Architecture for Efficient Reasoning with Long Generation. arXiv:2507.06607. Introduces the SambaY architecture (Phi-4-mini-flash). 2025. URL: https://arxiv.org/abs/2507.06607.

149. Yang S, Kautz J, and Hatamizadeh A. Gated Delta Networks: Improving Mamba2 with Delta Rule. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: https://arxiv.org/abs/2412.06464.